KSI
KUMARAGURU
SCHOOL OF
INNOVATION

24CSI014-DESIGN AND ANALYSIS OF ALGORITHM

NISHANTH P – 24BAD405

2025-2026

LAB MANUAL

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

INDEX

Ex No	Date	Title of the Experiment	Marks
1.	17.01.2026	Implementation of Algorithms	30
2.	24.01.2026	Linear search and binary search	30
3.	02.02.2026	Recursive and non-recursive algorithms	29
4.	09.02.2026	Brute force string matching	30
5.	09.02.2026	Merge sort, Quick Sort and Binary search	30
6.	23.02.2026	Maximum subarray and Strassen's matrix multiplication	30
7.	23.02.2026	Fractional knapsack, activity selection and Huffman coding	30
8.	02.03.2026	Matrix chain multiplication and LCS	27
9.	16.03.2026	Optimal binary search tree and Floyd-Warshall's algorithm	30
10.	23.03.2026	N queens, Hamiltanion circuit, Map-coloring and Trave ling salesman problem	29
11	30.03.2026	Vertex Cover Problem	28

29

Average marks **Faculty Signature**

S.No	Date	Title	Mark	Pg no	Faculty Sign
1	12/1/26	Implementation of Algorithms		1	
2	29/1/26	Implementation of Linear and Binary Search	30	9	
3	02/02/26	Implementation and comparison of recursive and non-recursive algorithm	29	14	
4	09.02.26	Brute Force: string matching	30	17	
5	09.02.26	Divide and conquer Merge and Quick sort Decrease and Conquer Binary Search	30	20	
6	23.02.26	Divide & Conquer Maximum Subarray Problem Strassen's Matrix Multiplication	30	30	
7	23.02.26	Greedy strategy Fractional knapsack Activities selection Huffman coding	30	37	
8	2.3.26	Matrix chain multiplication Longest common subsequence	27	44	

Exno	Date	Title	Marks	Faculty Signature
9	16/03/26	Optimal Binary Search Trees All Pairs Shortest Paths Floyd-Warshall Algorithm	30	
10	23/03/26	Back Tracking • N-Queens Problem • Hamilton Circuit Problem • Graph Color Problem Branch & Bound Travelling Sales Man	29	
11	30/03/26	Vertex-Cover Problem [Approximation Algorithm]	28	
		Total	323	
		Average	29	

EX NO: 01 **17.01.2026**

IMPLEMENTATION OF ALGORITHMS

Ex No: 1

Implementation of Algorithms

Pre-Lab Questions

1 Define the Divide and Conquer strategy and what are the three standard steps involved in this paradigm?

Divide and Conquer is an algorithm design technique where a problem is divided into smaller subproblems, solved individually, and then combined to get final solution.

Divide - problem is divided into smaller subproblems.

Conquer - Each subproblem is sloved recursively

Combine - solution of the subproblems are combined to form the solution

2 Compare Merge Sort and Quick Sort in terms of their "Best-case" and "Worst-case" time complexities.

Merge Sort

Best Case: O(nlog n)

Worst Case: O(nlog n)

Merge Sort always divides the array into two equal halves and merges them in a sorted manner, so its time complexity remains the same in all cases.

Quick Sort

Best Case: $O(n \log n)$

Worst Case: $O(n^2)$

Quick Sort performs efficiently when the pivot divides the array evenly. The worst case occurs when the pivot as always the smallest or largest element, resulting in unbalanced partitions.

3. Why is stability important when two elements have the same frequency?

Stability ensures that elements with the same frequency maintain their original relative order after sorting, which is important when sorting data with multiple attributes.

4. What is an "Inversion" in an array? If an array is sorted in descending order, what is the total number of inversions for an array of size N?

An inversion is a pair (i, j) such that $i < j$ and $A[i] > A[j]$.

Descending order array inversions:

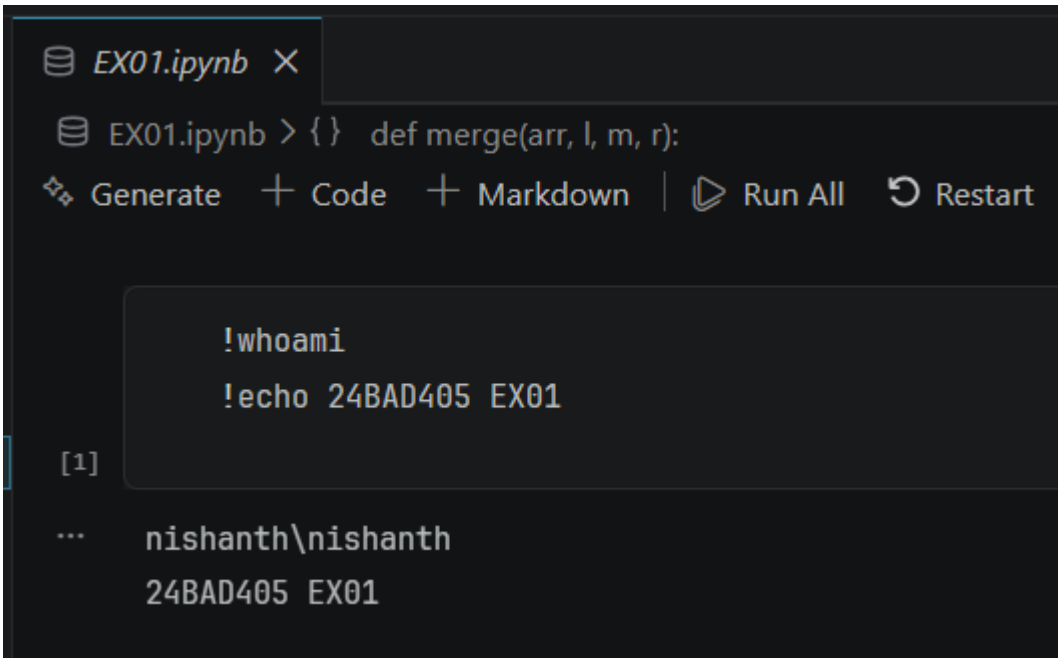
$$\frac{N(N-1)}{2}$$

5. How much extra space (Space Complexity) does Merge Sort require compared to Quick Sort?

Merge Sort: $O(n)$ extra space.

Quick Sort: $O(\log n)$ extra space

Code:


EX01.ipynb
EX01.ipynb > { } def merge(arr, l, m, r):
Generate
Code
Markdown
Run All
Restart
!whoami
!echo 24BAD405 EX01
[1]
nishanth\nishanth
24BAD405 EX01

```python
def merge(arr, l, m, r):
    n1 = m - l + 1
    n2 = r - m
    L = arr[l:l + n1]
    R = arr[m + 1:m + 1 + n2]
    i = j = 0
    k = l
    while i < n1 and j < n2:
        if L[i] <= R[j]:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    while i < n1:
        arr[k] = L[i]
        i += 1
        k += 1
    while j < n2:
        arr[k] = R[j]
        j += 1
        k += 1
def merge_sort(arr, l, r):
    if l < r:
        m = (l + r) // 2
        merge_sort(arr, l, m)
        merge_sort(arr, m + 1, r)
```

```
rge(arr, l, m, r)
    arr1 = [4, 1, 3, 9, 7]
    merge_sort(arr1, 0, len(arr1) - 1)
    print(*arr1)
    arr2 = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
    merge_sort(arr2, 0, len(arr2) - 1)
    print(*arr2)
```

```
1 3 4 7 9
1 2 3 4 5 6 7 8 9 10
```

```
def inv(a):
    if len(a)<2:return 0
    m=len(a)//2;l=a[:m];r=a[m:]
    c=inv(l)+inv(r);i=j=k=0
    while i<len(l) and j<len(r):
        if l[i]<=r[j]:a[k]=l[i];i+=1
        else:a[k]=r[j];c+=len(l)-i;j+=1
        k+=1
    a[k:]=l[i:]+r[j:];return c

print(inv([2,4,1,3,5]))
print(inv([2,3,4,5,6]))
print(inv([10,10,10]))
```

```
3
0
0
```

```python
from collections import Counter
def sort_by_frequency(arr):
    freq = Counter(arr)
    sorted_arr = sorted(arr, key=lambda x: (-freq[x], x))
    return sorted_arr
arr1 = [5, 5, 4, 6, 4]
print(*sort_by_frequency(arr1))
arr2 = [9, 9, 9, 2, 5]
print(*sort_by_frequency(arr2))
```

```
4 4 5 5 6
9 9 9 2 5
```

```python
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] ⩽ pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
arr1 = [4, 1, 3, 9, 7]
quick_sort(arr1, 0, len(arr1) - 1)
print(*arr1)
arr2 = [2, 1, 6, 10, 4, 1, 3, 9, 7]
quick_sort(arr2, 0, len(arr2) - 1)
print(*arr2)
```

```
1 3 4 7 9
1 1 2 3 4 6 7 9 10
```

```python
def partition(arr, l, r):
    pivot = arr[r]
    i = l
    for j in range(l, r):
        if arr[j] < pivot:
            arr[i], arr[j] = arr[j], arr[i]
            i += 1
    arr[i], arr[r] = arr[r], arr[i]
    return i
def kth_smallest(arr, l, r, k):
    if l <= r:
        pos = partition(arr, l, r)
        if pos == k - 1:
            return arr[pos]
        if pos > k - 1:
            return kth_smallest(arr, l, pos - 1, k)
        return kth_smallest(arr, pos + 1, r, k)
arr1 = [7, 10, 4, 3, 20, 15]
print(kth_smallest(arr1, 0, len(arr1) - 1, 3))
arr2 = [7, 10, 4, 20, 15]
print(kth_smallest(arr2, 0, len(arr2) - 1, 4))
```

```
7
15
```

```python
def sum_of_middle_elements(ar1, ar2, n):
    i = j = 0
    merged = []
    while i < n and j < n:
        if ar1[i] <= ar2[j]:
            merged.append(ar1[i])
            i += 1
        else:
            merged.append(ar2[j])
            j += 1
    while i < n:
        merged.append(ar1[i])
        i += 1
    while j < n:
        merged.append(ar2[j])
        j += 1
    return merged[n - 1] + merged[n]
ar1 = [1, 2, 4, 6, 10]
ar2 = [4, 5, 6, 9, 12]
print(sum_of_middle_elements(ar1, ar2, 5))
ar1 = [1, 12, 15, 26, 38]
ar2 = [2, 13, 17, 30, 45]
print(sum_of_middle_elements(ar1, ar2, 5))
```

```
11
32
```

Post-Lab Questions

1. How does the "Merge" step of Merge Sort allow us to count inversions more efficiently that a simple nested loop?

   while merging two stand sorted subarrays, if L[i] > R[j], all remaining elements

in L form inversions with R[j]
Efficiently counts inversions during merge.
O(n log n) instead of O(n²).

2. How would picking a "Median-of-Three" pivot affect the performance on an already sorted array?
Choosing pivot as median of first, middle, last element prevents worst-case.
Ensures roughly balanced partitions → maintains O(n log n) performance.

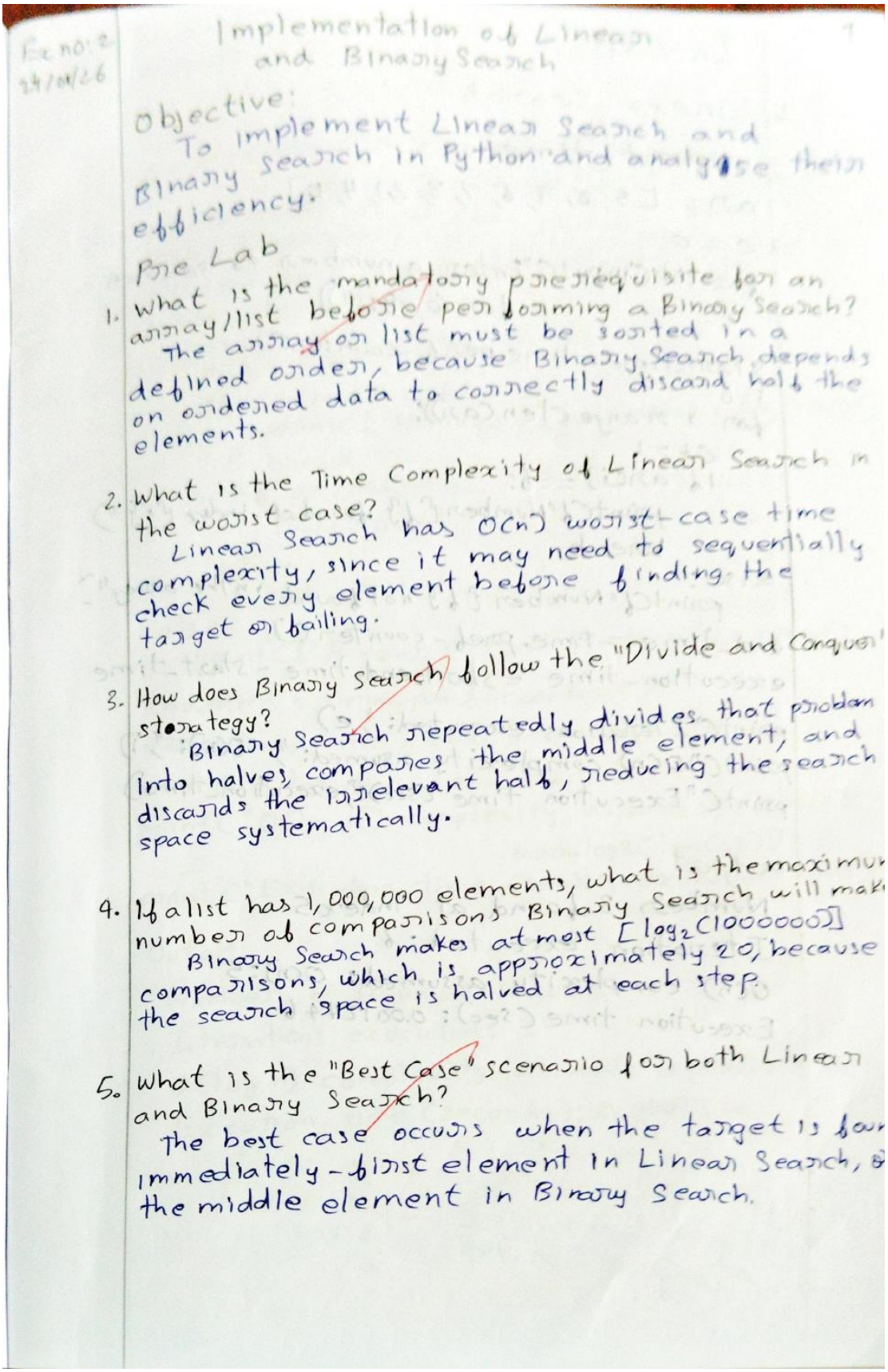
3. Is there a way to modify the QuickSort Partition logic to find the k^{th} element in O(N) average time instead of O(N log N)?
use Quick Select: partition and recurse only on the subarray containing k^{th} element.
Average time: O(n) instead of O(n log n)

4. If the two arrays were of size 10^6 each, describe a way to find the middle sum without creating a third array of size 2×10^6?
use two pointer merge: iterate through both arrays, track count, stop at middle two elements
Sum middle elements without extra array.
Space: O(1), Time: O(n).

5. Timsort is more similar to Merge Sort or Quick Sort, and explain why.
Timsort is more like MergeSort
Divides array into runs and merges them.
Stable, adaptive and merge-based sorting.

EX NO: 02 **….24.01.2026**

LINEAR SEARCH AND BINARY SEARCH



Ex no: 2
24/01/26

Implementation of Linear and Binary Search

Objective:
To implement Linear Search and Binary search in Python and analyse their efficiency.

Pre Lab

1. What is the mandatory prerequisite for an array/list before performing a Binary Search?
The array or list must be sorted in a defined order, because Binary Search depends on ordered data to correctly discard half the elements.

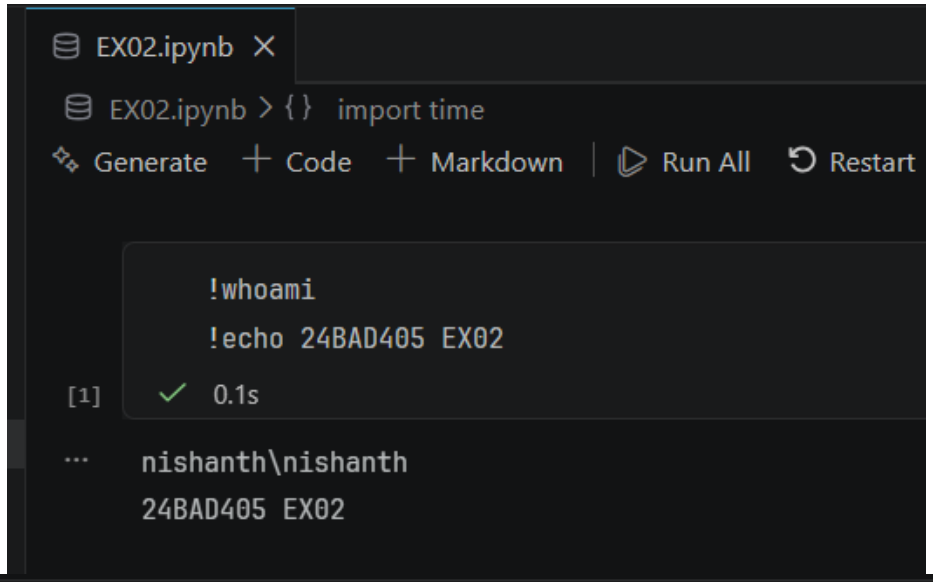
2. What is the Time Complexity of Linear Search in the worst case?
Linear Search has O(n) worst-case time complexity, since it may need to sequentially check every element before finding the target or failing.

3. How does Binary Search follow the "Divide and Conquer" strategy?
Binary Search repeatedly divides that problem into halves, compares the middle element, and discards the irrelevant half, reducing the search space systematically.

4. If a list has 1,000,000 elements, what is the maximum number of comparisons Binary Search will make?
Binary Search makes at most $\lfloor \log_2(1000000) \rfloor$ comparisons, which is approximately 20, because the search space is halved at each step.

5. What is the "Best Case" scenario for both Linear and Binary Search?
The best case occurs when the target is found immediately - first element in Linear Search, and the middle element in Binary Search.

Code:




```python
import time

ar = [5,8,9,6,7,3,2,1,4,0]
c = 0

f = int(input("Enter a number to search for (0-9): "))

start_time = time.perf_counter()
i=0
for i in range(len(ar)):
    c += 1
    if ar[i] == f:
        print(f"Number {f} found at index {i}")
        break
else:
    print(f"Number {f} not found in the array")

end_time = time.perf_counter()

execution_time = end_time - start_time

print("Iterations executed:", c)
print("O(n) complexity assumed:", "O(n):",i )
print("Execution time (seconds):", execution_time)
```

```
Number 3 found at index 5
Iterations executed: 6
O(n) complexity assumed: O(n): 5
Execution time (seconds): 0.0012446000000063656
```

```python
import time, math

ar = [1,2,3,4,5,6,7,8,9,10]
c = 0

f = int(input("Enter a number to search for (0-9): "))

low = 0
high = len(ar) - 1

start_time = time.perf_counter()

while low ⩽ high:
    c += 1
    mid = (low + high) // 2

    if ar[mid] == f:
        print(f"Number {f} found at index {mid}")
        break
    elif ar[mid] < f:
        low = mid + 1
    else:
        high = mid - 1
else:
    print(f"Number {f} not found in the array")

end_time = time.perf_counter()

execution_time = end_time - start_time
```

```
            high = mid - 1
    else:
        print(f"Number {f} not found in the array")

    end_time = time.perf_counter()

    execution_time = end_time - start_time

    print("Iterations executed:", c)
    print("O(log n) complexity assumed:", "log2(n):", math.log2(len(ar)))
    print("Execution time (seconds):", execution_time)
```

```
Number 6 found at index 5
Iterations executed: 3
O(log n) complexity assumed: log2(n): 3.321928094887362
Execution time (seconds): 0.0008844000003591646
```

Post-Lab Questions

1. Based on your output, compare the number of comparisions. why is the difference so significant?
Linear search check elements sequentially so comparisons grow linearly. Binary search halves the search space each step drastically reducing comparisons as input size increase.

2. If you have an unsorted list and need to perform only one search, which algorithm
Linear search is better because it works on unsorted data and avoids the extra sorting cost required by binary search for a single lookup

3. Why is the mid calculation (low+high)/2 potentially problematic in languages like C++ but generally fine in python
In C++, (low+high) may overflow due to fixed integer limits. Python uses arbitary precision integers, so over flow does not occur.

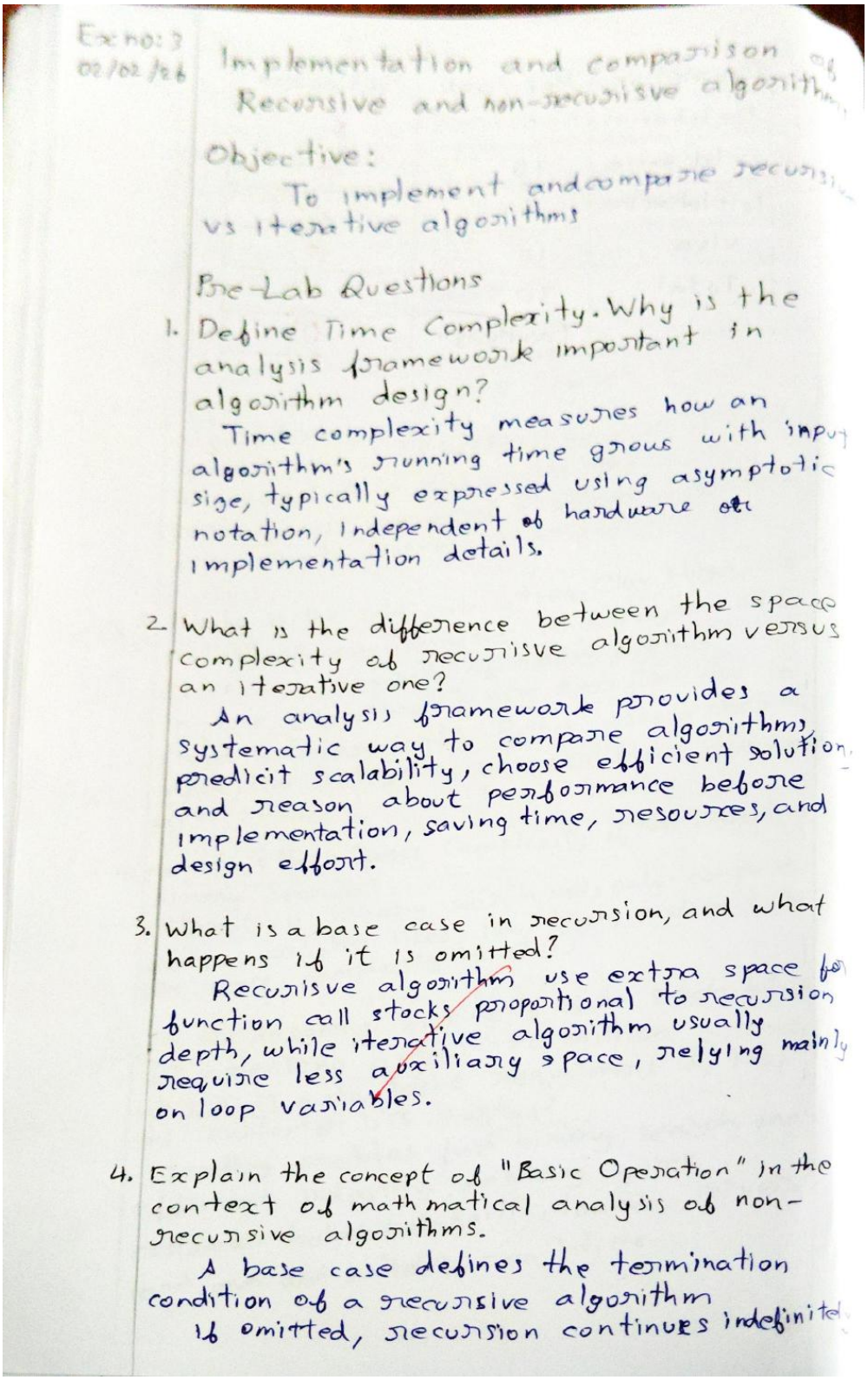
4. What is the Space Complexity of the iterative Binary Search?
Iterative binary search uses only constant extra variables likes low, high, and mid, so its space complexity is O(1)

5. You are designing complexity of the search future for a banking app with 50 million customer why is it a strategic requirment to keep the customer list sorted?
sorting enables fast binary search and efficient indexing, ensuring quick customer lookup, scalability, lower server load, and and reliable real-time

Description	Max Marks	Mark awarded
Pre lab exercises	5	5
In lab exercise	10	10
Post lab exercise	5	5
viva	10	10
Total	30	30
	Faculty Sign	

EX NO: 03 **02.02.2026**

RECURSIVE AND NON-RECURSIVE ALGORITHMS



Ex no: 3
02/02/26

Implementation and comparison of Recursive and non-recursive algorithms

Objective:

To implement and compare recursive vs iterative algorithms

Pre-Lab Questions

1. Define Time Complexity. Why is the analysis framework important in algorithm design?

Time complexity measures how an algorithm's running time grows with input size, typically expressed using asymptotic notation, independent of hardware or implementation details.

2. What is the difference between the space complexity of recursive algorithm versus an iterative one?

An analysis framework provides a systematic way to compare algorithms, predicit scalability, choose efficient solutions and reason about performance before implementation, saving time, resources, and design effort.

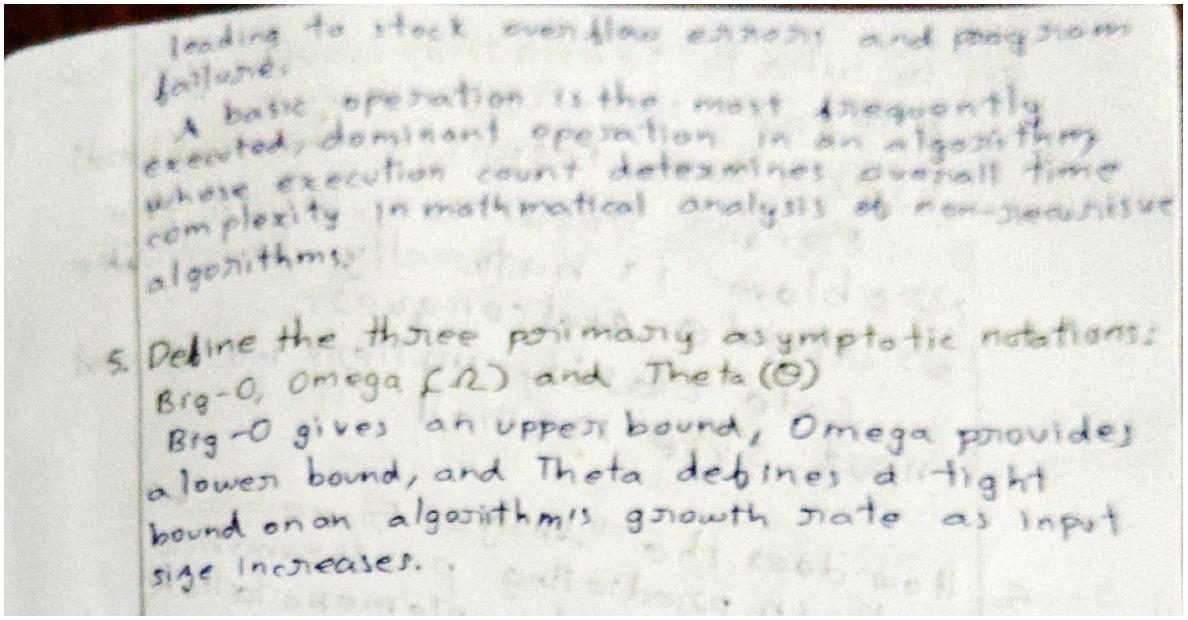
3. What is a base case in recursion, and what happens if it is omitted?

Recursive algorithm use extra space for function call stacks proportional to recursion depth, while iterative algorithm usually require less auxiliary space, relying mainly on loop variables.

4. Explain the concept of "Basic Operation" in the context of mathematical analysis of non-recursive algorithms.

A base case defines the termination condition of a recursive algorithm

If omitted, recursion continues indefinitely



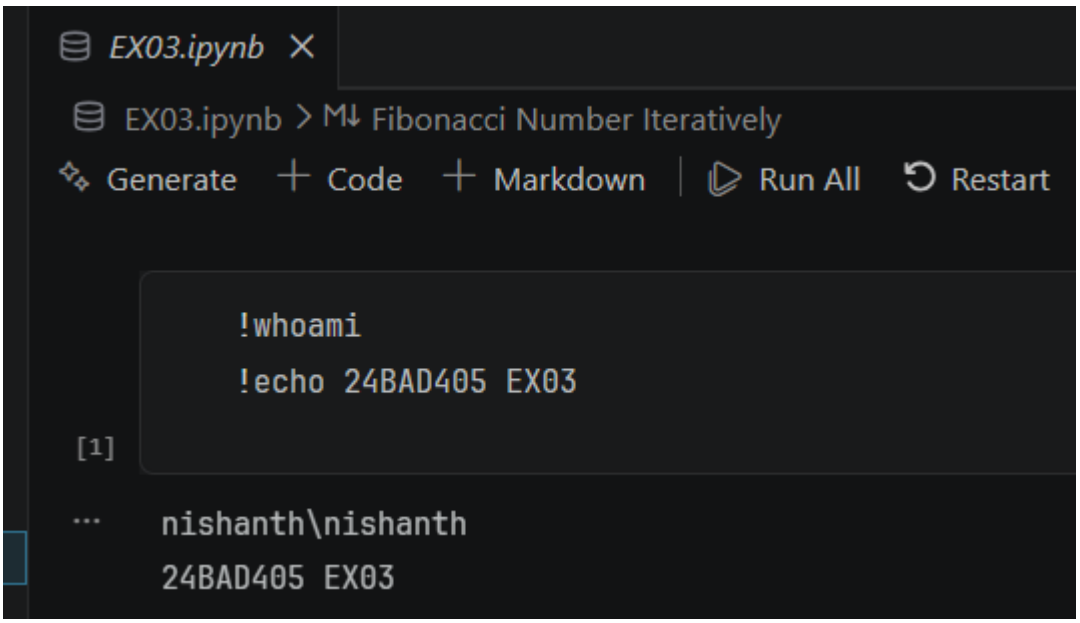
leading to stack overflow errors and program failure.

A basic operation is the most frequently executed, dominant operation in an algorithm whose execution count determines overall time complexity in mathematical analysis of non-recursive algorithms.

5. Define the three primary asymptotic notations: Big-O, Omega (Ω) and Theta (Θ)

Big-O gives an upper bound, Omega provides a lower bound, and Theta defines a tight bound on an algorithm's growth rate as input size increases.

Code:



Fibonacci Number Iteratively

```
def fibonacci_iterative(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

Factorial Number Recursively

```
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)
```

display the time complexity

```
import time

test_values = [10, 20, 30, 40]
```

```python
    print("Fibonacci (Iterative) Execution Time:")
    for n in test_values:
        start = time.perf_counter()
        fibonacci_iterative(n)
        end = time.perf_counter()
        print(f"n = {n} → Time = {end - start:.10f} seconds")

    print("\nFactorial (Recursive) Execution Time:")
    for n in test_values:
        start = time.perf_counter()
        factorial_recursive(n)
        end = time.perf_counter()
        print(f"n = {n} → Time = {end - start:.10f} seconds")
```

```
Fibonacci (Iterative) Execution Time:
n = 10 → Time = 0.0000182000 seconds
n = 20 → Time = 0.0000118000 seconds
n = 30 → Time = 0.0000080000 seconds
n = 40 → Time = 0.0000056000 seconds

Factorial (Recursive) Execution Time:
n = 10 → Time = 0.0000154000 seconds
n = 20 → Time = 0.0000136000 seconds
n = 30 → Time = 0.0000161000 seconds
n = 40 → Time = 0.0000188000 seconds
```

Post Lab Questions

1. Based on your results, why is the iterative version significantly faster for large values of n?

The iterative Fibonacci is O(n) with constant work per step, while recursive Fibonacci is exponential O(φ^n) due to massive redundant calculations of the same subproblems.

2. Draw the recursion tree for fib_recursive (4). How many redundant calls are made?

Recursion tree for fib(4) has 9 calls total.

fib(2) called twice, fib(1) four times, fib(0) three times → 5 redundant calls

3. What is the recurrence relation for the recursive Fibonacci algorithm?

The recurrence relation for recursive Fibonacci is

$T(n) = T(n-1) + T(n-2) + \Theta(1)$,

with $T(0) = \Theta(1), T(1) = \Theta(1)$

4. In what scenario would you prefer recursion over iteration despite the overhead?

Prefer recursion when the problem is naturally tree like or divide and conquer. code clarity matters more, or input size is small

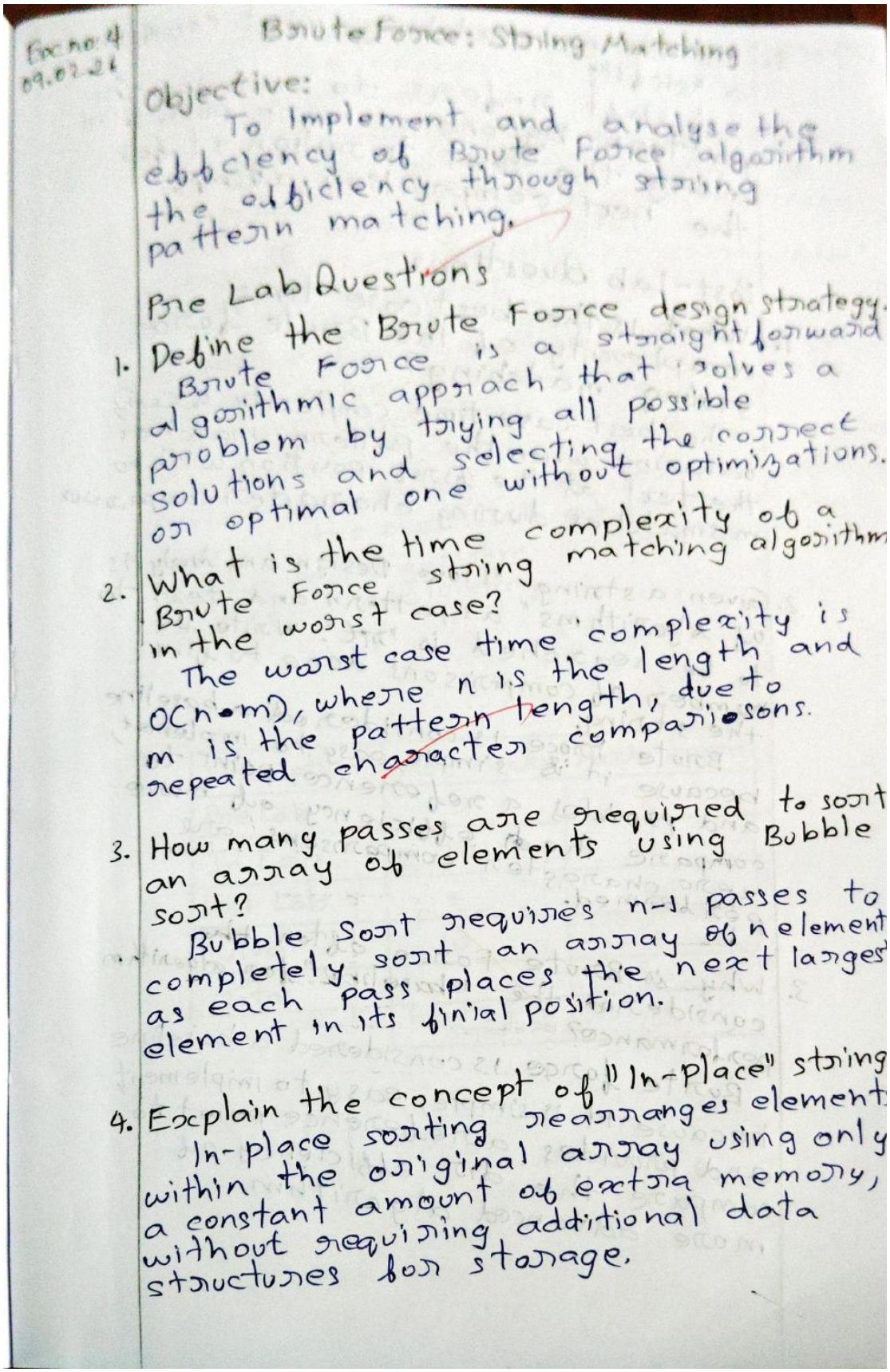
5. How does the "Analysis Framework" help in predicting performance before actual implementation?

The analysis framework predicts growth rate theoretically, allowing comparision of algorithms and identification of inefficient ones before coding and testing.

Description	Max Marks	Marks Award.
Pre Lab	5	5
In Lab	10	10
Post Lab	5	5
viva	10	9
total	30	29
	Faculty signature	

EX NO: 04 **...09.02.2026**

BRUTE FORCE STRING MATCHING



Ex no 4
09.02.26

Brute Force: String Matching

Objective:
To Implement and analyse the efficiency of Brute Force algorithm the efficiency through string pattern matching.

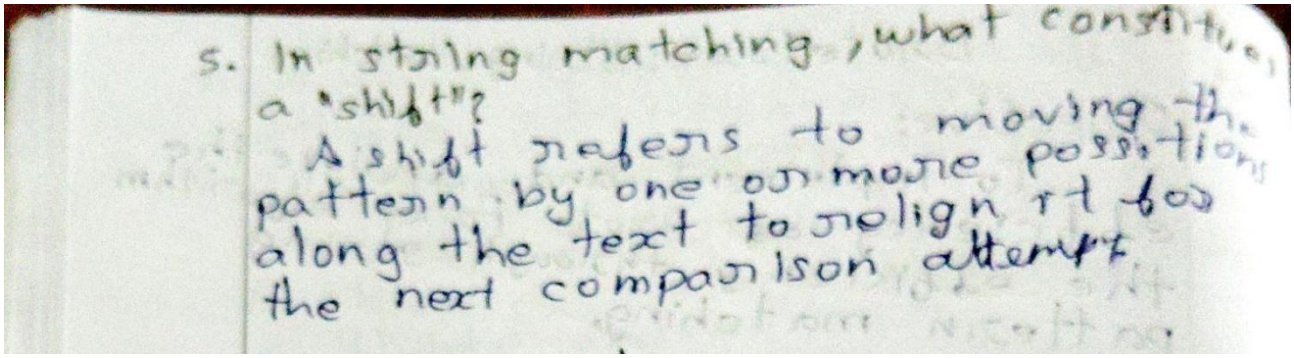
Pre Lab Questions

1. Define the Brute Force design strategy.
Brute Force is a straightforward algorithmic approach that solves a problem by trying all possible solutions and selecting the correct or optimal one without optimizations.

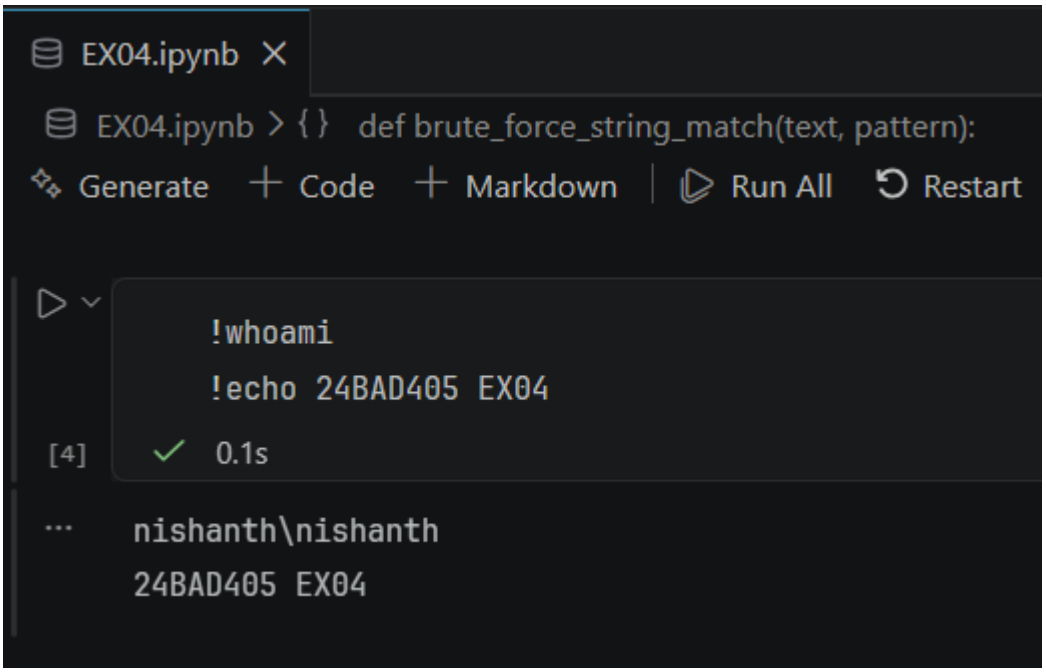
2. What is the time complexity of a Brute Force string matching algorithm in the worst case?
The worst case time complexity is O(n*m), where n is the length and m is the pattern length, due to repeated character comparisons.

3. How many passes are required to sort an array of elements using Bubble sort?
Bubble Sort requires n-1 passes to completely sort an array of n element as each pass places the next largest element in its finial position.

4. Explain the concept of "In-place" string
In-place sorting rearranges element within the original array using only a constant amount of extra memory, without requiring additional data structures for storage.


5. In string matching, what constitutes a "shift"?
A shift refers to moving the pattern by one or more positions along the text to align it for the next comparison attempt.

Code:


EX04.ipynb
EX04.ipynb > { } def brute_force_string_match(text, pattern):
Generate
Code
Markdown
Run All
Restart
!whoami
!echo 24BAD405 EX04
[4]
0.1s
nishanth\nishanth
24BAD405 EX04

NISHANTH P - 24BAD405

```python
def brute_force_string_match(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0

    for i in range(n - m + 1):
        text_window = text[i:i+m]
        comparisons += 1

        if text_window == pattern:
            return i, comparisons

    return -1, comparisons
```

```python
text = "BAGBBAD"
pattern = "BAD"

index, comparisons = brute_force_string_match(text, pattern)

print("First index:", index)
print("Total character comparisons:", comparisons)
```

```
First index: 15
Total character comparisons: 16
```

Post-Lab Questions

1. What is the best case time complexity of the Brute force string matching?

The best case time complexity is O(m), occurring when the pattern matches the text at the first position with no mismatches during character comparis...

2. Given a string "I love Design and Analysis of Algorithms" as pattern and text to be searched is "rit". Write the number of comparisons done to find the string.

Brute force is considered a baseline because it is simple, easy to implemen... and provides a reference point to compare the efficiency of more zero character comparisons are performed.

3. Why is Brute force often considered the "baseline" for algorithm performance?

Brute force is considered a baseline because it is simple, easy to implement, and provides a reference point to compare the efficiency of more advanced algorithm...

4. How does the length of the pattern (m) affect the number of comparison in string matching?

As the pattern length increases, especially the number of character comparisons per alignment increase linearly leading to higher total comparisons, especially in worst-case matching scenarios.

5. Identify a scenario where brute-force string matching is faster than more complex algorithms.

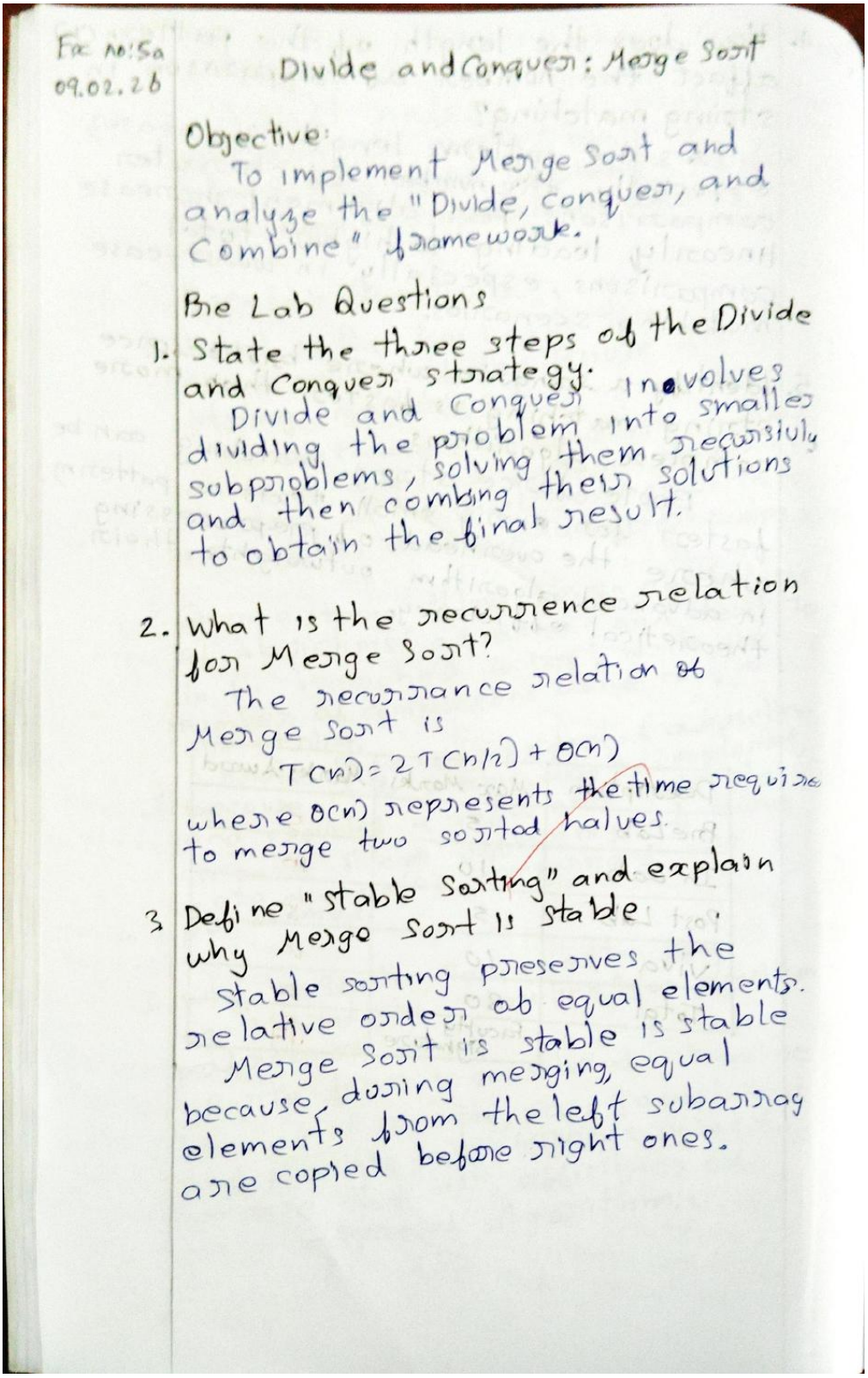
Brute force string matching can be faster for very small texts or patterns where the overhead of preprocessing in advanced algorithm outweighs their theoretical efficiency.

Description	Max Marks	Marks Awood
PreLab	5	5
In Lab	10	10
Post Lab	5	5
viva	10	10
Total	30	30
	Faculty Signature	

EX NO: 05 **…..09.02.206**

MERGE SORT, QUICK SORT AND BINARY SEARCH

EX NO: 05a DIVIDE AND CONQUER: MERGE SORT



Ex No:5a
09.02.26

Divide and Conquer: Merge Sort

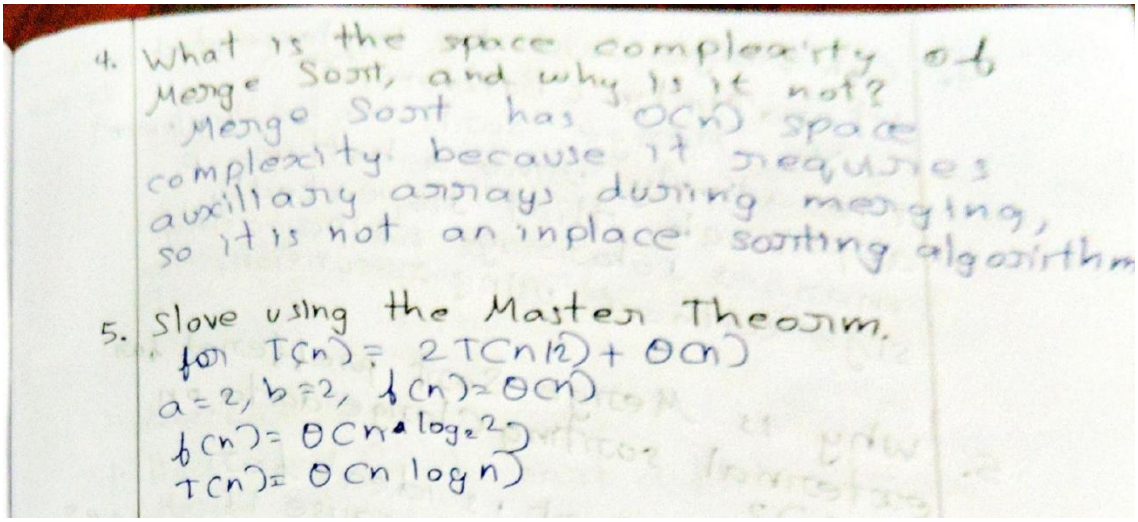
Objective:
To implement Merge Sort and analyze the "Divide, conquer, and Combine" framework.

Pre Lab Questions

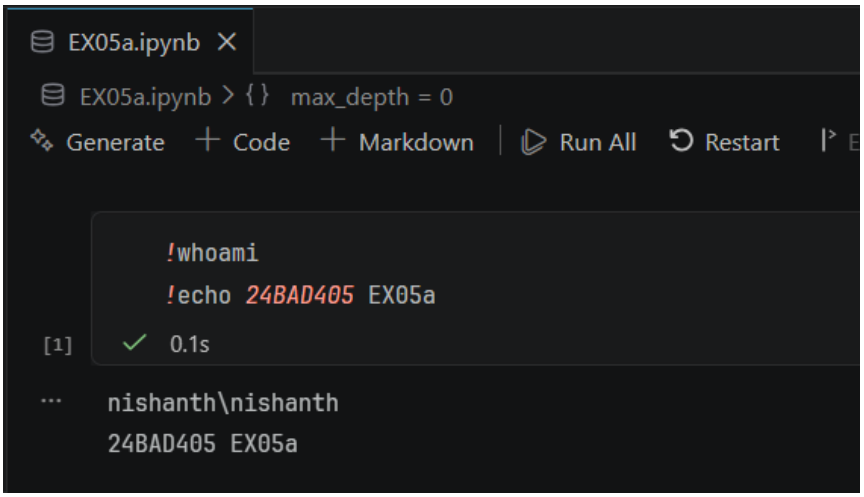
1. State the three steps of the Divide and Conquer strategy.
Divide and Conquer involves dividing the problem into smaller subproblems, solving them recursively and then combing their solutions to obtain the final result.

2. What is the recurrence relation for Merge Sort?
The recurrance relation of Merge Sort is
$T(n) = 2T(n/2) + O(n)$
where $O(n)$ represents the time required to merge two sorted halves.

3. Define "stable Sorting" and explain why Merge Sort is stable
Stable sorting preserves the relative order of equal elements.
Merge Sort is stable is stable because, during merging, equal elements from the left subarray are copied before right ones.



Code:




```python
max_depth = 0

def merge_sort(arr, depth=1):
    global max_depth
    max_depth = max(max_depth, depth)

    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], depth + 1)
    right_half = merge_sort(arr[mid:], depth + 1)

    return merge(left_half, right_half)
```

```python
def merge(left, right):
    merged = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] ⩽ right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged
```

```python
arr = [38, 27, 43, 3, 9, 82, 10]

sorted_arr = merge_sort(arr)

print("Original array:", arr)
print("Sorted array:", sorted_arr)
print("Maximum recursion depth:", max_depth)
```

```
Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array: [3, 9, 10, 27, 38, 43, 82]
Maximum recursion depth: 4
```

Post Lab Questions

1. Compare the time complexity of Merge Sort() with Bubble Sort() for.

Merge Sort runs in $\Theta(n \log n)$ for all cases,

while Bubble Sort runs in $\Theta(n^2)$, making Merge Sort far more efficient for large n.

2. Explain how the "Combine" step in the most critical part of Merge Sort.

The combine step merges two sorted subarrays efficiently in linear time, ensuring correctness and maintaining overall $\Theta(n \log n)$ performance of the algorithm.

3. What happens to the space complexity if you implement Merge Sort on a Linked List instead of an array?

When implemented on Linked List, Merge Sort requires only $O(\log n)$ extra space for recursion, since merging

4. Can Merge Sort be Implemented iteratively (Bottom-up)?

Yes, Merge Sort can be Implemented iteratively using a bottom-up approach that repeatedly merges subarrays of increasing size without using recursion.

5. Why is Merge Sort preffered for external sorting (large data on disk)?

Merge Sort is ideal for external sorting because it access data sequentially, minimizes random disk access, and efficiently handles very large datasets stored on disk.

EX NO: 05b DIVDE AND CONQUER: QUICK SORT

Divide and Conquer: Quick Sort

Objective:

To implement Quick Sort using different partitioning schemes and analyse how pivot selection affects the algorithm's performance and recursion depth.

Pre-Lab Questions

1. How does Quick Sort differ from Merge Sort?

Quick Sorts swaps elements across partitions, changing relative order. while Merge Sort divides array and merges them, requiring extra memory for merging

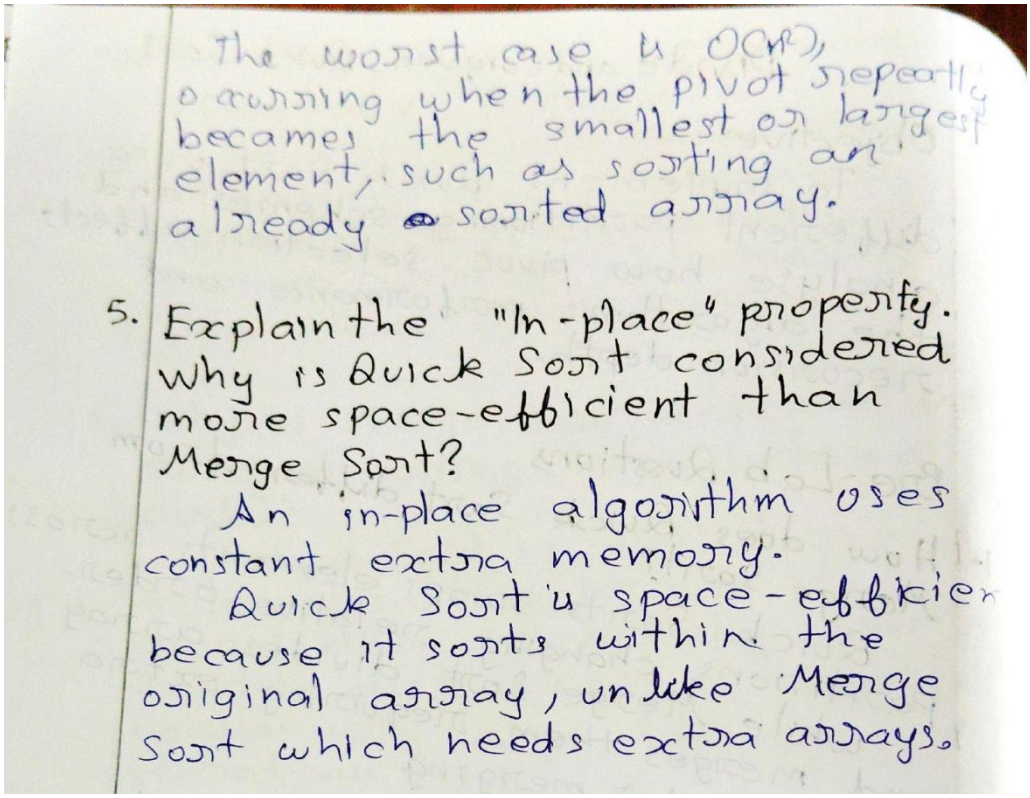
2. Define a "Pivot" element.

A pivot is a selection element in Quick Sort used to divide the array into smaller subarrays containing elements less than and greater than itself.

3. What is the role of pivot in the partitioning process?

The pivot rearranges elements so smaller values move to its left an large values to its right, placing the pivot in its correct sorted position.

4. What is the worst-case time complexity of Quick Sort? Describe a scenario that triggers it.


The worst case is O(n²), occurring when the pivot repeatedly becomes the smallest or largest element, such as sorting an already sorted array.

5. Explain the "In-place" property. Why is Quick Sort considered more space-efficient than Merge Sort?

An in-place algorithm uses constant extra memory.

Quick Sort is space-efficient because it sorts within the original array, unlike Merge Sort which needs extra arrays.

Code:

```
!whoami
!echo 24BAD405 EX05b
```

[1]

```
nishanth\nishanth
24BAD405 EX05b
```

```
def quick_sort_last_pivot(arr, low, high, stats):
    if low < high:
        pi = partition_last(arr, low, high, stats)
        quick_sort_last_pivot(arr, low, pi - 1, stats)
        quick_sort_last_pivot(arr, pi + 1, high, stats)


def partition_last(arr, low, high, stats):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        stats["comparisons"] += 1
        if arr[j] ⩽ pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

[1]

```python
import random

def randomized_quick_sort(arr, low, high, stats):
    if low < high:
        pi = randomized_partition(arr, low, high, stats)
        randomized_quick_sort(arr, low, pi - 1, stats)
        randomized_quick_sort(arr, pi + 1, high, stats)


def randomized_partition(arr, low, high, stats):
    rand_index = random.randint(low, high)
    arr[rand_index], arr[high] = arr[high], arr[rand_index]
    return partition_last(arr, low, high, stats)
```

Post-Lab Questions

1. Why is Quick Sort not a "stable" sorting algorithm? Provide an example where the relative order of equal elements changes.

Quick Sort swaps elements across partitions, changing relative order.

2. Based Task3, how did randomisation affect the number of comparisons? Why is this helpful for real-world data?

Randomisation reduces consistent poor pivot choices, lowering comparisons on average and avoiding worst-case behaviour, making performance more reliable for real-world, partially sorted data.

3. What is the auxiliary space complexity of Quick Sort? Does it come from data movement or the function call stack?

Auxiliary space is O(log n) on average due to recursion.
It comes from the function call stack, not from extra arrays

4. Explain the "Median of Three" rule. How does it improve pivot selection?

Median of three selects the median of first, middle, and last elements as pivot, reducing chances of extreme pivots and improving balance in partioning.

5. In what scenario is Merge sort better than Quick Sort?

Merge Sort is better when stability is required, data is linked list based
guaranteed O(n log n) performance is needed regardless of input order.

EX NO:5c DECREASE AND CONQUER: BINARY SEARCH

Ex No 5c
9.2.26

Decrease and Conquer
Binary Search

Objective

To implement Binary Search and compare its logarithmic growth against Linear Search.

Pre-Lab Questions

1. What is the mandatory prerequisite for an array before performing Binary Search?

Binary Search works by repeatedly halving the search space, which is only possible when elements follow an order.

2. Differentiate between "Divide and Conquer" and "Decrease and Conquer".

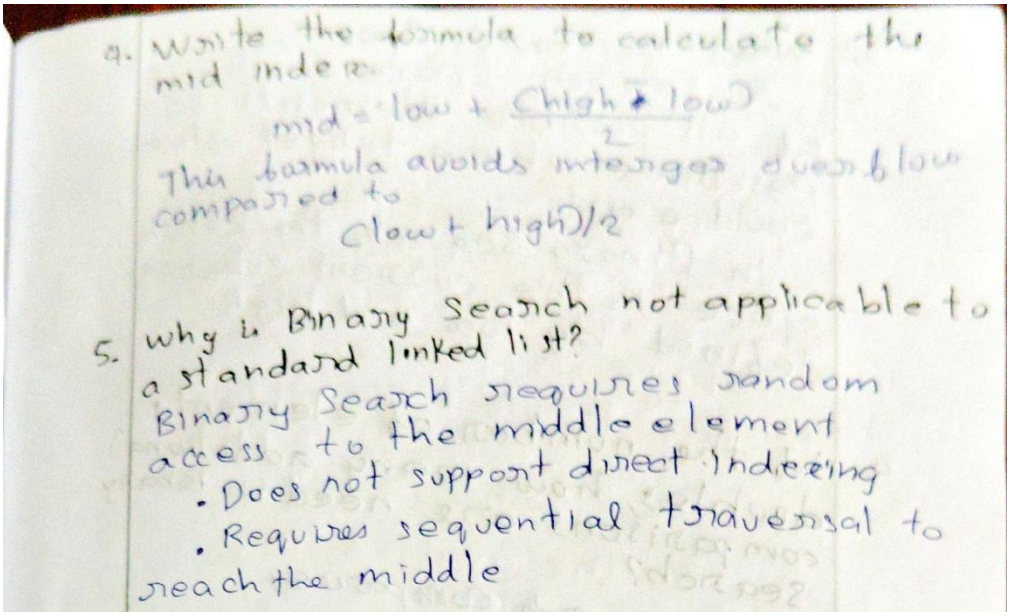
Aspect	Divide and Conquer	Decrease and Conquer
Problem division	Divide the problem into multiple subproblem	Reduces the problem size by fixed amount
Subproblem solving	solves all subproblem recursively	solves one smaller subproblem
Combination step	combines result of subproblems	usually no combine step

3. If an array has 1024 elements, what is the maximum number of comparison for Binary Search?

Max comparisons = $\log_2(n)+1$

for $n=1024=2^{10}$

$\log_2(1024)+1 = 10+1 = 11$



Code:

```
!whoami
!echo 24BAD405 EX05c
```

```
nishanth\nishanth
24BAD405 EX05c
```

```python
def binary_search_recursive(arr, low, high, target):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] > target:
        return binary_search_recursive(arr, low, mid - 1, target)
    else:
        return binary_search_recursive(arr, mid + 1, high, target)
```

```python
def binary_search_iterative(arr, target):
    low = 0
    high = len(arr) - 1

    while low ⩽ high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] > target:
            high = mid - 1
        else:
            low = mid + 1

    return -1
```

```python
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

```python
import time

n = 1_000_000
arr = list(range(n))
target = n - 1

start = time.time()
linear_search(arr, target)
linear_time = time.time() - start

start = time.time()
binary_search_iterative(arr, target)
binary_iter_time = time.time() - start

start = time.time()
binary_search_recursive(arr, 0, len(arr) - 1, target)
binary_rec_time = time.time() - start

print("Linear Search Time:", linear_time)
print("Iterative Binary Search Time:", binary_iter_time)
print("Recursive Binary Search Time:", binary_rec_time)
```

```
Linear Search Time: 0.07845950126647949
Iterative Binary Search Time: 9.608268737792969e-05
Recursive Binary Search Time: 9.584426879882812e-05
```

Post-Lab Questions

1. Why does iterative Binary Search use O(1) space while the recursive version uses O(log n) space?

- Iterative Binary Search uses loop and only few variables, so it needs constant auxiliary space – O(1)
- Recursive Binary Search makes recursive function calls.

Each call is stored on the call stack and maximum depth of recursion is $\log_2 n$, so it uses O(log n) space.

2. Explain Average case complexity for Binary Search

The average considers the expected number of comparisons when the target element is equally likely to be anywhere in the array.

3. What is Search Space?

A search space is the set of all possible elements or positions where target value could exist

In Binary Search, the search space is the current subarray defined by low and high

4. If the number of elements doubles, how many additional comparisons are need in Binary Search?

Binary search comparisons

Comparisons = $\log_2 n$

input size doubles n to $2n$:

$\log_2(2n) = \log_2 n + 1$

5. How does Binary Search apply to finding roots of mathematical functions?

Binary search is used in root finding when:

The function is continuous

The function changes sign in an interval $[a, b]$

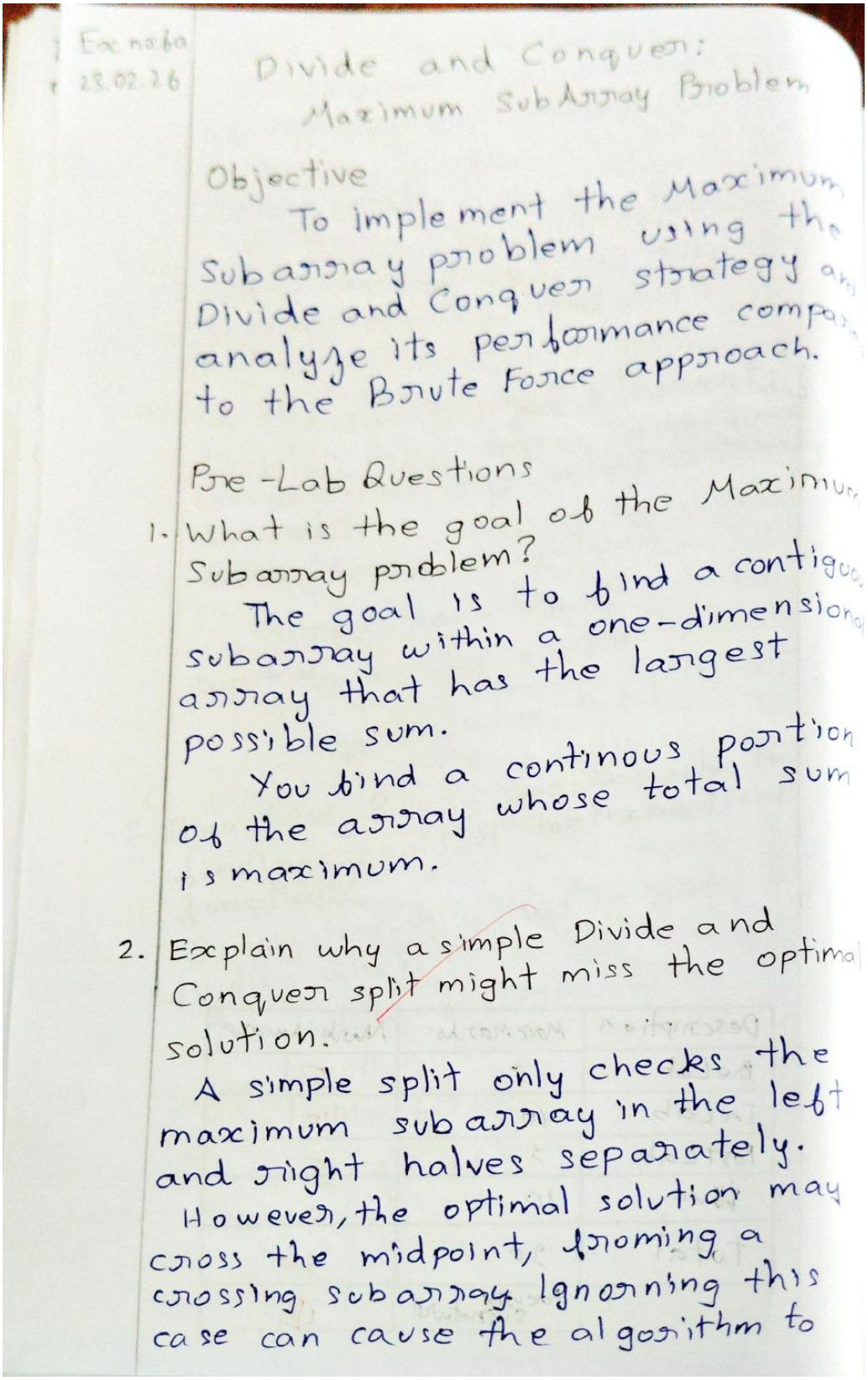
$f(a) \cdot f(b) < 0$

Description	Max Marks	Marks Awarded
Pre Lab	5	5
In Lab	10	10
Post Lab	5	5
Viva	10	10
total	30	30
	Faculty signature	

EX NO: 06 **23.02.2026**

MAXIMUM SUBARRAY AND STRASSEN'S MATRIX MULTIPLICATION

EX NO: 06a DIVIDE AND CONQUER: MAXIMUM SUBARRAY PROBLEM



Ex no:6a
23.02.26

Divide and Conquer:
Maximum SubArray Problem

Objective

To implement the Maximum Subarray problem using the Divide and Conquer strategy and analyze its performance compared to the Brute Force approach.

Pre-Lab Questions

1. What is the goal of the Maximum Subarray problem?

The goal is to find a contiguous subarray within a one-dimensional array that has the largest possible sum.

You find a continous portion of the array whose total sum is maximum.

2. Explain why a simple Divide and Conquer split might miss the optimal solution.

A simple split only checks the maximum subarray in the left and right halves separately.

However, the optimal solution may cross the midpoint, froming a crossing subarray Ignorning this case can cause the algorithm to

miss the true maximum sum.

3. What is the complexity of the cross subarray function?

The cross subarray function scans elements from the midpoint outward to both sides to compute the maximum crossing sum.

Since it may examine all elements once, its time complexity is linear, which is O(n)

4. Write the recurrence relation for divide and conquer version

$T(n) = 2T(n/2) + O(n)$

This represents solving two subproblems of size n/2 and combining their results using a linear-time crossing calculation.

5 In the base case, what should the algorithm return?

In the base case, when the array contains only one element the algorithm should return that element, itself the maximum subarray for that portion of the array.

Code:

```
!whoami
!echo 24BAD405 EX06a
```

```
nishanth\nishanth
24BAD405 EX06a
```

```
import random
import time
import matplotlib.pyplot as plt
from typing import Tuple, List
```

```
def max_subarray_brute_force(arr: List[int]) -> Tuple[int, int, int]:
    """
    Brute Force - O(n³) time
    Returns: (max_sum, start_index, end_index)
    """
    if not arr:
        return 0, -1, -1

    n = len(arr)
    max_sum = float('-inf')
    best_start = best_end = -1

    for i in range(n):
        for j in range(i, n):
            current_sum = 0
            for k in range(i, j+1):
                current_sum += arr[k]
            if current_sum > max_sum:
                max_sum = current_sum
                best_start = i
                best_end = j

    return max_sum, best_start, best_end
```

```python
def max_subarray_divide_conquer(arr: List[int]) -> Tuple[int, int, int]:
    """
    Divide and Conquer - O(n log n) time
    Returns: (max_sum, start_index, end_index)
    """
    def conquer(left: int, right: int) -> Tuple[int, int, int, int, int]:

        if left == right:
            val = arr[left]
            return val, val, val, val, (left, left)

        mid = (left + right) // 2

        left_total, left_prefix, left_suffix, left_max, left_indices = conquer(left, mid)
        right_total, right_prefix, right_suffix, right_max, right_indices = conquer(mid+1, right)

        cross_sum = left_suffix + right_prefix


        if left_max ≥ right_max and left_max ≥ cross_sum:
            max_sum = left_max
            best_indices = left_indices
        elif right_max ≥ cross_sum:
            max_sum = right_max
            best_indices = right_indices
        else:
            max_sum = cross_sum
            best_indices = (left_indices[0], right_indices[1])
```


```python
        if left_max ≥ right_max and left_max ≥ cross_sum:
            max_sum = left_max
            best_indices = left_indices
        elif right_max ≥ cross_sum:
            max_sum = right_max
            best_indices = right_indices
        else:
            max_sum = cross_sum
            best_indices = (left_indices[0], right_indices[1])
```


```python
        total = left_total + right_total

        max_prefix = max(left_prefix, left_total + right_prefix)

        max_suffix = max(right_suffix, right_total + left_suffix)

        return total, max_prefix, max_suffix, max_sum, best_indices

    if not arr:
        return 0, -1, -1

    _, _, _, max_sum, (start, end) = conquer(0, len(arr)-1)
    return max_sum, start, end
```

```python
def max_subarray_kadane(arr: List[int]) -> Tuple[int, int, int]:
    """
    Kadane's algorithm - O(n) time (for reference & verification)
    """
    if not arr:
        return 0, -1, -1

    max_ending_here = max_so_far = arr[0]
    start = end = temp_start = 0

    for i in range(1, len(arr)):
        if arr[i] > max_ending_here + arr[i]:
            max_ending_here = arr[i]
            temp_start = i
        else:
            max_ending_here += arr[i]

        if max_ending_here > max_so_far:
            max_so_far = max_ending_here
            start = temp_start
            end = i

    return max_so_far, start, end
```

```python
def test_max_subarray():
    test_cases = [
        [-2, 1, -3, 4, -1, 2, 1, -5, 4],
        [1, -2, 3, 10, -4, 7, 2, -5],
        [-1, -2, -3, -4],
        [5],
        [-2, -3, 4, -1, -2, 1, 5, -3],
        list(range(-10, 15)) + list(range(10, -15, -1))
    ]

    print("VERIFICATION RESULTS")

    for idx, arr in enumerate(test_cases, 1):
        bf_sum, bf_s, bf_e = max_subarray_brute_force(arr)
        dc_sum, dc_s, dc_e = max_subarray_divide_conquer(arr)
        kd_sum, kd_s, kd_e = max_subarray_kadane(arr)

        print(f"\nTest {idx}:")
        print(f"  Array : {arr}")
        print(f"  Brute : sum={bf_sum:4d}  [{bf_s}..{bf_e}]")
        print(f"  D&C   : sum={dc_sum:4d}  [{dc_s}..{dc_e}]")
        print(f"  Kadane: sum={kd_sum:4d}  [{kd_s}..{kd_e}]")

        if not (bf_sum == dc_sum == kd_sum):
            print("  !!! INCONSISTENT RESULT !!!")
```

```python
def performance_comparison():
    sizes = [10, 50, 100, 250, 500]
    bf_times = []
    dc_times = []


    print("PERFORMANCE COMPARISON")


    for n in sizes:
        arr = [random.randint(-100, 100) for _ in range(n)]

        t1 = time.perf_counter()
        max_subarray_brute_force(arr)
        t2 = time.perf_counter()
        bf_time = (t2 - t1) * 1000  # ms

        t1 = time.perf_counter()
        max_subarray_divide_conquer(arr)
        t2 = time.perf_counter()
        dc_time = (t2 - t1) * 1000  # ms

        bf_times.append(bf_time)
        dc_times.append(dc_time)

        print(f"n = {n:4d} | Brute: {bf_time:8.3f} ms | D&C: {dc_time:8.3f} ms | ratio: {bf_time/dc_time:6.1f}x")

    plt.figure(figsize=(10, 6))
```

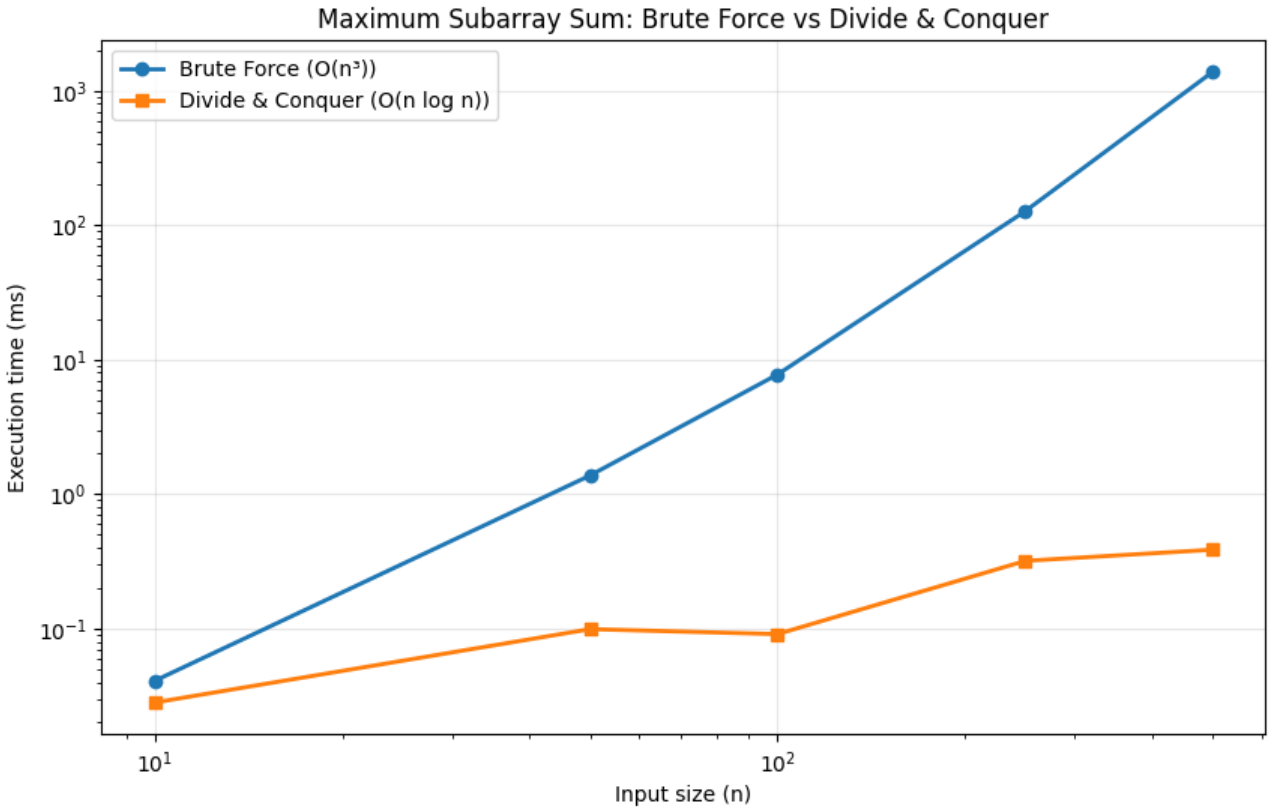
```python
    plt.figure(figsize=(10, 6))
    plt.plot(sizes, bf_times, 'o-', label='Brute Force (O(n³))', linewidth=2)
    plt.plot(sizes, dc_times, 's-', label='Divide & Conquer (O(n log n))', linewidth=2)
    plt.xlabel('Input size (n)')
    plt.ylabel('Execution time (ms)')
    plt.title('Maximum Subarray Sum: Brute Force vs Divide & Conquer')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.yscale('log')
    plt.xscale('log')
    plt.show()
```

14]

```python
if __name__ == "__main__":
    print("Maximum Subarray Sum Implementations\n")

    test_max_subarray()

    performance_comparison()
```

15]

```
Maximum Subarray Sum Implementations

VERIFICATION RESULTS

Test 1:
  Array : [-2, 1, -3, 4, -1, 2, 1, -5, 4]
  Brute : sum=   6  [3..6]
  D&C   : sum=   6  [3..8]
  Kadane: sum=   6  [3..6]

Test 2:
  Array : [1, -2, 3, 10, -4, 7, 2, -5]
  Brute : sum=  18  [2..6]
  D&C   : sum=  18  [2..6]
  Kadane: sum=  18  [2..6]

Test 3:
  Array : [-1, -2, -3, -4]
  Brute : sum=  -1  [0..0]
  D&C   : sum=  -1  [0..0]
  Kadane: sum=  -1  [0..0]

Test 4:
  Array : [5]
  Brute : sum=   5  [0..0]
...
n =   50 | Brute:    1.375 ms | D&C:    0.099 ms | ratio:   13.9x
n =  100 | Brute:    7.750 ms | D&C:    0.091 ms | ratio:   85.2x
n =  250 | Brute:  127.088 ms | D&C:    0.319 ms | ratio:  398.5x
n =  500 | Brute: 1381.452 ms | D&C:    0.386 ms | ratio: 3578.0x
```



Post-Lab Questions

1. If the input array consists only of negative numbers, what result does your algorithm provide?

If every element is negative, the algorithm return the largest single value

because any longer subarray decreases the sum compared to selecting the maximum

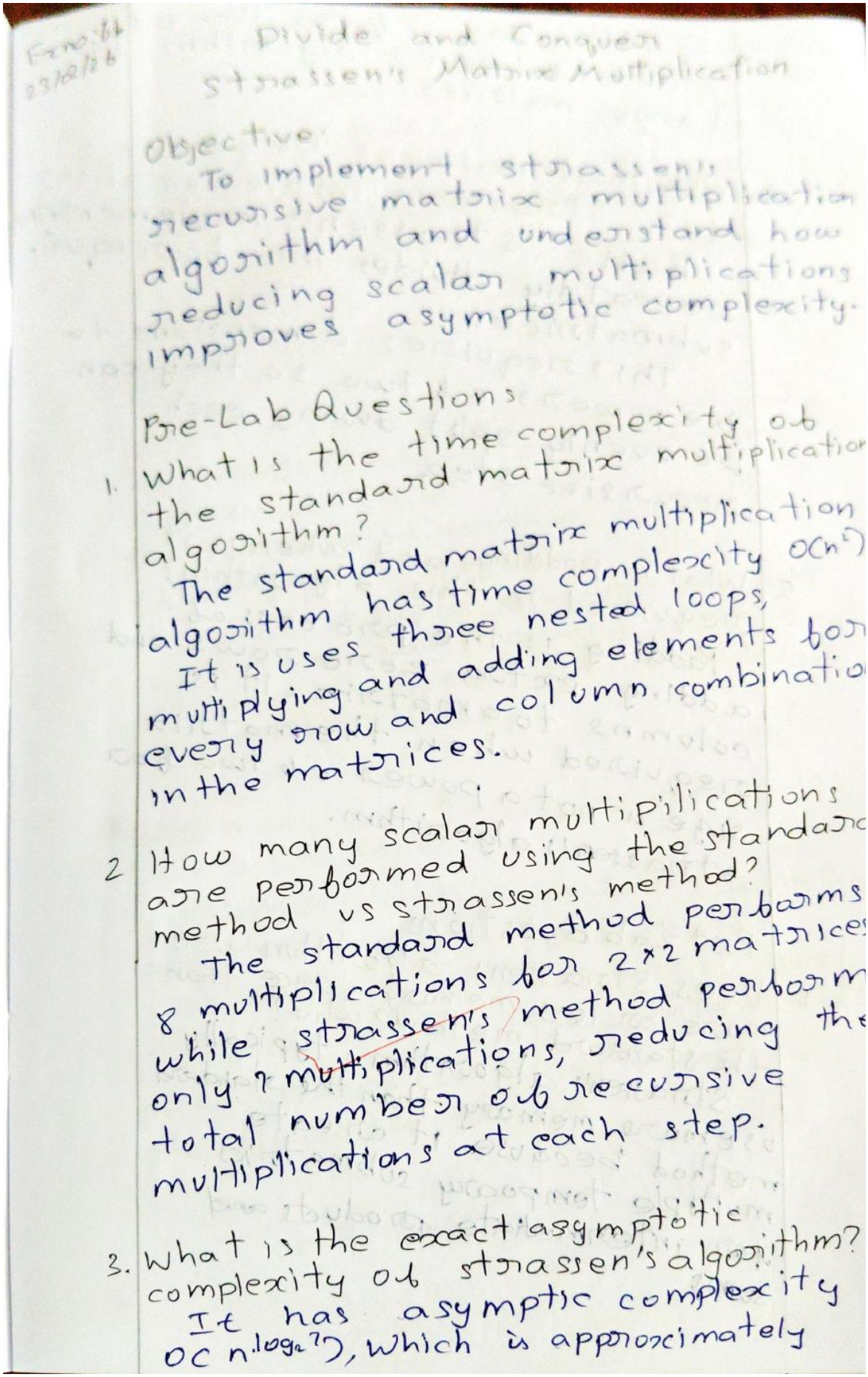
2. Compare the performance of this algorithm with Kadane's algorithm
Divide and conquer runs in time due to recursive splitting and merging, whereas Kadane's algorithm achieves optimal linear time

3. How many levels are there in the recursion tree for an input of size?
For input size, the recursion tree height is because each recursive step halves the array until reaching base cases of single element

4 What is the space complexity of this recursive implementation?
The recursive implementation uses auxillary space from the call stack, since only one branch of recursive calls exists simultaneously

5. Describe a real world scenario where finding the maximum subarray is useful
In stock analysis, maximum subarray identifiers the best buy and sell window by finding a continous period with maximum profit

EX NO: 06b DIVIDE AND CONQUER: STRASSEN'S MATRIX MULTIPLICATION



Exno:6b
23/08/26

Divide and Conquer
Strassen's Matrix Multiplication

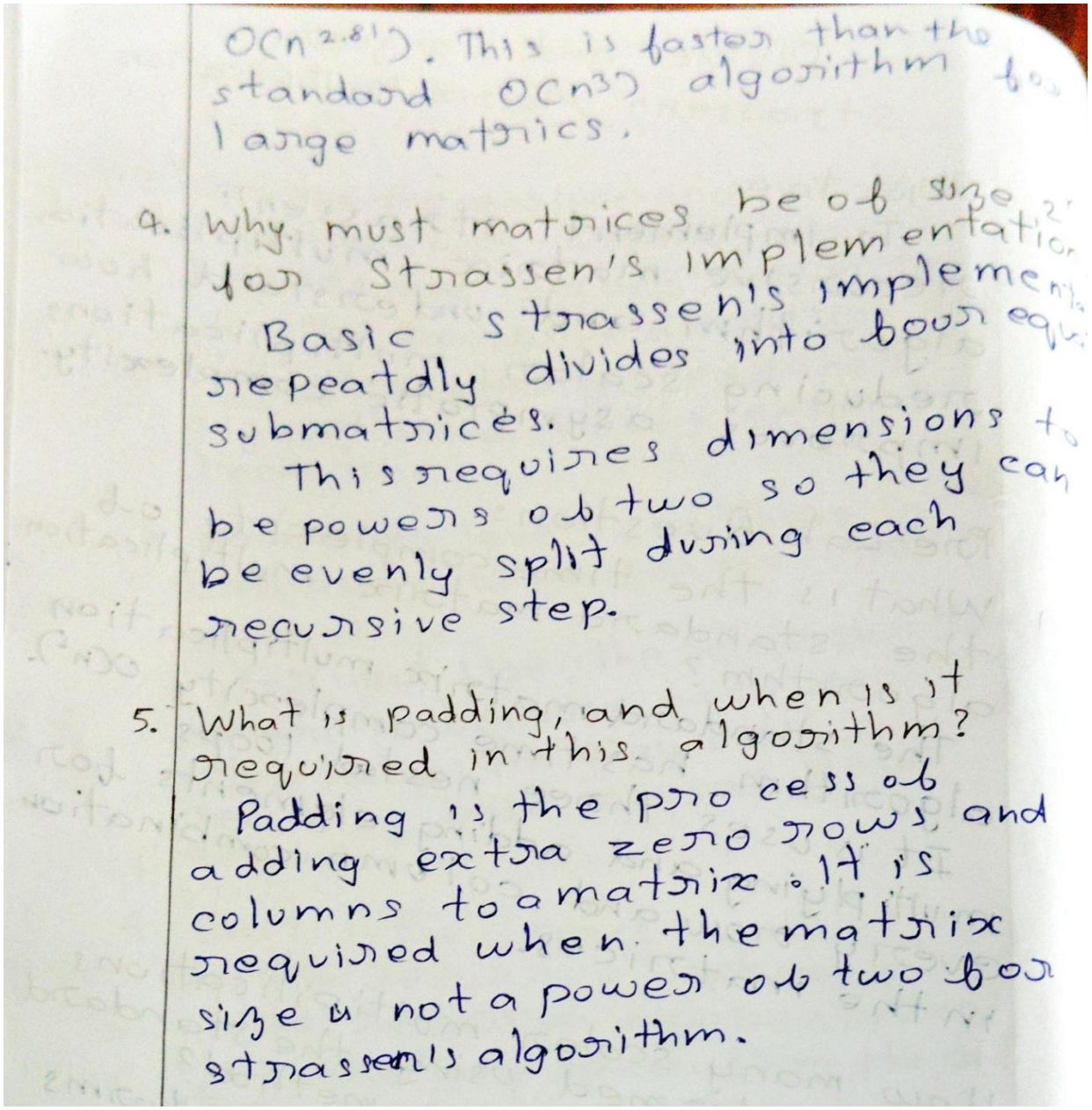
Objective:
To implement strassen's recursive matrix multiplication algorithm and understand how reducing scalar multiplications improves asymptotic complexity.

Pre-Lab Questions

1. What is the time complexity of the standard matrix multiplication algorithm?
The standard matrix multiplication algorithm has time complexity $O(n^3)$ It is uses three nested loops, multiplying and adding elements for every row and column combinations in the matrices.

2. How many scalar multiplications are performed using the standard method vs strassen's method?
The standard method performs 8 multiplications for 2×2 matrices while strassen's method performs only 7 multiplications, reducing the total number of recursive multiplications at each step.

3. What is the exact asymptotic complexity of strassen's algorithm?
It has asymptic complexity $O(n^{\log_2 7})$, which is approximately



$O(n^{2.81})$. This is faster than the standard $O(n^3)$ algorithm for large matrices.

4. Why must matrices be of size 2^n for Strassen's implementation

Basic strassen's implement repeatdly divides into four equ submatrices.

This requires dimensions to be powers of two so they can be evenly split during each recursive step.

5. What is padding, and when is it required in this algorithm?

Padding is the process of adding extra zero rows and columns to a matrix. It is required when the matrix size is not a power of two for strassen's algorithm.

Code:

```
!whoami
!echo 24BAD405 EX06b
```

```
nishanth\nishanth
24BAD405 EX06b
```

```
import numpy as np
import time

def create_random_matrix(n):
    return np.random.randint(-9, 10, size=(n, n))

def matrix_add(A, B):
    return np.add(A, B)

def matrix_subtract(A, B):
    return np.subtract(A, B)

def split_matrix(M):
    n = M.shape[0] // 2
    return (M[:n, :n], M[:n, n:],
            M[n:, :n], M[n:, n:])

def combine_matrix(C11, C12, C21, C22):
    n = C11.shape[0]
    C = np.zeros((2*n, 2*n), dtype=C11.dtype)
    C[:n, :n]   = C11
```

```python
def matmul_naive(A, B):
    n = A.shape[0]
    C = np.zeros((n, n), dtype=A.dtype)
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i,j] += A[i,k] * B[k,j]
    return C
```

```python
scalar_multiplications = 0

def reset_counter():
    global scalar_multiplications
    scalar_multiplications = 0

def get_counter():
    global scalar_multiplications
    return scalar_multiplications

def matmul_strassen(A, B):
    global scalar_multiplications

    n = A.shape[0]

    if n == 1:
        scalar_multiplications += 1
        return A * B
```

```
    A11, A12, A21, A22 = split_matrix(A)
    B11, B12, B21, B22 = split_matrix(B)

    M1 = matmul_strassen(A11 + A22, B11 + B22)
    M2 = matmul_strassen(A21 + A22, B11)
    M3 = matmul_strassen(A11, B12 - B22)
    M4 = matmul_strassen(A22, B21 - B11)
    M5 = matmul_strassen(A11 + A12, B22)
    M6 = matmul_strassen(A21 - A11, B11 + B12)
    M7 = matmul_strassen(A12 - A22, B21 + B22)

    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6

    return combine_matrix(C11, C12, C21, C22)
```

```
reset_counter()
A4 = create_random_matrix(4)
B4 = create_random_matrix(4)

C_naive = matmul_naive(A4, B4)
C_strassen = matmul_strassen(A4, B4)

print("Naive  vs  Strassen  match →", np.allclose(C_naive, C_strassen))
print("Naive multiplications    :", 4**3)
```

```
Naive  vs  Strassen  match → True
Naive multiplications    : 64
Strassen multiplications : 49
```

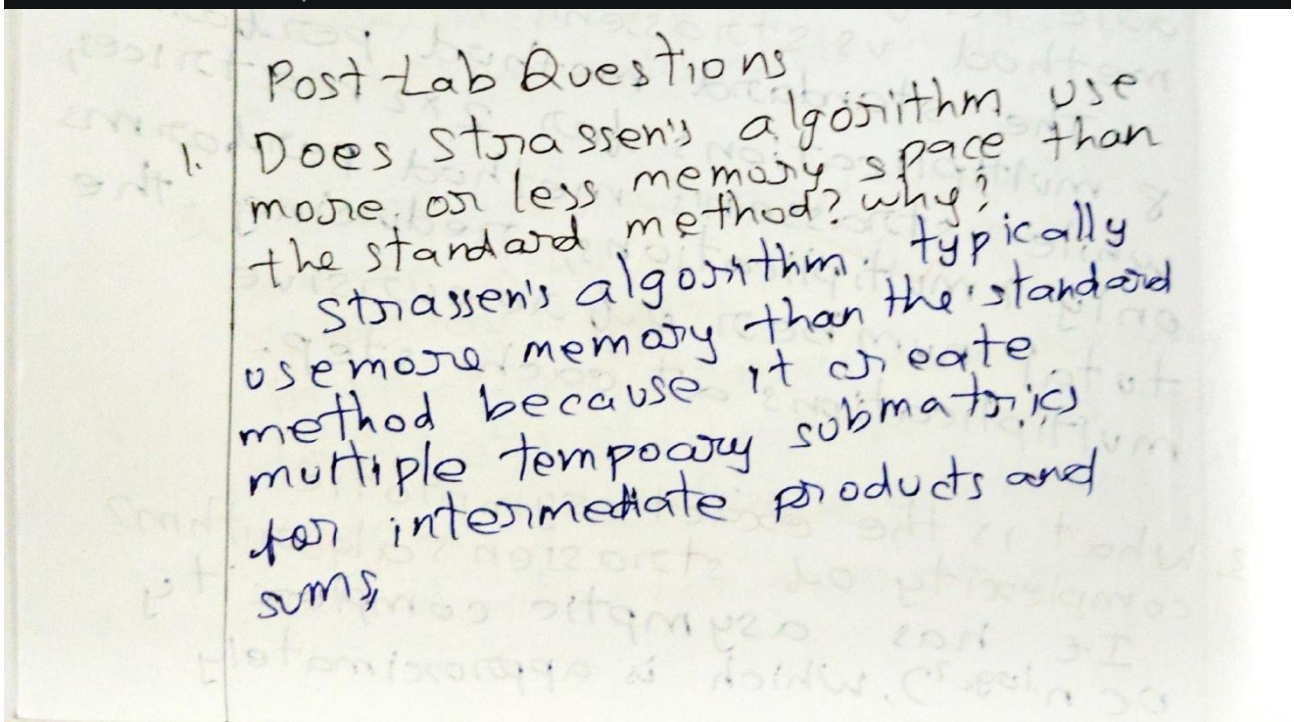
```
    reset_counter()
    A8 = create_random_matrix(8)
    B8 = create_random_matrix(8)

    _ = matmul_naive(A8, B8)
    naive_mult_8 = 8**3

    reset_counter()
    _ = matmul_strassen(A8, B8)
    strassen_mult_8 = get_counter()

    print(f"8×8 naive multiplications    : {naive_mult_8}")
    print(f"8×8 Strassen multiplications : {strassen_mult_8}")
    print(f"Ratio (naive/Strassen)       : {naive_mult_8 / strassen_mult_8:.2f}x")
```

```
8×8 naive multiplications    : 512
8×8 Strassen multiplications : 343
Ratio (naive/Strassen)       : 1.49x
```

```python
def next_power_of_2(x):
    return 1 if x == 0 else 2 ** (x-1).bit_length()

def pad_matrix(M, new_size):
    n = M.shape[0]
    if n == new_size:
        return M
    padded = np.zeros((new_size, new_size), dtype=M.dtype)
    padded[:n, :n] = M
    return padded

def unpad_matrix(M, original_size):
    return M[:original_size, :original_size]

def matmul_strassen_any_size(A, B):
    n = A.shape[0]
    m = B.shape[1]
    if A.shape[1] != B.shape[0]:
        raise ValueError("Matrix dimensions incompatible")

    size = max(n, A.shape[1], m)
    new_size = next_power_of_2(size)

    Ap = pad_matrix(A, new_size)
    Bp = pad_matrix(B, new_size)

    reset_counter()
    Cp = matmul_strassen(Ap, Bp)
```

```python
for size in [3, 5, 6, 7]:
    print(f"\n=== {size}×{size} ===")

    A = create_random_matrix(size)
    B = create_random_matrix(size)

    C_np = np.dot(A, B)

    C_str, mults = matmul_strassen_any_size(A, B)

    print(f"NumPy vs padded-Strassen match → {np.allclose(C_np, C_str, atol=1e-10)}")
    print(f"Strassen scalar multiplications: {mults}")
```

```
=== 3×3 ===
NumPy vs padded-Strassen match → True
Strassen scalar multiplications: 49

=== 5×5 ===
NumPy vs padded-Strassen match → True
Strassen scalar multiplications: 343

=== 6×6 ===
NumPy vs padded-Strassen match → True
Strassen scalar multiplications: 343

=== 7×7 ===
NumPy vs padded-Strassen match → True
Strassen scalar multiplications: 343
```



Post-Lab Questions

1. Does Strassen's algorithm use more or less memory space than the standard method? why?

Strassen's algorithm typically use more memory than the standard method because it create multiple tempoary submatrics for intermediate products and sums;

2. Explain the "cross over point" - why is the standard method often faster than strassen's for small matrix?
The crossover point is the matrix size where strassen becomes faster than classical multiplication. For small matrices, recursion overhead, extra additional and cache effects

3 Mention one significant disadvantage of strassen's algorithm regarding numerical stability
strassen's algorithm amplifies floating point rounding error because it replaces multiplications with many additions and subtraction, causing loss of percision and reduced numerical stability

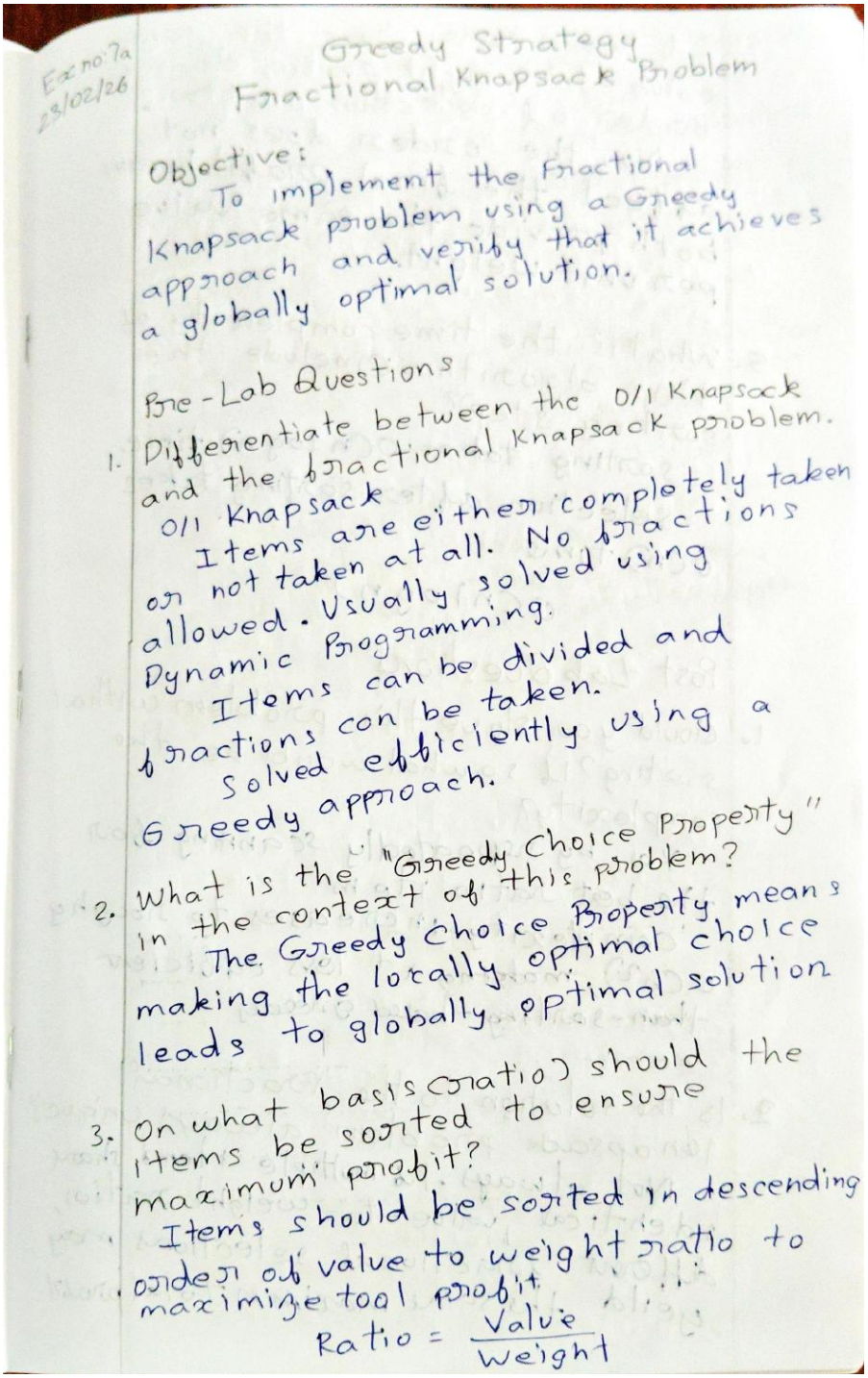
4 How many additional/subtraction in strassen logic?
strassen's method performs 18 matrix additions method performs 18 subtraction per recursion level to compute the seven intermediate products and combine them into the final four result

Description	Max marks	Marks awarded
Pre Lab	5	5
In Lab	10	10
Post Lab	5	5
Viva	10	10
Total	30	30
	Faculty signature	

EX NO:07 **23.02.2026**

FRACTIONAL KNAPSACK, ACTIVITY SELECTION AND HUFFMAN CODING

EX NO: 07a GREEDY STRATEGY: FRACTIONAL KNAPSACK PROBLEM



Ex no:7a
23/02/26

Greedy Strategy
Fractional Knapsack Problem

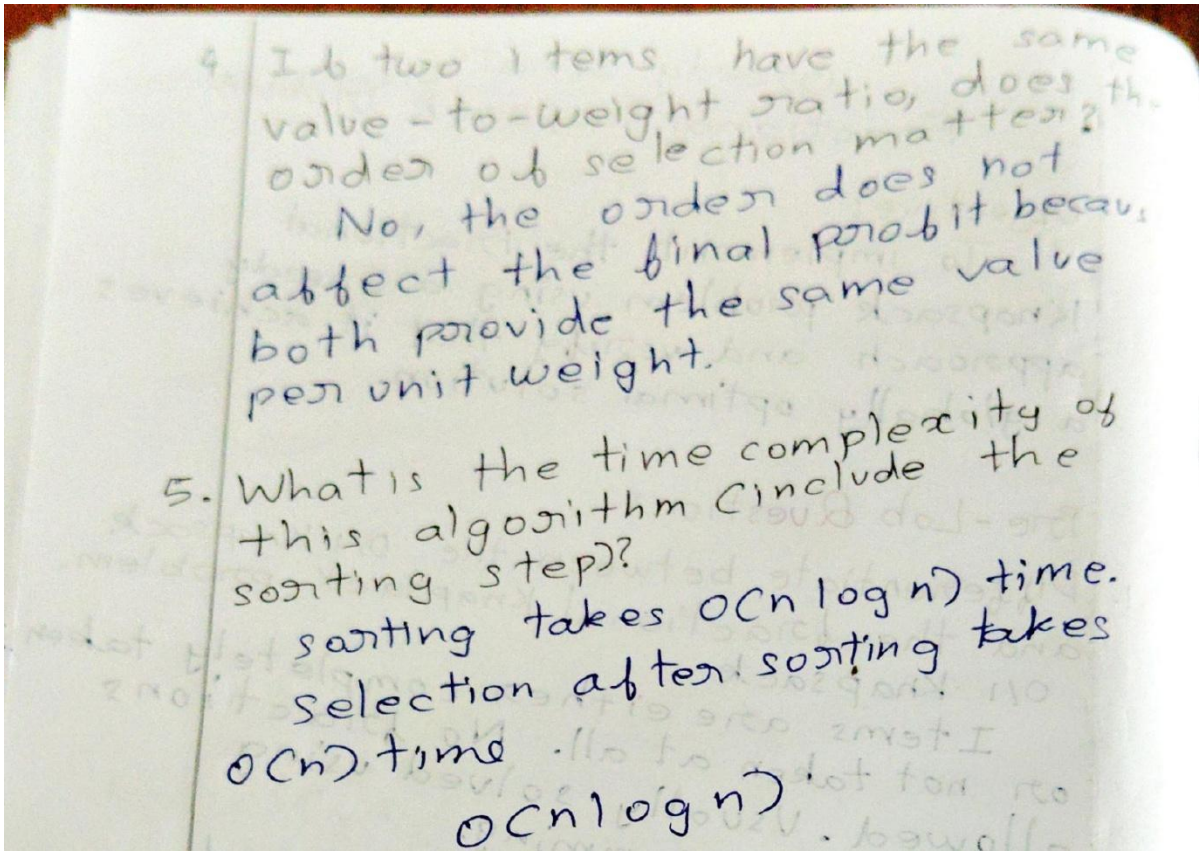
Objective:
To implement the Fractional Knapsack problem using a Greedy approach and verify that it achieves a globally optimal solution.

Pre-Lab Questions

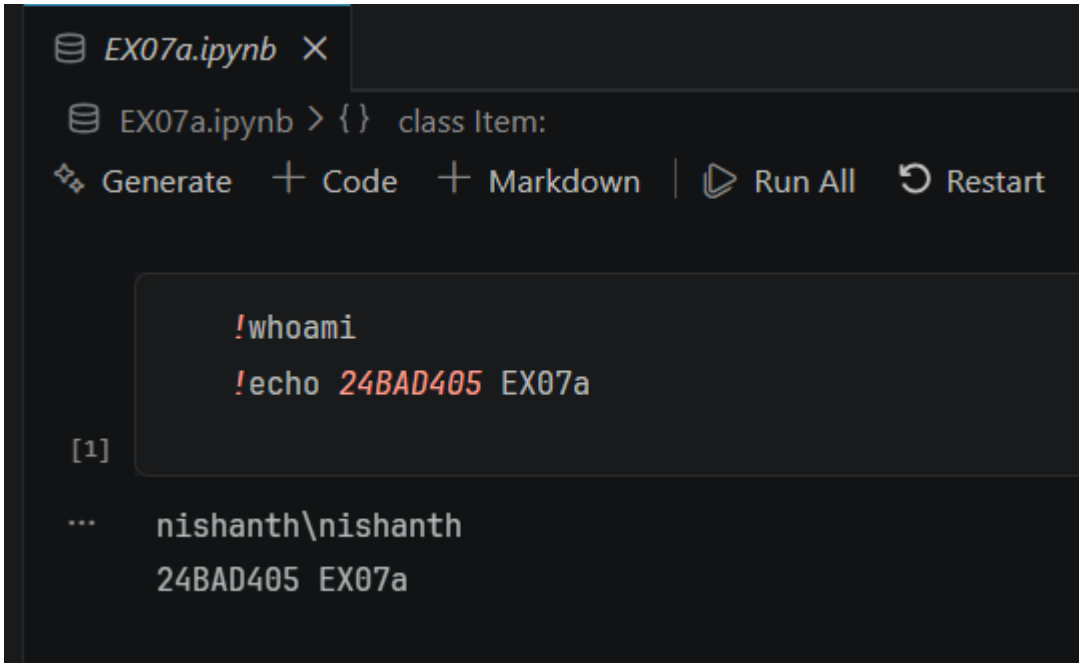
1. Differentiate between the 0/1 Knapsack and the fractional Knapsack problem.
0/1 Knapsack
Items are either completely taken or not taken at all. No fractions allowed. Usually solved using Dynamic Programming.
Items can be divided and fractions can be taken.
Solved efficiently using a Greedy approach.

2. What is the "Greedy Choice Property" in the context of this problem?
The Greedy Choice Property means making the locally optimal choice leads to globally optimal solution.

3. On what basis (ratio) should the items be sorted to ensure maximum profit?
Items should be sorted in descending order of value to weight ratio to maximize tool profit.
Ratio = Value / Weight


4. If two items have the same value-to-weight ratio, does the order of selection matter?
No, the order does not affect the final profit because both provide the same value per unit weight.
5. What is the time complexity of this algorithm (include the sorting step)?
sorting takes O(n log n) time.
selection after sorting takes O(n) time.
O(n log n)

Code:


EX07a.ipynb
EX07a.ipynb > { } class Item:
Generate + Code + Markdown | Run All Restart
!whoami
!echo 24BAD405 EX07a
[1]
nishanth\nishanth
24BAD405 EX07a

```python
class Item:
    def __init__(self, id: int, value: float, weight: float):
        self.id = id
        self.value = value
        self.weight = weight
        self.ratio = value / weight if weight != 0 else 0

    def __repr__(self):
        return f"Item{self.id}(v={self.value}, w={self.weight}, r={self.ratio:.3f})"


def fractional_knapsack(items: list[Item], capacity: float):
    items_sorted = sorted(items, key=lambda x: x.ratio, reverse=True)

    total_value = 0.0
    current_capacity = capacity

    steps = []

    for item in items_sorted:
        if current_capacity <= 0:
            break

        if item.weight <= current_capacity:
            weight_taken = item.weight
            value_added = item.value
            fraction = 1.0
        else:
            fraction = current_capacity / item.weight
            value_added = item.value * fraction

        total_value += value_added
        current_capacity -= weight_taken

        steps.append({
            'item_id': item.id,
            'weight_taken': round(weight_taken, 2),
            'value_added': round(value_added, 2),
            'fraction': round(fraction, 3),
            'remaining_capacity': round(current_capacity, 2)
        })

    return total_value, steps


def print_knapsack_steps(total_value, steps, original_capacity):
    print(f"\n{'Item':<6} {'Weight Taken':<13} {'Value Added':<12} {'Fraction':<10} {'Remaining Capacity':<18} {'Cumulative Value':<16}")
    print("-" * 80)

    cum_value = 0.0
    for step in steps:
        cum_value += step['value_added']
        print(f"{step['item_id']:<6} "
              f"{step['weight_taken']:<13.2f} "
              f"{step['value_added']:<12.2f} "
              f"{step['fraction']:<10.3f} "
              f"{step['remaining_capacity']:<18.2f} "
              f"{cum_value:<16.2f}")
```

Post Lab Questions

1. Could you slove this problem without sorting? If so, what would be the complexity?

Yes, by repeatedly scanning for the best ratio item

complexity increases to roughly $O(n^2)$, making it less efficient than sorting-based greedy

2. Is the solution to the Fractional Knapsack problem always unique?

Not always. If multiple items share identical value-to-weight ratio, different fractional selections may yeild the same maximum total profit

3 Identify a real life application for the fractional knapsack logic

Bandwidth allocation in networks where partial resource can be assigned proportionally to maximize throughput under limited capacity.

4 If you sorted by shortest Duration instead of Finish Time, would you always get the optimal result Explain

No shorter duration may block future compatible activities.

Sorting by earliest finish time guarantees maximum non-overlapping activities

EX NO: 07b GREEDY STRATEGY: ACTIVITY SELECTION PROBLEM

Ex no: 7b
25.02.26

Greedy Strategy
Activity Selection Problem

Objective:

To implement the activity Selection problem to find the maximum number of non-overlapping activities that can be performed by a single resource.

Pre Lab Questions:

1. Define a "compatible" activity in this problem.

Two activities are compatible if the start time of one activity is greater than or equal to the finish time of the previously selected activity.

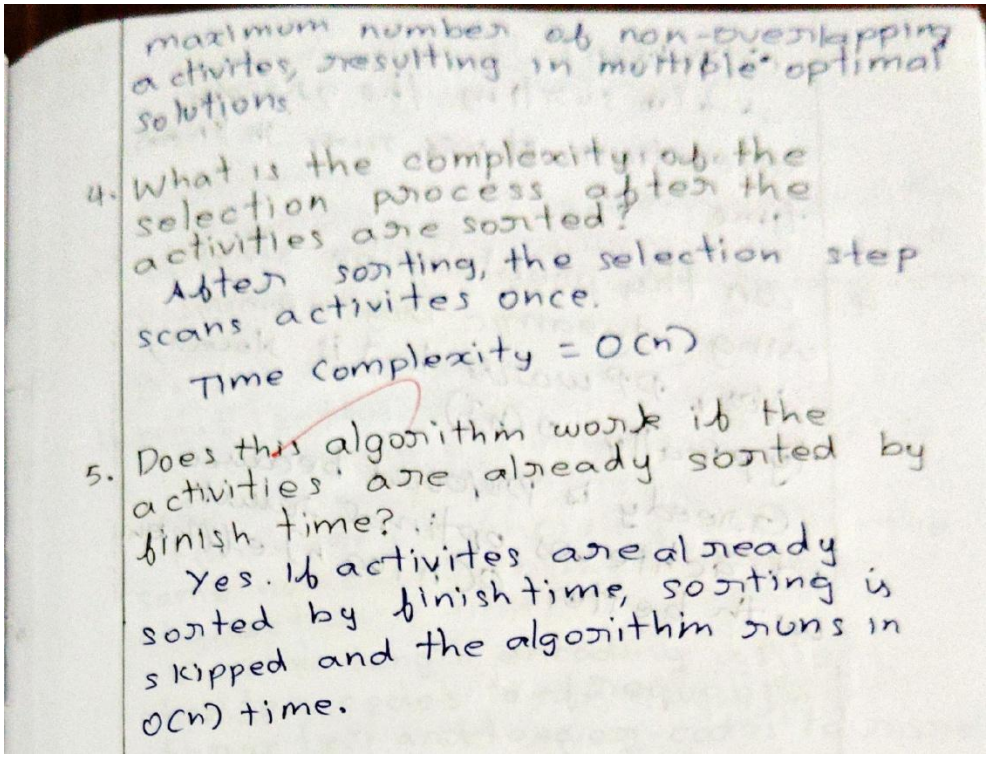
2. The Greedy strategy suggests sorting activites by their Finish Time. Why not start time?

Sorting by finish time allows us to select activities that end earliest, leaving maximum time for remaining activites.

Sorting by start time may block better future choices.

3. Is it possible to have multiple optimal set of activities?

Yes, Different combinations of activites can produce the same


maximum number of non-overlapping activites, resulting in multiple optimal solutions

4. What is the complexity of the selection process after the activities are sorted?
After sorting, the selection step scans activites once.
Time complexity = O(n)

5. Does this algorithm work if the activities are already sorted by finish time?
Yes. If activites are already sorted by finish time, sorting is skipped and the algorithm runs in O(n) time.

Code:

```
!whoami
!echo 24BAD405 EX07b
```

```
nishanth\nishanth
24BAD405 EX07b
```

```
class Activity:
    def __init__(self, id: int, start: int, finish: int):
        self.id = id
        self.start = start
        self.finish = finish
        self.duration = finish - start
    def __repr__(self):
        return f"A{self.id}({self.start}→{self.finish})"


def activity_selection_finish_time(activities):
    sorted_acts = sorted(activities, key=lambda x: x.finish)

    selected = []
    last_finish = -1

    for act in sorted_acts:
        if act.start ⩾ last_finish:
            selected.append(act)
```

(method) def __repr__(self: Self@Activity) → str

```python
def activity_selection_shortest_duration(activities):
    sorted_acts = sorted(activities, key=lambda x: x.duration)

    selected = []
    last_finish = -1

    for act in sorted_acts:
        if act.start ⩾ last_finish:
            selected.append(act)
            last_finish = act.finish

    return selected


def print_selection(title, selected):
    print(f"\n{title}")
    print("-" * 50)
    if not selected:
        print("No activities selected.")
    else:
        ids = [a.id for a in selected]
        print(f"Selected {len(selected)} activities: {ids}")
        print("Activities:")
        for a in selected:
            print(f"  A{a.id}:  {a.start:2d} → {a.finish:2d}  (duration {a.duration})")
    print()
```

```
    activities1 = [
        Activity(1, 1,  4),
        Activity(2, 3,  5),
        Activity(3, 0,  6),
        Activity(4, 5,  7),
        Activity(5, 3,  9),
        Activity(6, 5,  9),
        Activity(7, 6, 10),
        Activity(8, 8, 11),
        Activity(9, 8, 12),
        Activity(10,2,14)
    ]

    print("== Test Case 1 - Classic Example ==")
    selected_finish = activity_selection_finish_time(activities1)
    print_selection("→ Sorted by FINISH TIME (OPTIMAL)", selected_finish)


    activities2 = [
        Activity(1, 1, 10),
        Activity(2, 2,  6),
        Activity(3, 5,  9),
        Activity(4, 7, 11),
        Activity(5, 10,12)
    ]

    print("\n== Test Case 2 - 5 Activities with overlaps ==")
    selected_finish2 = activity_selection_finish_time(activities2)
    print_selection("→ Sorted by FINISH TIME", selected_finish2)
```

```python
    print_selection("→ Correct: Sorted by FINISH TIME", selected_finish_c)
    print_selection("→ WRONG: Sorted by SHORTEST DURATION", selected_short_c)

    print("→ Conclusion: Sorting by duration selected fewer activities!")
```

```
=== Test Case 1 - Classic Example ===

→ Sorted by FINISH TIME (OPTIMAL)
------------------------------------------------
Selected 3 activities: [1, 4, 8]
Activities:
  A1:   1 →  4  (duration 3)
  A4:   5 →  7  (duration 2)
  A8:   8 → 11  (duration 3)


=== Test Case 2 - 5 Activities with overlaps ===

→ Sorted by FINISH TIME
------------------------------------------------
Selected 2 activities: [2, 4]
Activities:
  A2:   2 →  6  (duration 4)
  A4:   7 → 11  (duration 4)
```

```
═══ Task 3 - Counterexample showing shortest duration is NOT optimal ═══

→ Correct: Sorted by FINISH TIME
------------------------------------------------
Selected 4 activities: [2, 3, 4, 5]
Activities:
  A2:   2 →  4  (duration 2)
  A3:   4 →  6  (duration 2)
  A4:   6 →  8  (duration 2)
  A5:   8 → 10  (duration 2)


→ WRONG: Sorted by SHORTEST DURATION
------------------------------------------------
Selected 4 activities: [2, 3, 4, 5]
Activities:
  A2:   2 →  4  (duration 2)
  A3:   4 →  6  (duration 2)
  A4:   6 →  8  (duration 2)
  A5:   8 → 10  (duration 2)

→ Conclusion: Sorting by duration selected fewer activities!
```

Post-Lab Questions

1. What happens if the capacity of the knapsack is greater than the total weight of all available item?

Activity Selection maximizes compatible tasks using one resource, whereas interval partitioning minimizes the number of resources needed to schedule

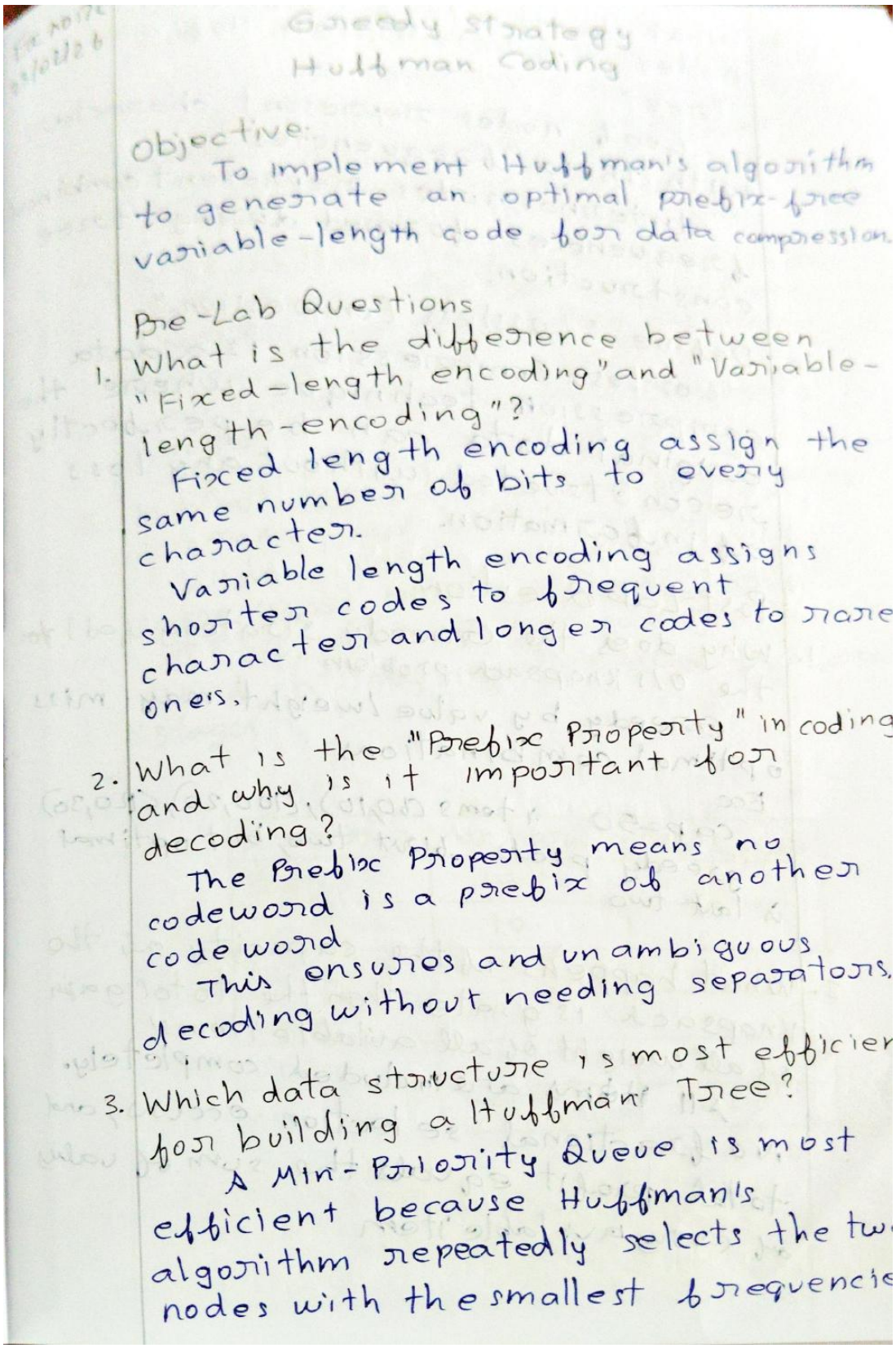
2. What is the time complexity if the input finish times are not sorted?

sorting domainates the runtime, giving overall complexity

O(n log n).
Due to sorting the greedy selection itself runs in linear time

4. Can this problem be solved using dynamic Programming?
Yes, DP works but is slower, typically $O(n^2)$.
Greedy is preferred because it achieves optimal result with better $O(n \log n)$ efficiency.

EX NO:07c GREEDY STRATEGY: HUFFMAN CODING



Greedy Strategy
Huffman Coding

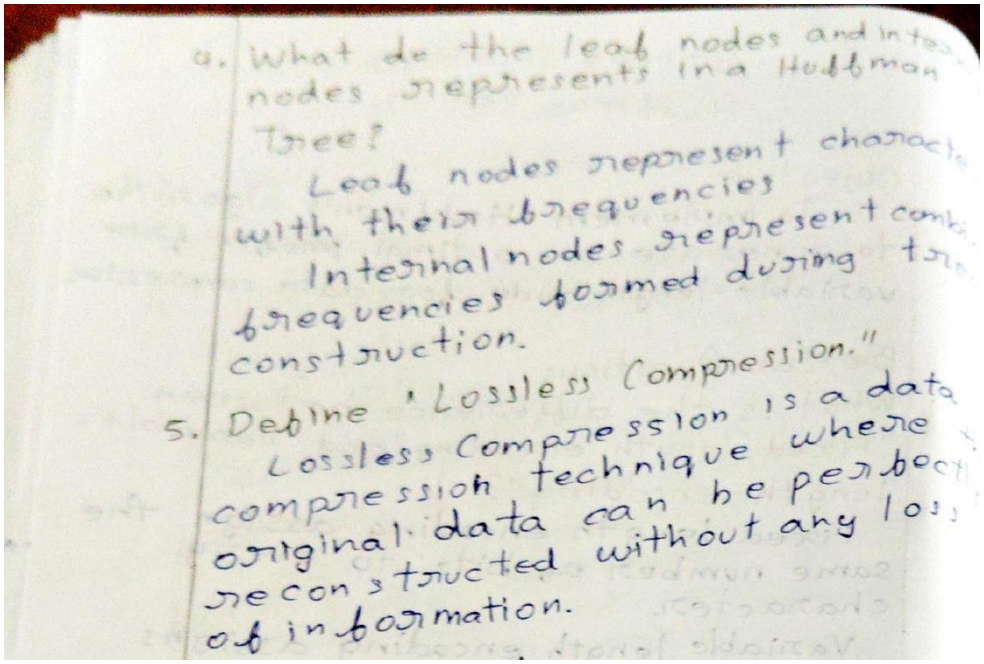
Objective:
To implement Huffman's algorithm to generate an optimal prefix-free variable-length code for data compression.

Pre-Lab Questions

1. What is the difference between "Fixed-length encoding" and "Variable-length-encoding"?
Fixed length encoding assign the same number of bits to every character.
Variable length encoding assigns shorter codes to frequent character and longer codes to rare ones.

2. What is the "Prefix Property" in coding and why is it important for decoding?
The Prefix Property means no codeword is a prefix of another codeword
This ensures and unambiguous decoding without needing separators.

3. Which data structure is most efficier for building a Huffman Tree?
A Min-Priority Queue is most efficient because Huffmanis algorithm repeatedly selects the tw nodes with the smallest frequencie


a. What do the leaf nodes and inte
nodes represents in a Huffman
Tree?
Leaf nodes represent charact
with their frequencies
Internal nodes represent comb
frequencies formed during tre
construction.
5. Define 'Lossless Compression."
Lossless Compression is a data
compression technique where
original data can be perfect
reconstructed without any los
of information.

Code:

```
!whoami
!echo 24BAD405 EX07c
```

```
nishanth\nishanth
24BAD405 EX07c
```

Generate + Code

```
import heapq
from collections import Counter

class Node:
    def __init__(self, char=None, freq=0, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):
        return self.freq < other.freq


def build_frequency_table(text):
    return Counter(text)
```

```python
def build_frequency_table(text):
    return Counter(text)


def build_huffman_tree(freq_table):
    priority_queue = []
    for char, freq in freq_table.items():
        heapq.heappush(priority_queue, Node(char, freq))

    while len(priority_queue) > 1:
        left = heapq.heappop(priority_queue)
        right = heapq.heappop(priority_queue)
        merged = Node(None, left.freq + right.freq, left, right)
        heapq.heappush(priority_queue, merged)

    return priority_queue[0] if priority_queue else None



def generate_huffman_codes(node, current_code="", codes=None):
    if codes is None:
        codes = {}

    if node is None:
        return codes

    if node.char is not None:
        if current_code == "":
```

```python
def calculate_compression_stats(text, codes):
    if not text:
        return 0, 0, 0.0

    ascii_bits = len(text) * 8

    huffman_bits = 0
    for char in text:
        huffman_bits += len(codes.get(char, ""))

    if ascii_bits == 0:
        compression_ratio = 0.0
    else:
        compression_ratio = (1 - huffman_bits / ascii_bits) * 100

    return ascii_bits, huffman_bits, compression_ratio



text = "BEEP BOOP BEER"

freq = build_frequency_table(text)
root = build_huffman_tree(freq)
huffman_codes = generate_huffman_codes(root)

ascii_total, huffman_total, savings_pct = calculate_compression_stats(text, huffman_codes)

print(f"Original text: {text}")
print(f"Length: {len(text)} characters\n")
```

```
    print("Huffman Codes:")
    for char, code in sorted(huffman_codes.items()):
        display_char = repr(char) if char.isspace() else char
        print(f"  {display_char:>3} : {code}")

    print("\nCompression Statistics:")
    print(f"  ASCII (8-bit)         : {ascii_total} bits")
    print(f"  Huffman encoded       : {huffman_total} bits")
    print(f"  Space saved           : {savings_pct:.2f}%")
    print(f"  Compression ratio     : {huffman_total / ascii_total:.3f}")
```

(variable) huffman_total: int

```
Original text: BEEP BOOP BEER
Length: 14 characters

Huffman Codes:
  ' ' : 100
    B : 00
    E : 11
    O : 101
    P : 011
    R : 010

Compression Statistics:
  ASCII (8-bit)         : 112 bits
  Huffman encoded       : 35 bits
  Space saved           : 68.75%
  Compression ratio     : 0.312
```

Post-Lab Question

1. Why does the Greedy strategy fail for the 0/1 knapsack problem

Greedy by value/weight may miss optimal combinations

Ex

cap=50 items (60,10), (100,20), (120,30)

greedy picks first two, but optimal is last two

2. What happens if the capacity of the knapsack is grater than the totol graph of all weight of all avilable items?

All items are included completely. No fractional selection occurs, and total profit equals the sum of value of every available item

3. What is the time complexity of building the Huffman tree for N unique characters using a min priority queue, construction requires repeated extractions and insertions, resulting in overall time complexity of O(n log n)

4. If character 'F' has code 01, can another character have the code 011? Why or why not?
No, Huffman codes follow the prefix property
no code word can be a prefix of another

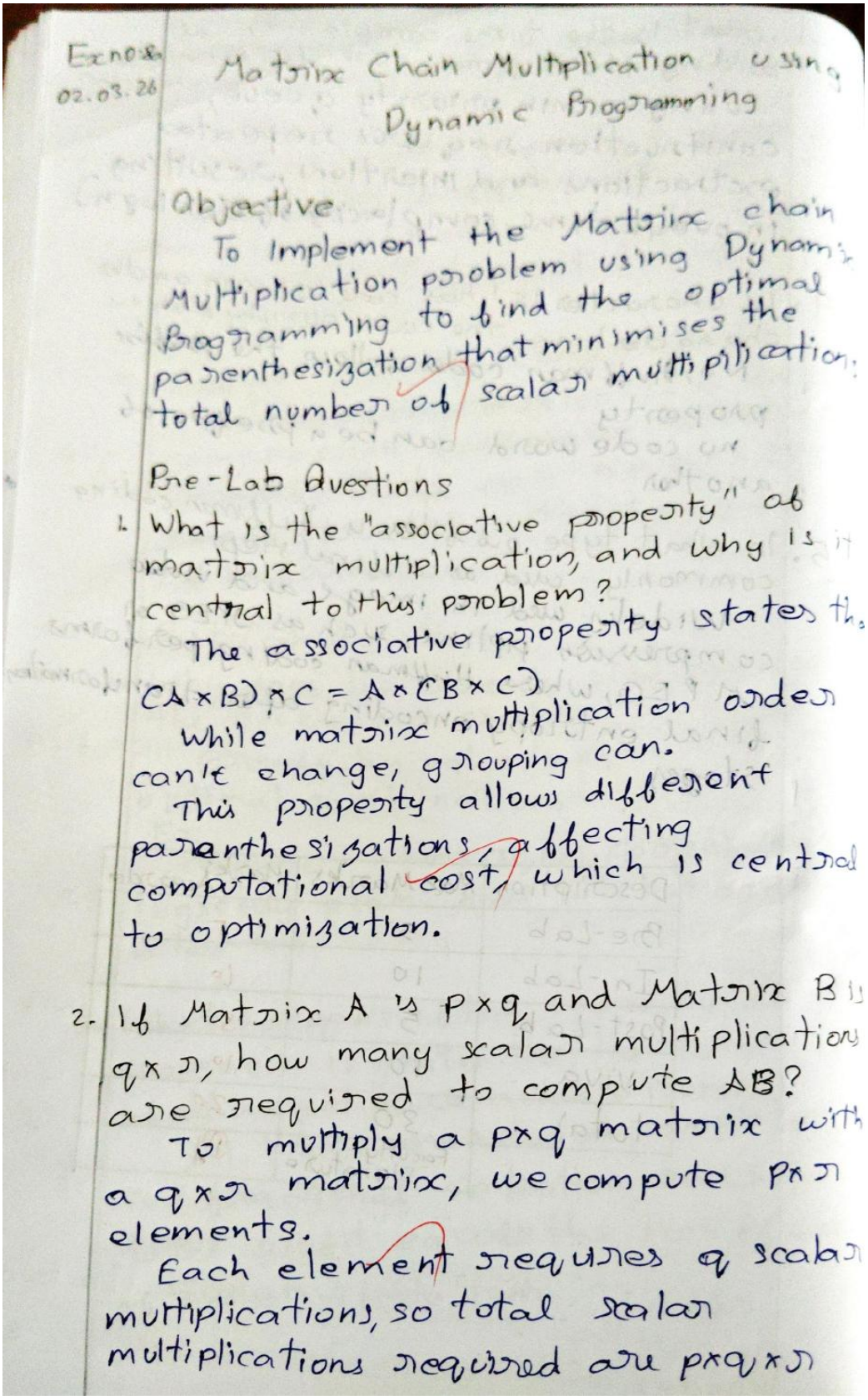
5. In what type of files is Huffman coding commonly used as a final step?
Widely used in image and video compression pipline, such as JPEG or MPEG, where Huffman coding performs final entropy encoding after transform stages.

Description	Max Marks	Marks Awarded
Pre-Lab	5	5
In-Lab	10	10
Post-Lab	5	5
Viva	10	10
total	30	30
	Faculty Signature	

EX NO: 08 **…02.03.2026**

MATRIX CHAIN MULTIPLICATION AND LCS

EX NO: 08a MATRIX CHAIN MULTIPLICATION USING DYNAMIC PROGRAMMING



Ex no:8a
02.03.26

Matrix Chain Multiplication using Dynamic Programming

Objective

To Implement the Matrix chain Multiplication problem using Dynamic Programming to find the optimal parenthesization that minimises the total number of scalar multiplication.

Pre-Lab Questions

1. What is the "associative property" of matrix multiplication, and why is it central to this problem?

The associative property states that $(A \times B) \times C = A \times (B \times C)$

While matrix multiplication order can't change, grouping can.

This property allows different parenthesizations, affecting computational cost, which is central to optimization.

2. If Matrix A is $p \times q$ and Matrix B is $q \times r$, how many scalar multiplications are required to compute AB?

To multiply a $p \times q$ matrix with a $q \times r$ matrix, we compute $p \times r$ elements.

Each element requires q scalar multiplications, so total scalar multiplications required are $p \times q \times r$

3 Define 'Optimal Substructure' in the context of the MCM problem

Optimal substructure means the optimal solution to the full matrix chain multiplication problem contains optimal solutions to its subproblems. If a penthesisation is optimal, its subchains must also be optimally parenthesized

4 Why is the Brute Force approach inefficient as the number of matrices (n) increases?

The number of possible parenthesizations grows exponentially with n.

As n increases, possibilites explode combinatorially, making brute force computationally impractical for even moderately larger n.

5. What do the two tables, often named m (cost) and s (split points), represent in this algorithm?

The m table stores the minium scalar multiplying matrices from i to j. The s table records the index k where the optimal split occurs, enabling reconstruction of optimal paranthesize

Code:

```
!whoami
!echo 24BAD405 EX08a
```

```
nishanth\nishanth
24BAD405 EX08a
```

Task 1

```
def is_compatible(matrices):
    """
    matrices: list of tuples [(r1,c1), (r2,c2), ...]
    Returns True if multiplication is possible
    """
    for i in range(len(matrices) - 1):
        if matrices[i][1] != matrices[i+1][0]:
            return False
    return True
```

```
matrices = [(10, 30), (30, 5), (5, 60)]

if is_compatible(matrices):
    print("Matrices are compatible.")
else:
    print("Not compatible.")
```

```
Matrices are compatible.
```

Task 2

```
import sys

def matrix_chain_recursive(p, i, j):
    if i == j:
        return 0

    min_cost = sys.maxsize

    for k in range(i, j):
        cost = (matrix_chain_recursive(p, i, k)
                + matrix_chain_recursive(p, k+1, j)
                + p[i-1] * p[k] * p[j])
```

```
            if cost < min_cost:
                min_cost = cost

        return min_cost
```

```
p = [10, 30, 5, 60]
n = len(p) - 1

print("Minimum cost (Recursive):",
      matrix_chain_recursive(p, 1, n))
```

```
Minimum cost (Recursive): 4500
```

Task 3 & 4

```
def matrix_chain_order(p):
    n = len(p) - 1

    # Create M and S tables
    M = [[0]*(n+1) for _ in range(n+1)]
    S = [[0]*(n+1) for _ in range(n+1)]

    # L = chain length
    for L in range(2, n+1):
```

```
    # L = chain length
    for L in range(2, n+1):
        for i in range(1, n-L+2):
            j = i + L - 1
            M[i][j] = sys.maxsize

            for k in range(i, j):
                cost = (M[i][k] + M[k+1][j]
                        + p[i-1]*p[k]*p[j])

                if cost < M[i][j]:
                    M[i][j] = cost
                    S[i][j] = k

    return M, S
```

```
p = [10, 30, 5, 60]
M, S = matrix_chain_order(p)

print("Minimum cost (DP):", M[1][len(p)-1])
```

```
Minimum cost (DP): 4500
```

Task 5

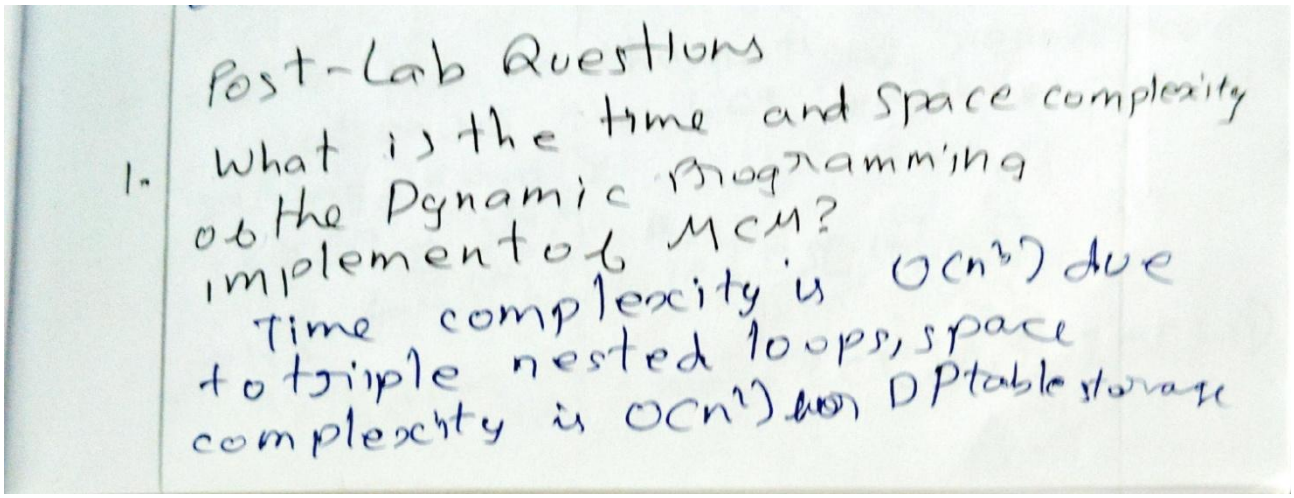
```
def print_optimal_parens(S, i, j):
    if i == j:
        return f"A{i}"
    else:
        k = S[i][j]
        left = print_optimal_parens(S, i, k)
        right = print_optimal_parens(S, k+1, j)
        return f"({left}{right})"
```

[7]

```
p = [10, 30, 5, 60]
M, S = matrix_chain_order(p)

print("Optimal Parenthesization:",
      print_optimal_parens(S, 1, len(p)-1))
```

[8]

```
Optimal Parenthesization: ((A1A2)A3)
```



Post-Lab Questions

1. What is the time and space complexity of the Dynamic programming implemented of MCM?

Time complexity is $O(n^3)$ due to triple nested loops, space complexity is $O(n^2)$ for DP table storage

Compare the results: How much did optimal parenthesization reduce the cost compared to simple left-to-right multiplication for your test case?

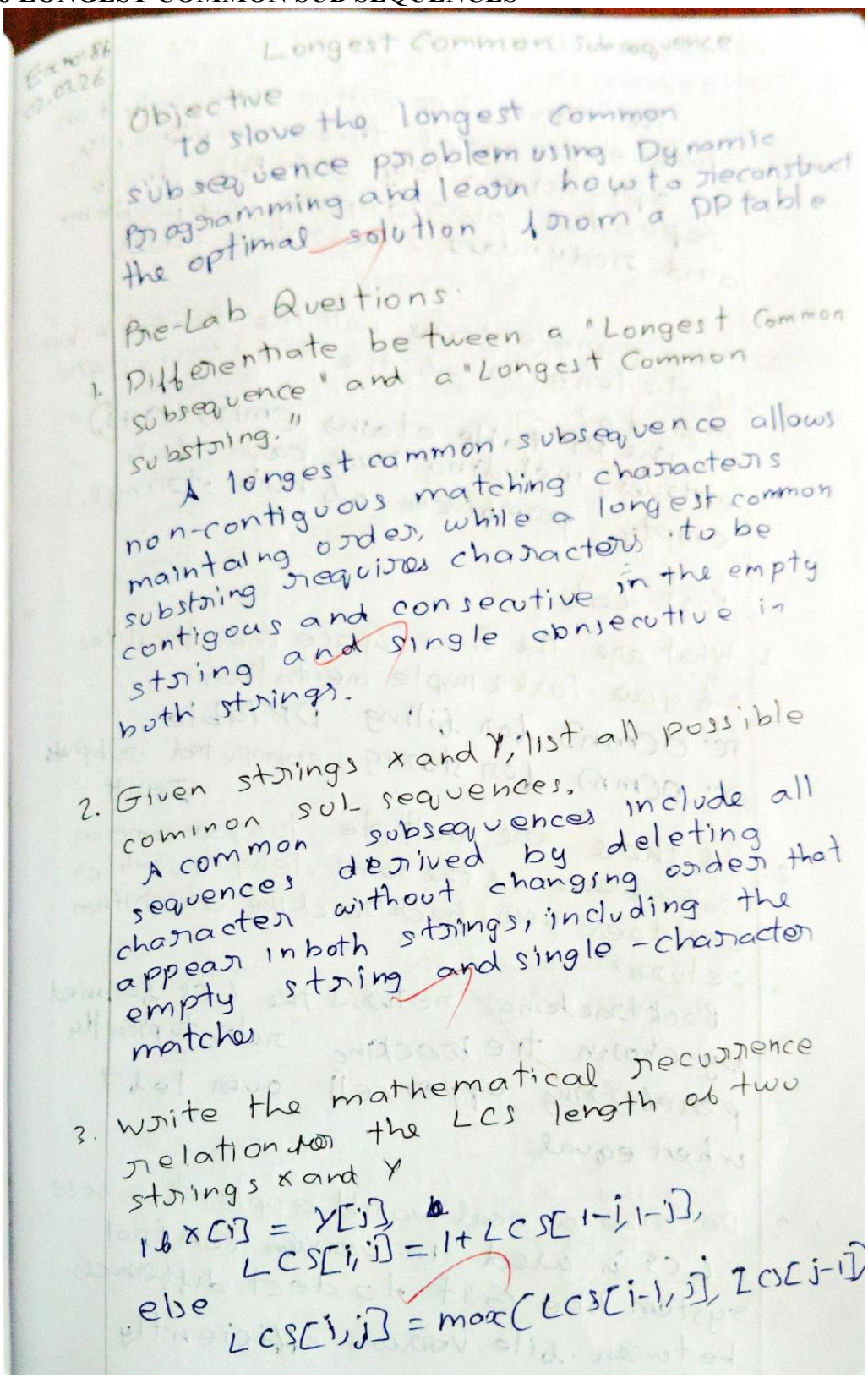
optimal parenthesization significantly reduced scalar multiplications, often saving thousands or millions of operations compared to left to right evalutaion

How does this problem demonstrate the "Overlapping Subproblem" property? Are the same subchain multiplication results repeatedly computed in naive recursion, showing overlapping subproblems reused in dynamic programming

Can this problem be solved using a Greedy strategy? Why or why not?

No, greedy fails because locally minimal multiplication cost does not guarantee globally optimal parenthesization.

EX NO:08 LONGEST COMMON SUB SEQUENCES



Ex no: 8
02.04.26

Longest Common Subsequence

Objective
to slove the longest common subsequence problem using Dynamic programming and learn how to reconstruct the optimal solution from a DP table

Pre-Lab Questions:

1. Differentiate between a "Longest Common Subsequence" and a "Longest Common substring."

A longest common subsequence allows non-contiguous matching characters maintaing order, while a longest common substring requires characters to be contigous and consecutive in the empty string and single consecutive in both strings.

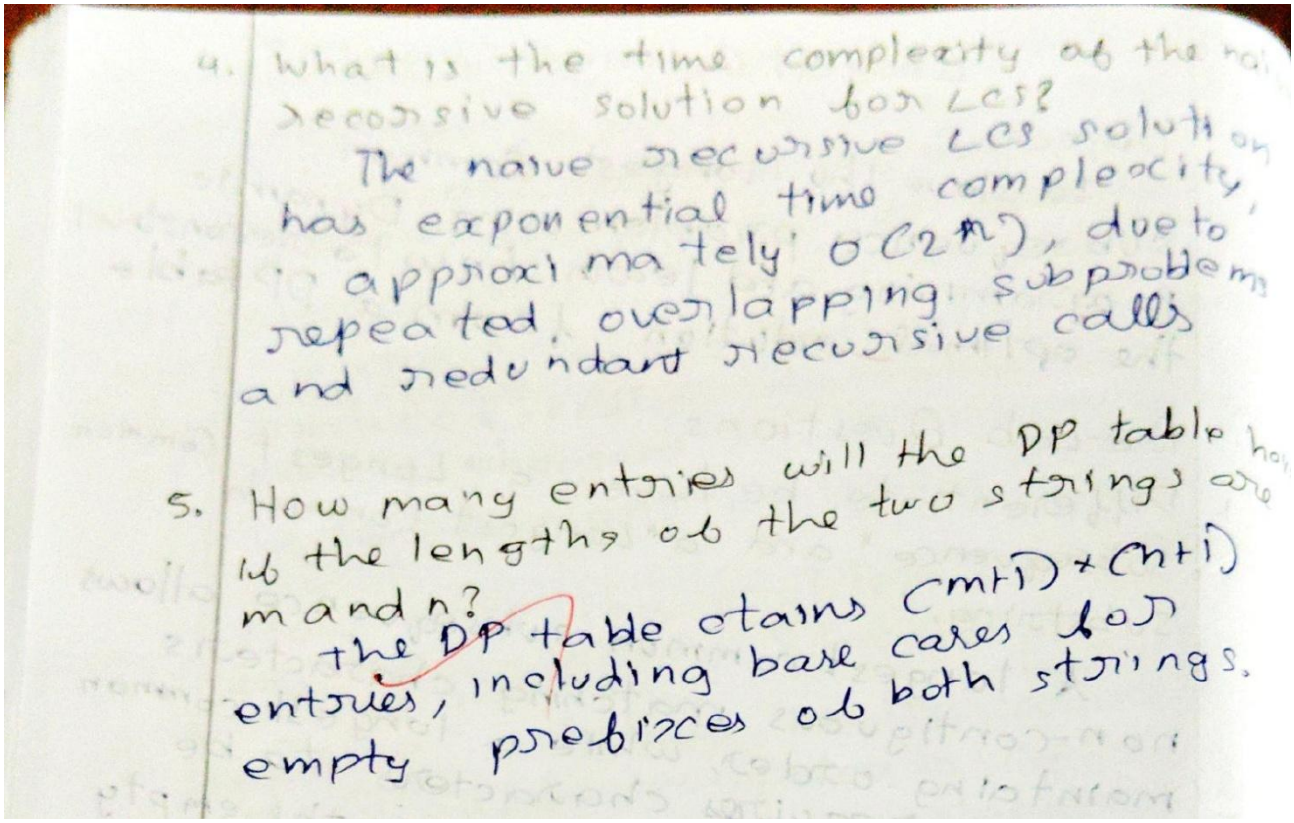
2. Given strings x and Y, list all possible common sub sequences.

A common subsequences include all sequences derived by deleting character without changing order that appear in both strings, including the empty string and single-character matches

3. Write the mathematical recurrence relation for the LCS length of two strings x and Y

if x[i] = Y[j]
LCS[i, j] = 1 + LCS[i-1, j-1],

else
LCS[i, j] = max(LCS[i-1, j], LCS[j-1]



4. What is the time complexity of the naive recursive solution for LCS?

The naive recursive LCS solution has exponential time complexity, approximately O(2^n), due to repeated, overlapping subproblems and redundant recursive calls

5. How many entries will the DP table have if the lengths of the two strings are m and n?

the DP table otains (m+1) * (n+1) entries, including base cases for empty prefixes of both strings.

Code:

```
!whoami
!echo 24BAD405 EX08b
```

```
nishanth\nishanth
24BAD405 EX08b
```

Task 1

```
def lcs_recursive(X, Y, m, n):
    if m == 0 or n == 0:
        return 0

    if X[m-1] == Y[n-1]:
        return 1 + lcs_recursive(X, Y, m-1, n-1)

    return max(lcs_recursive(X, Y, m, n-1),
               lcs_recursive(X, Y, m-1, n))
```

```
X = "AGGTABCDXY"
Y = "GXTXAYBCDZ"

print("LCS Length (Recursive):",
      lcs_recursive(X, Y, len(X), len(Y)))
```

```
LCS Length (Recursive): 6
```

Task 2

```python
def lcs_dp(X, Y):
    m = len(X)
    n = len(Y)

    # Create DP table
    L = [[0]*(n+1) for _ in range(m+1)]

    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                L[i][j] = 1 + L[i-1][j-1]
            else:
                L[i][j] = max(L[i-1][j], L[i][j-1])

    return L
```

```python
L = lcs_dp(X, Y)
print("LCS Length (DP):", L[len(X)][len(Y)])
```

```
LCS Length (DP): 6
```

Task 3

```
def lcs_with_direction(X, Y):
    m = len(X)
    n = len(Y)

    L = [[0]*(n+1) for _ in range(m+1)]
    direction = [[""]*(n+1) for _ in range(m+1)]

    for i in range(1, m+1):
        for j in range(1, n+1):
            if X[i-1] == Y[j-1]:
                L[i][j] = 1 + L[i-1][j-1]
                direction[i][j] = "D"   # Diagonal
            elif L[i-1][j] >= L[i][j-1]:
                L[i][j] = L[i-1][j]
                direction[i][j] = "U"   # Up
            else:
                L[i][j] = L[i][j-1]
                direction[i][j] = "L"   # Left

    return L, direction
```

Task 4

Generate

```
def print_lcs(direction, X, i, j):
    if i == 0 or j == 0:
        return ""

    if direction[i][j] == "D":
        return print_lcs(direction, X, i-1, j-1) + X[i-1]
    elif direction[i][j] == "U":
        return print_lcs(direction, X, i-1, j)
    else:
        return print_lcs(direction, X, i, j-1)
```

```
L, direction = lcs_with_direction(X, Y)

lcs_string = print_lcs(direction, X, len(X), len(Y))

print("LCS Length:", L[len(X)][len(Y)])
print("LCS String:", lcs_string)
```

```
LCS Length: 6
LCS String: GTABCD
```

Task 5

```
def lcs_space_optimized(X, Y):
    # Ensure Y is smaller string
    if len(X) < len(Y):
        X, Y = Y, X

    previous = [0]*(len(Y)+1)
    current = [0]*(len(Y)+1)

    for i in range(1, len(X)+1):
        for j in range(1, len(Y)+1):
            if X[i-1] == Y[j-1]:
                current[j] = 1 + previous[j-1]
            else:
                current[j] = max(previous[j], current[j-1])

        previous = current[:]

    return previous[len(Y)]
```

```
print("LCS Length (Space Optimized):",
      lcs_space_optimized(X, Y))
```

```
LCS Length (Space Optimized): 6
```

Post-Lab

1. What are the Time Space complexities of your Task2 implementation?
TC: O(mn) for filling DP table
SC: O(mn) for storing computed subproblem result

2. If there are multiple longest common subsequence of the same length, which one does your backtracking algorithm return?
Backtracking returns the LCS formed by chosen tie breaking rule, typically preferring upper cell over left when equal

3 Describe a real world application of LCS is used in version control system like Git to dect differences between file versions efficiently

4. How does the "diagonal" move in the DP table correspond to the characters in the strings?

A diagonal move means characters match
Include that character in LCS and move to subproblem (i-1, j-1)

5. Why is the first row and first column of the LCS DP table initialized to zero?

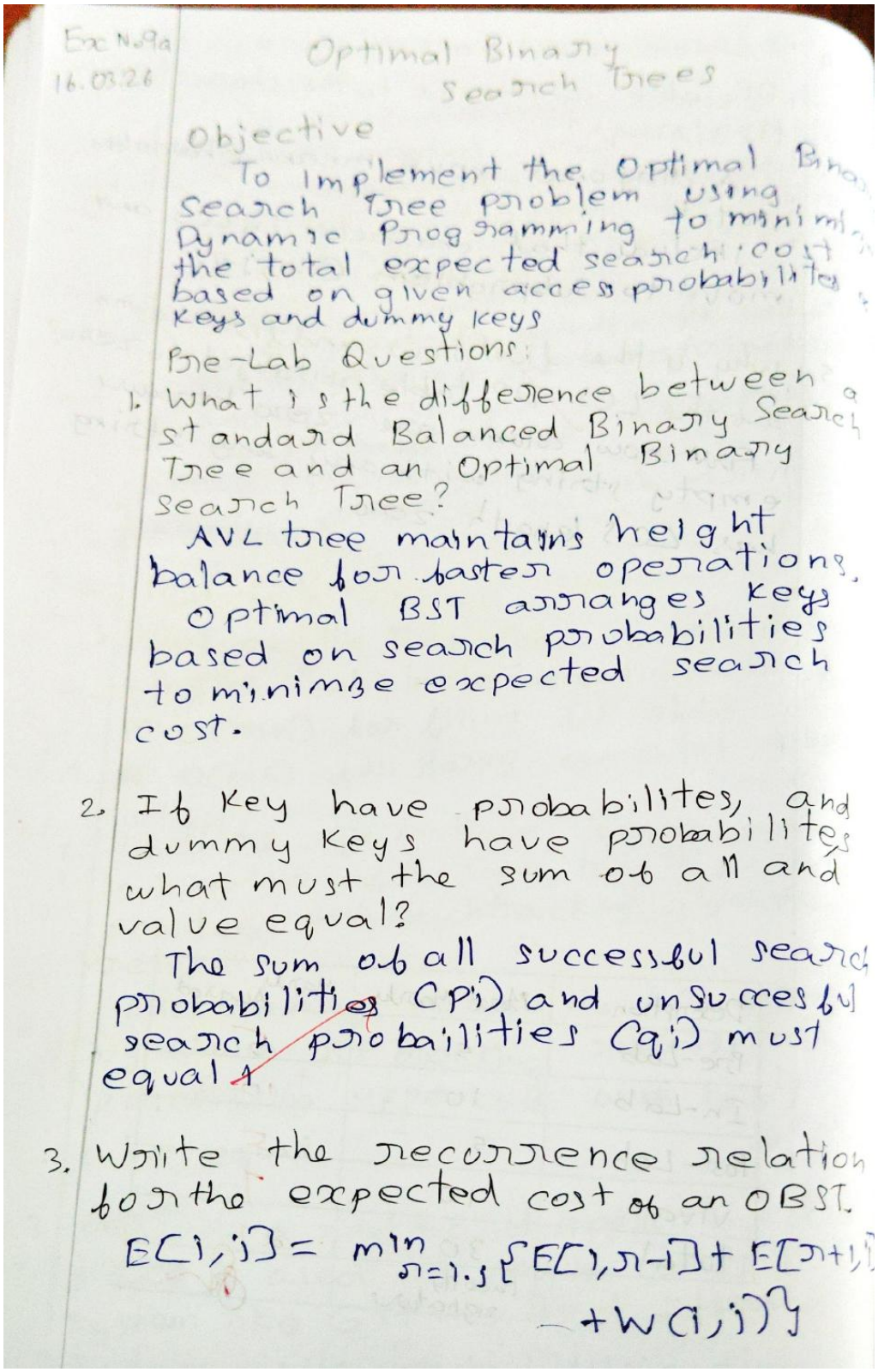
First row column are zero because empty string with any any string has LCS length zero.

Description	Max Marks	Marks Awarded
Pre-Lab	5	5
In-Lab	10	10
Post-Lab	5	5
viva	10	7
total	30	27
	Faculty signature	

EX NO: 09 **..16.03.2026**

OPTIMAL BINARY SEARCH TREE AND FLOYD-WARSHALL'S ALGORITHM

EX NO: 09a OPTIMAL BINARY SEARCH TREE



Ex No.9a
16.03.26

Optimal Binary Search Trees

Objective

To implement the Optimal Binary Search Tree problem using Dynamic Programming to minimize the total expected search cost based on given access probabilities of keys and dummy keys

Pre-Lab Questions:

1. What is the difference between a standard Balanced Binary Search Tree and an Optimal Binary Search Tree?

AVL tree maintains height balance for faster operations.

Optimal BST arranges keys based on search probabilities to minimize expected search cost.

2. If key have probabilites, and dummy keys have probabilites, what must the sum of all and value equal?

The sum of all successful search probabilities (pi) and unsuccesful search probabilities (qi) must equal 1

3. Write the recurrence relation for the expected cost of an OBST.

$$E[i,j] = \min_{r=i..j}\{E[i,r-1] + E[r+1,j] + w(i,j)\}$$

4. Define the "weight of a subtree" in the context of this problem.
Weight W(i,j) equals the sum of all successful probabilities $p_i \ldots p_j$ and unsuccessful probabilities $q_{i-1} \ldots q_j$

5. Why does the OBST algorithm have a time complexity of ? Can it be optimized?
Standard OBST uses dynamic programming with $O(n^3)$ time due to triple loops, optimization reduce it to $O(n^2)$.

Code:

```
!whoami
!echo 24BAD405 EX09a
```

```
nishanth\nishanth
24BAD405 EX09a
```

```
# Cell 1: Input data

# Keys must be in sorted order
keys = ['A', 'B', 'C', 'D']

# Successful search probabilities p1..pn
# Use index 1..n, so keep p[0] as dummy
p = [0, 0.15, 0.10, 0.05, 0.10]

# Unsuccessful search probabilities q0..qn
q = [0.05, 0.10, 0.05, 0.05, 0.10]

n = len(keys)

print("Keys:", keys)
print("Successful probabilities p:", p[1:])
print("Unsuccessful probabilities q:", q)
print("Number of keys =", n)
```

```
Keys: ['A', 'B', 'C', 'D']
Successful probabilities p: [0.15, 0.1, 0.05, 0.1]
Unsuccessful probabilities q: [0.05, 0.1, 0.05, 0.05, 0.1]
```

```python
# Cell 2: Create tables

# Weight table
w = [[0 for _ in range(n + 1)] for _ in range(n + 1)]

# Cost table
c = [[0 for _ in range(n + 1)] for _ in range(n + 1)]

# Root table
r = [[0 for _ in range(n + 1)] for _ in range(n + 1)]

print("Tables created successfully.")
```

```
Tables created successfully.
```

```python
# Cell 3: Initialize base cases

for i in range(n + 1):
    w[i][i] = q[i]
    c[i][i] = 0
    r[i][i] = 0

print("Base cases initialized.")
```

```
Base cases initialized.
```

```python
# Cell 3: Initialize base cases

for i in range(n + 1):
    w[i][i] = q[i]
    c[i][i] = 0
    r[i][i] = 0

print("Base cases initialized.")
```

```
Base cases initialized.
```

```python
# Cell 4: OBST Dynamic Programming

for length in range(1, n +  (variable) length: int
    for i in range(0, n - length + 1):
        j = i + length

        # Compute weight table
        w[i][j] = w[i][j - 1] + p[j] + q[j]

        # Initialize cost to infinity
        c[i][j] = float('inf')

        # Try every key as root
        for k in range(i + 1, j + 1):
            cost = c[i][k - 1] + c[k][j] + w[i][j]
```

```
                if cost < c[i][j]:
                    c[i][j] = cost
                    r[i][j] = k

    print("OBST tables computed successfully.")
```

```
OBST tables computed successfully.
```

```
    # Cell 5: Display tables neatly

    def print_table(table, name):
        print(f"\n{name} Table:")
        for row in table:
            for val in row:
                if isinstance(val, float):
                    print(f"{val:.2f}", end="\t")
                else:
                    print(val, end="\t")
            print()

    print_table(w, "Weight (w)")
    print_table(c, "Cost (c)")
    print_table(r, "Root (r)")
```

```
Weight (w) Table:
0.05    0.30    0.45    0.55    0.75
0       0.10    0.25    0.35    0.55
0       0       0.05    0.15    0.35
0       0       0       0.05    0.25
0       0       0       0       0.10

Cost (c) Table:
0       0.30    0.70    1.00    1.55
0       0       0.25    0.50    1.05
0       0       0       0.15    0.50
0       0       0       0       0.25
0       0       0       0       0

Root (r) Table:
0       1       1       2       2
0       0       2       2       2
0       0       0       3       4
0       0       0       0       4
0       0       0       0       0
```

```python
# Cell 6: Recursive function to display OBST structure

def print_obst(r, keys, i, j, parent=None, child_type="root"):
    if i == j:
        return

    root_index = r[i][j]
    current_key = keys[root_index - 1]

    if parent is None:
        print(f"{current_key} is the root")
    else:
        print(f"{current_key} is the {child_type} child of {parent}")

    # Left subtree
    print_obst(r, keys, i, root_index - 1, current_key, "left")

    # Right subtree
    print_obst(r, keys, root_index, j, current_key, "right")
```

```python
# Cell 7: Print final OBST structure

print("Optimal Binary Search Tree Structure:\n")
print_obst(r, keys, 0, n)
```

```
Optimal Binary Search Tree Structure:
```

```
Optimal Binary Search Tree Structure:

B is the root
A is the left child of B
D is the right child of B
C is the left child of D
```

```
# Cell 8: Final result

print(f"\nMinimum expected search cost of OBST = {c[0][n]:.2f}")
```

```
Minimum expected search cost of OBST = 1.55
```

Generate + Cod

Post-Lab Question

1. How does the "Principle of Optimally" apply to the construction of an OBST?
   Optimal subtree structure ensures globally optimal tree
   each subtree minimizes expected cost independently using dynamic programming.

2. If all keys have equal probabilities, will the OBST always be a complete binary tree? Explain
   Yes, equal probabilities lead to balanced splits, producing a nearly complete binary tree minimizing average search cost

3. What is the space complexity of this algorithm and can it be reduced?

Space complexity is O(n^2) for ... and root tables
slight reduction possible using optimized storage techniques.

4. How do the dummy key represent the range of values not present in the actual key set?

Dummy keys represent unsuccessful searches between real key, covering value gaps and contributing to expected search cost

5. Compare the search performance of OBST versus a simple BST for a ... skewed probability distribution

OBST minimizes weighted search cost by placing frequent keys near root, outperforming simple BST in skewed distribution.

EX NO: 09b ALL PAIR SHORTEST PATHS FLOYD WARSHALL ALGORITHM

All Pairs Shortest Paths:
Floyd-Warshall Algorithm

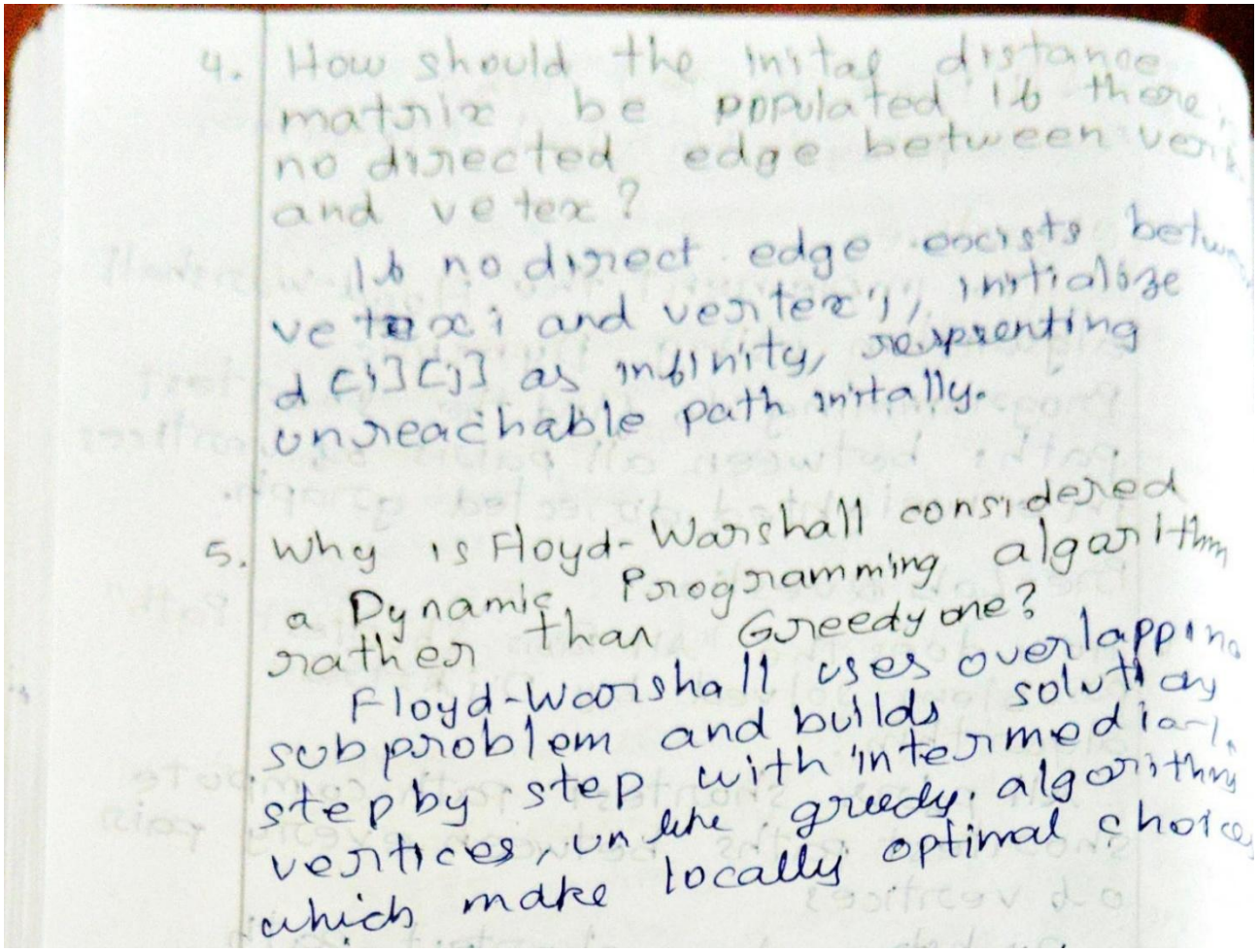
Objective:
To implement the Flogd-Warshall algorithm using Dynamic Brogramming to find the shortest paths between all pairs of vertices in a weighted directed graph.

Pre-Lab Questions

1. How does the "All Pairs Shortest Path" problem solved by Dijkstra's algorithm?
All pairs shortest path compute shortest paths between every pair of vertices
Dijkstra solves shortest path from one source to all vertices.

2. What happens if a graph contains a negative weight cycle? Can Floyd Warshall detect it?
Negative weight cycles cause indefinitely decreasing path costs.
Floyd-Warshall detects them when a diagonal element in the distance matrix becomes negatives

3. In the distance matrix, what does the super script signify?
The superscript k indicates shortest path costs considering only vertices 1 through k as intermediate vertics during dynamic programming iterations



4. How should the initial distance matrix be populated if there is no directed edge between vertex and vertex?

If no direct edge exists between vertex i and vertex j, initialize d[i][j] as infinity, representing unreachable path initially.

5. Why is Floyd-Warshall considered a Dynamic Programming algorithm rather than Greedy one?

Floyd-Warshall uses overlapping subproblem and builds solution step by step with intermediate vertices, unlike greedy algorithms which make locally optimal choices

Code:

```
!whoami
!echo 24BAD405 EX09b
```

```
nishanth\nishanth
24BAD405 EX09b
```

```
# Cell 1: Graph representation using adjacency matrix

INF = 9999

# Example graph with 4 vertices: 0, 1, 2, 3
graph = [
    [0,    3,    INF,  7],
    [8,    0,    2,    INF],
    [5,    INF,  0,    1],
    [2,    INF,  INF,  0]
]

n = len(graph)

print("Adjacency Matrix:")
for row in graph:
    print(row)
```

```
Adjacency Matrix:
[0, 3, 9999, 7]
[8, 0, 2, 9999]
[5, 9999, 0, 1]
```

```
[2, 9999, 9999, 0]
```

```
    # Cell 2: Initialize distance matrix and next matrix

    # Distance matrix starts as the graph itself
    dist = [row[:] for row in graph]

    # Next matrix for path reconstruction
    next_vertex = [[None for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if graph[i][j] != INF and i != j:
                next_vertex[i][j] = j

    print("Distance and path matrices initialized.")
```

```
Distance and path matrices initialized.
```

```python
# Cell 3: Floyd-Warshall Algorithm

for k in range(n):          # intermediate vertex
    for i in range(n):      # source
        for j in range(n):  # destination
            if dist[i][k] != INF and dist[k][j] != INF:
                if dist[i][j] > dist[i][k] + dist[k][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
                    next_vertex[i][j] = next_vertex[i][k]

print("Floyd-Warshall computation completed.")
```

```
Floyd-Warshall computation completed.
```

Generate

```python
# Cell 4: Print shortest distance matrix

print("Shortest Distance Matrix:")
for row in dist:
    print(row)
```

```
Shortest Distance Matrix:
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]
```

```python
# Cell 5: Print next vertex matrix

print("Next Vertex Matrix:")
for row in next_vertex:
    print(row)
```

```
Next Vertex Matrix:
[None, 1, 1, 1]
[2, None, 2, 2]
[3, 3, None, 3]
[0, 0, 0, None]
```

```python
# Cell 6: Function to print shortest path from i to j

def print_path(i, j):
    if next_vertex[i][j] is None:
        print(f"No path exists from {i} to {j}")
        return

    path = [i]
    while i ≠ j:
        i = next_vertex[i][j]
        path.append(i)

    print("Shortest path:", " → ".join(map(str, path)))
```

```
    # Cell 7: Example path queries

    print_path(0, 3)
    print_path(3, 1)
    print_path(1, 0)
```

```
Shortest path: 0 → 1 → 2 → 3
Shortest path: 3 → 0 → 1
Shortest path: 1 → 2 → 3 → 0
```

```
    # Cell 8: Check for negative weight cycles

    has_negative_cycle = False

    for i in range(n):
        if dist[i][i] < 0:
            has_negative_cycle = True
            break

    if has_negative_cycle:
        print("Graph contains a NEGATIVE WEIGHT CYCLE.")
    else:
        print("No negative weight cycle detected.")
```

```
No negative weight cycle detected.
```

```python
# Cell 9: Final summary

print("\n=== FINAL RESULT ===")

print("\nShortest Distance Matrix:")
for row in dist:
    print(row)

print("\nExample Shortest Paths:")
print_path(0, 3)
print_path(3, 1)

print("\nNegative Cycle Check:")
if has_negative_cycle:
    print("Graph contains a NEGATIVE WEIGHT CYCLE.")
else:
    print("No negative weight cycle detected.")
```

```
=== FINAL RESULT ===

Shortest Distance Matrix:
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]
```

```
=== FINAL RESULT ===

Shortest Distance Matrix:
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]

Example Shortest Paths:
Shortest path: 0 → 1 → 2 → 3
Shortest path: 3 → 0 → 1

Negative Cycle Check:
No negative weight cycle detected.
```

Post-Lab Questions

1. What are the Time and space complexities of the Floyd-Warshall algorithm?
   Time complexity is $O(n^3)$ due to triple nested loops
   space complexity is $O(n^2)$ for distance matrix

2. If you ran Dijkstra's algorithm n times how does it compare for dense graphs?
   Repeating Dijkstra give $O(n^2)$ with matrix
   similar to Floyd-Warshall, but with higher constants and implementation overhead.

weight cycle using the diagonal elements?
If any diagonal element becomes negative
it indicates a negative weight cycle reachable from the vertex

4. Why is the loop order (k, i, j) critical for correctness?
Order ensures intermediate vertices are considered progressively, maintaing correctness of dynamic programming state transitions

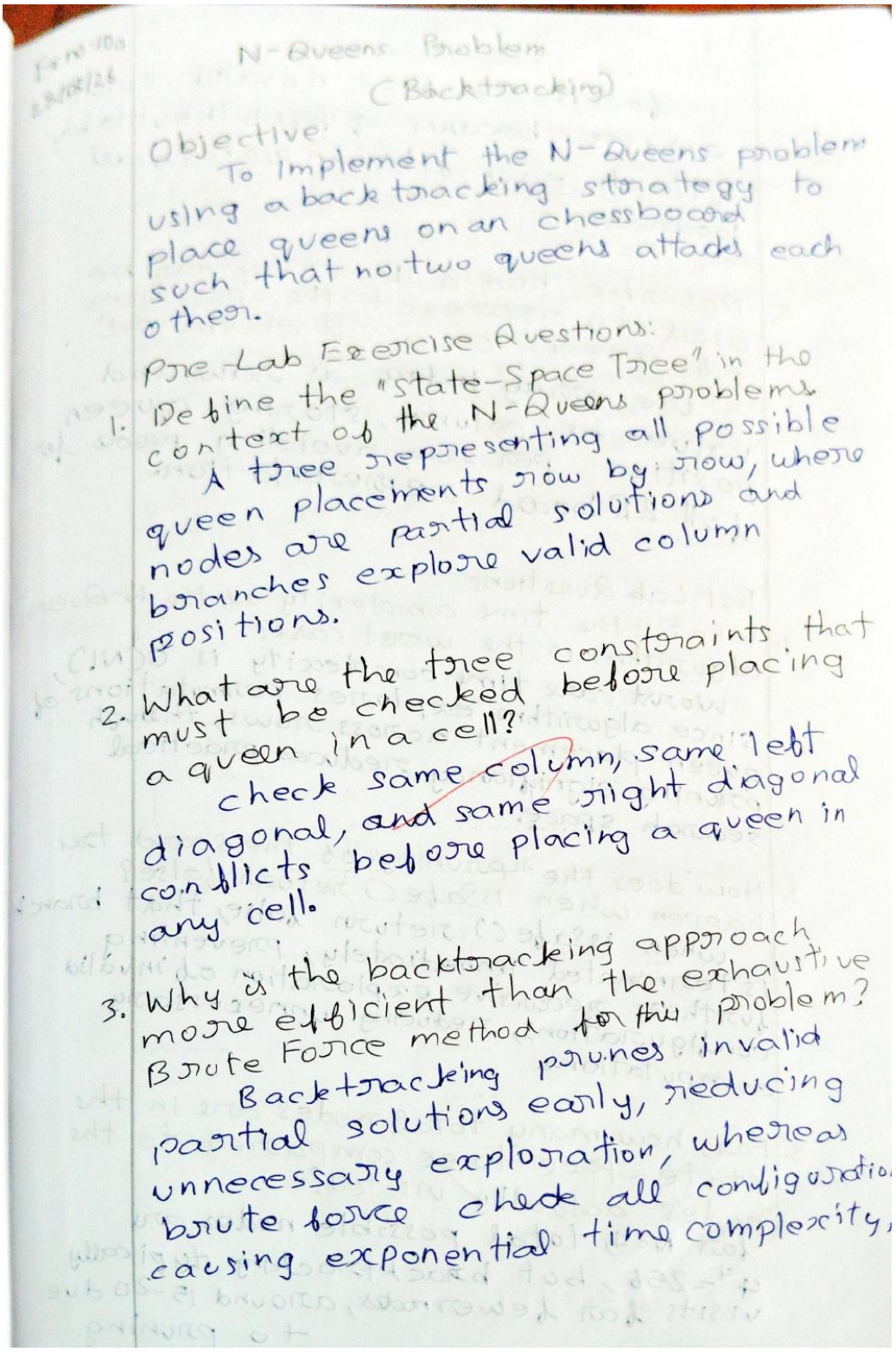
5. Give a real-world scenario for all-pairs shortest path usage:
Network routing tables, traffic system, or airline route planning, require shortest path between every pair of locations.

Description	Max Marks	Marks Allocated
Pre Lab	5	5
In Lab	10	10
Post Lab	5	5
viva	10	10
total	30	30
	Faculty Signature	

EX NO: 10 **….23.03.2026**

N QUEENS, HAMILTANION CIRCUIT, MAP-COLOURING AND TRAVELLING SALESMAN PROBLEM

EX NO: 10a BACKTRACKING: N-QUEENS PROBLEM



N-Queens Problem
(Backtracking)

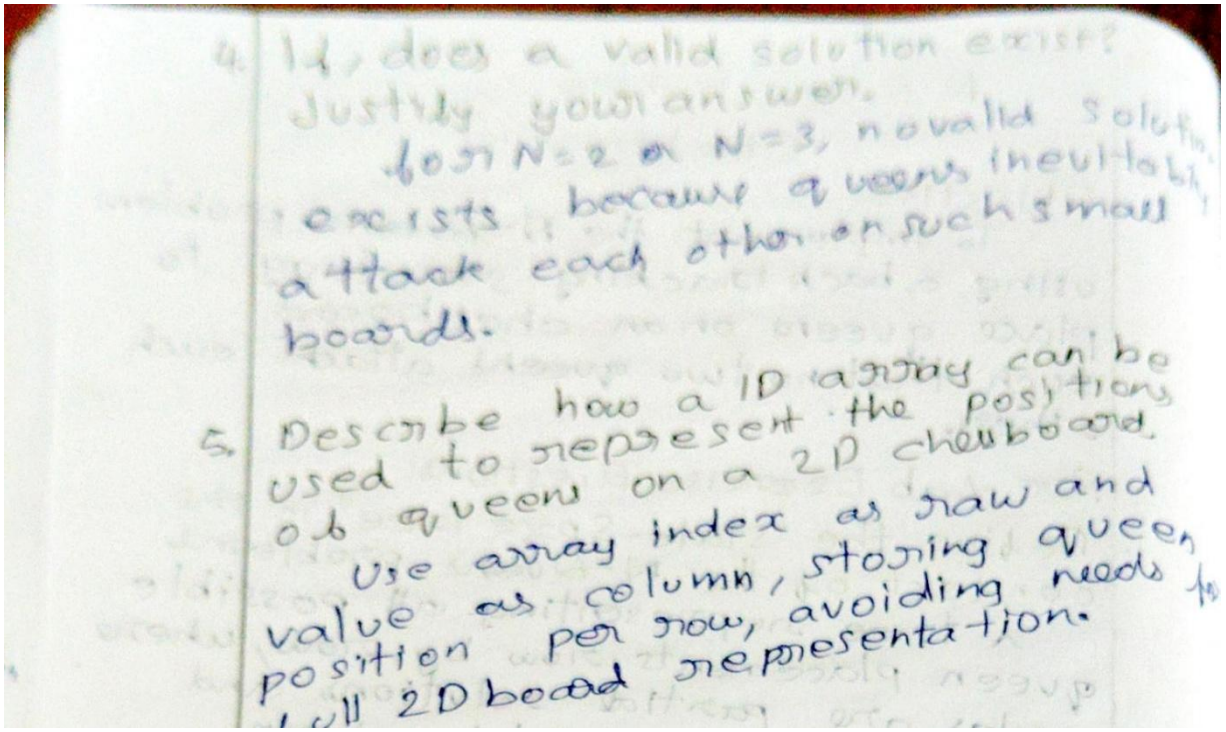
Objective:
To Implement the N-Queens problem using a backtracking strategy to place queens on an chessboard such that no two queens attacks each other.

Pre-Lab Exercise Questions:

1. Define the "State-Space Tree" in the context of the N-Queens problem.
A tree representing all possible queen placements row by row, where nodes are partial solutions and branches explore valid column positions.

2. What are the tree constraints that must be checked before placing a queen in a cell?
check same column, same left diagonal, and same right diagonal conflicts before placing a queen in any cell.

3. Why is the backtracking approach more efficient than the exhaustive Brute Force method for this problem?
Backtracking prunes invalid partial solutions early, reducing unnecessary exploration, whereas brute force check all configuratio causing exponential time complexity,


4. If, does a valid solution exist? Justify your answer.
for N=2 & N=3, no valid solution exists because queens inevitably attack each other on such small boards.

5. Describe how a 1D array can be used to represent the positions of queens on a 2D chessboard.
Use array index as row and value as column, storing queen position per row, avoiding needs for full 2D board representation.

Code:

```
!whoami
!echo 24BAD405 EX10a
```

```
nishanth\nishanth
24BAD405 EX10a
```

```python
def isSafe(board, row, col, n):

    for i in range(col):
        if board[row][i] == 1:
            return False

    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    for i, j in zip(range(row, n, 1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False

    return True
```

```python
def solveNQueensUtil(board, col, n, solutions):

    if col == n:
        solutions.append([row[:] for row in board])
        return

    for row in range(n):
        if isSafe(board, row, col, n):
            board[row][col] = 1

            solveNQueensUtil(board, col + 1, n, solutions)

            board[row][col] = 0
```

```python
def solveNQueens(n, find_all=False):

    board = [[0 for _ in range(n)] for _ in range(n)]
    solutions = []

    solveNQueensUtil(board, 0, n, solutions)

    if not find_all and solutions:
        return [solutions[0]]
    return solutions
```

```python
def printBoard(board):
    for row in board:
        print(" ".join("Q" if cell == 1 else "." for cell in row))
    print()
```

```python
def main():
    print("Task 2: First solution for N = 4")
    sol = solveNQueens(4, find_all=False)
    if sol:
        printBoard(sol[0])
    else:
        print("No solution")
```

```python
def main():
    print("Task 2: First solution for N = 4")
    sol = solveNQueens(4, find_all=False)
    if sol:
        printBoard(sol[0])
    else:
        print("No solution")

    print("\nTask 3: All solutions and count for N = 4")
    all_solutions = solveNQueens(4, find_all=True)
    print(f"Total number of distinct solutions = {len(all_solutions)}\n")
```

```
Task 2: First solution for N = 4
. . Q .
Q . . .
. . . Q
. Q . .


Task 3: All solutions and count for N = 4
Total number of distinct solutions = 2

Solution #1:
. . Q .
Q . . .
. . . Q
. Q . .

Solution #2:
. Q . .
. . . Q
Q . . .
. . Q .
```

Post Lab Questions

1. What is the time complexity of the N-Queens algorithm in the worst case?

Worst case time complexity is O(N!), since algorithm explores permutations of queen placement across rows, though pruning significiancy reduces practical search space.

2. How does the "pruning" of the search tree happen when isSafe() return false?

when isSafe() return false, that branch is terminated immediately, preventing further recursive exploration of invalid configurations, reducing unnecessary computations.

3. For, how many total nodes are in the state-space tree compared to the nodes actually visited?

for N=4, total possible nodes are $4^4 = 256$, but backtracking typically visits far fewer nodes, around 15-20 due to pruning.

4. Explain how using a bitmask could potentially speed up the conflict-checking process.

Bitmasking uses binary operations to track occupied columns and diagonals, enabling constant-time conflict checks instead of scanning arrays, improving performance significantly.

5. Name a real-world application that uses logic similar to the N-Queens constraint satisfaction.

task scheduling and resource allocation problems use similar constraint satisfaction logic, ensuring no conflicts while assigning limited resource across competing tasks

EX NO: 10b BACKTRACKING: HAMILTONIAN CIRCUIT PROBLEM

Ex no 10b
23/03/26

Hamiltonian Circuit Problem (Backtracking)

Objective:
To implement a backtracking algo... to find a Hamiltonian circuit in a giv... undirected graph, which visits every vertex exactly once and returns to the starting vertex.

Pre-Lab Questions

1. Differentiable between a Hamiltonian Path and Hamiltonian Circuit.
Hamiltonian Path visits every vertex once Hamiltonian Circuit additionaly return to starting vertex, forming a complete cycle

2. What is the necessary condition for a graph to potentially contain a Hamilton circuit regarding its vertex degrees?
Each vertex must have degree at least otherwise, forming a cycle visting all ver... because impossible

3. How is the graph represented as input for this algorithm?
Gaph is typically represented using an adjancy matrix, where entry indicates edge existence between vertex pairs.

4. What is the base case for the recursive funct... in the Hamiltonian Circuit algorithm?
Base case occurs when all vertices are included and last vertex connects connec... back to starting vertex, forming a valid circuit.

5. Is the Hamiltonian Circuit problem classified as P, NP or NP-complete?
Hamiltonian Circuits problem is NP-Com... meaning no known polynomial-time solution exists, but solutions can be verified

Code:

```
!whoami
!echo 24BAD405 EX10b
```

```
nishanth\nishanth
24BAD405 EX10b
```

```
def isSafe(v, graph, path, pos):

    if graph[path[pos-1]][v] == 0:
        return False

    for i in range(pos):
        if path[i] == v:
            return False

    return True
```

```
def printHamiltonianCircuit(path, n):
    print("Hamiltonian Circuit found:")
    for i in range(n):
        print(path[i], end=" → ")
    print(path[0])
    print()
```

```python
def hamCycleUtil(graph, path, pos, n, found):

    if pos == n:
        if graph[path[pos-1]][path[0]] == 1:
            printHamiltonianCircuit(path, n)
            found[0] = True
            return True
        return False


    for v in range(1, n):
        if isSafe(v, graph, path, pos):
            path[pos] = v

            if hamCycleUtil(graph, path, pos + 1, n, found):
                return True

            path[pos] = -1

    return False
```

Generate

```python
def hasHamiltonianCycle(graph, n):
    path = [-1] * n
    path[0] = 0

    found = [False]
```

```
    found = [False]

    if hamCycleUtil(graph, path, 1, n, found):
        return True

    print("No Hamiltonian Circuit exists in this graph.")
    return False


def test_hamiltonian_circuit():
    print("=== TEST 1: Cycle Graph (should have Hamiltonian Circuit) ===\n")


    n1 = 5
    graph_cycle = [
        [0, 1, 0, 0, 1],
        [1, 0, 1, 0, 0],
        [0, 1, 0, 1, 0],
        [0, 0, 1, 0, 1],
        [1, 0, 0, 1, 0]
    ]

    print("Graph (adjacency matrix):")
    for row in graph_cycle:
        print(row)
    print()

    hasHamiltonianCycle(graph_cycle, n1)
```

```
    print("\n=== TEST 2: Star Graph (should NOT have Hamiltonian Circuit) ===\n")

    n2 = 5
    graph_star = [
        [0, 1, 1, 1, 1],
        [1, 0, 0, 0, 0],
        [1, 0, 0, 0, 0],
        [1, 0, 0, 0, 0],
        [1, 0, 0, 0, 0]
    ]

    print("Graph (adjacency matrix):")
    for row in graph_star:
        print(row)
    print()

    hasHamiltonianCycle(graph_star, n2)
```

Generate + Code + Markdown

Start Chat to Generate Code (Ctrl+I)

```
if __name__ == "__main__":
    test_hamiltonian_circuit()
```

```
=== TEST 1: Cycle Graph (should have Hamiltonian Circuit) ===

Graph (adjacency matrix):
[0, 1, 0, 0, 1]
[1, 0, 1, 0, 0]
[0, 1, 0, 1, 0]
[0, 0, 1, 0, 1]
[1, 0, 0, 1, 0]

Hamiltonian Circuit found:
0 → 1 → 2 → 3 → 4 → 0


=== TEST 2: Star Graph (should NOT have Hamiltonian Circuit) ===

Graph (adjacency matrix):
[0, 1, 1, 1, 1]
[1, 0, 0, 0, 0]
[1, 0, 0, 0, 0]
[1, 0, 0, 0, 0]
[1, 0, 0, 0, 0]

No Hamiltonian Circuit exists in this graph.
```

1. What happens to the execution time as the number of edges (density) in the graph increases?
As edge density increase, execution may improve since more connectivity reduces dead ends, but excessive edges increase branching possibilities and search overhead.

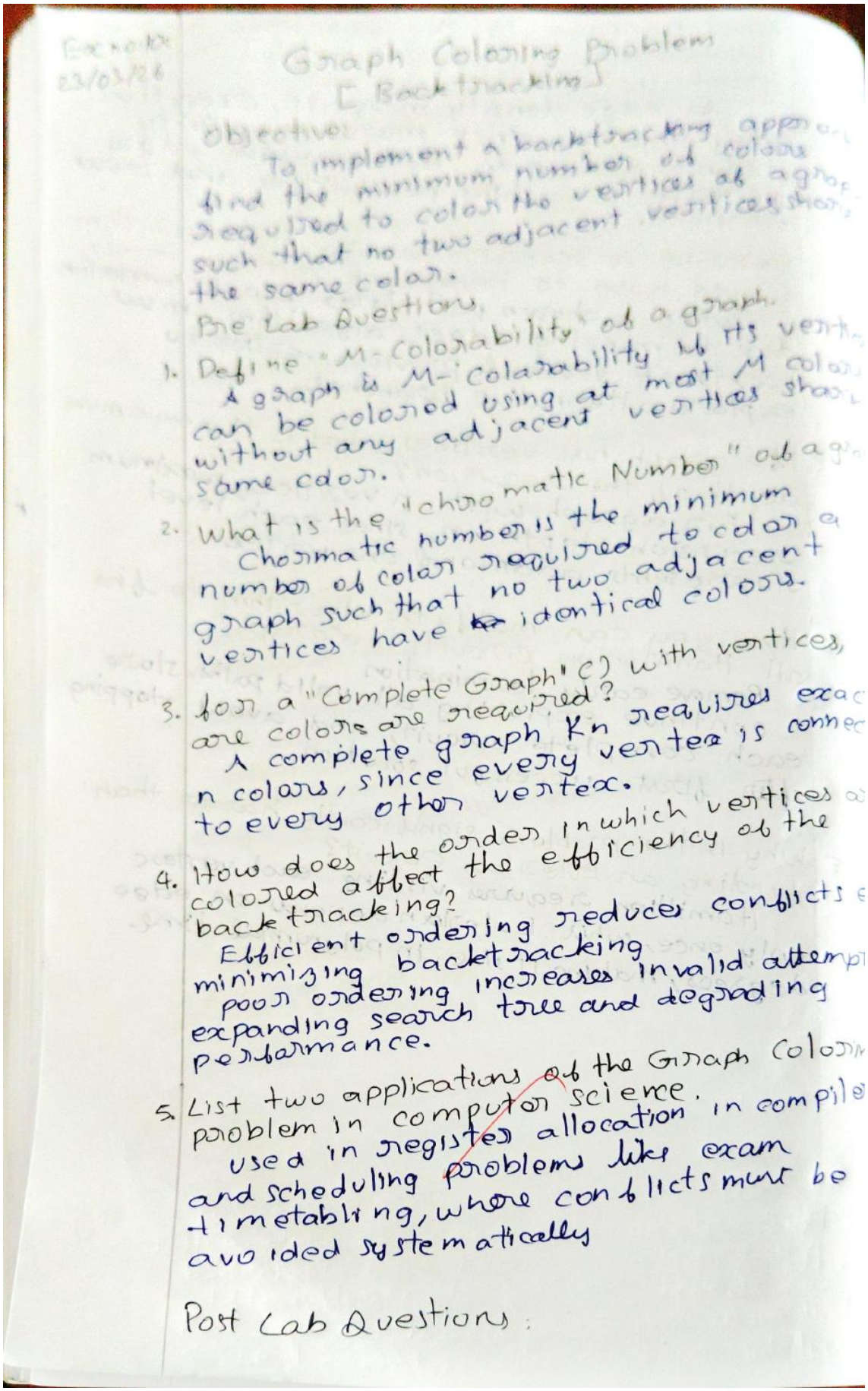
2. Describe a scenario where the algorithm would have to backtrack multiple levels, when a chosen path later blocks completion due to missing edges, algorithm must backtrack several previous vertices to explore alternative branches.

3. If a graph has vertices, what is the maximum depth of the recursion?
For a graph with n vertices, maximum recursion depth is n, since each level represents additional one vertex.

4. How you can modify the algorithm to find all Hamitonion circuits in a graph?
Remove early termination continue exploring all valid paths, store each complete circuit, and avoid stopping after first successful solution.

5. Why is the problem significantly harder than finding an Eulerian circuit?
Hamiltion requires visiting each vertex only once, while Eulerian depends on edge degrees, making it slow in polynomial time.

EX NO: 10c BACKTRACKING: GRAPH COLORING PROBLEM



Ex no:10c
23/03/26

Graph Coloring Problem
[Backtracking]

Objective:
To implement a backtracking approach to find the minimum number of colours required to color the vertices of a graph such that no two adjacent vertices share the same color.

Pre Lab Questions:

1. Define "M-Colorability" of a graph.
A graph is M-Colorability if its vertices can be colored using at most M colors without any adjacent vertices sharing same color.

2. What is the "chromatic Number" of a graph?
Chormatic number is the minimum number of color required to color a graph such that no two adjacent vertices have identical colors.

3. for a "Complete Graph" (K) with vertices, are colors are required?
A complete graph Kn requires exactly n colors, since every vertex is connected to every other vertex.

4. How does the order in which vertices are colored affect the efficiency of the backtracking?
Efficient ordering reduces conflicts & minimizing backtracking
poor ordering increases invalid attempts expanding search tree and degrading performance.

5. List two applications of the Graph Coloring problem in computer science.
used in register allocation in compiler and scheduling problems like exam timetabling, where conflicts must be avoided systematically

Post Lab Questions:

Code:

```
!whoami
!echo 24BAD405 EX10c
```

```
nishanth\nishanth
24BAD405 EX10c
```

```
def isSafe(v, color, c, graph, n):
    for i in range(n):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True
```

```
def graphColoringUtil(graph, m, color, v, n):
    if v == n:
        return True

    for c in range(1, m+1):
        if isSafe(v, color, c, graph, n):
            color[v] = c
            if graphColoringUtil(graph, m, color, v+1, n):
                return True
            color[v] = 0

    return False
```

```python
def findChromaticNumber(graph, n):
    color = [0] * n

    for m in range(1, n+1):
        for i in range(n):
            color[i] = 0

        if graphColoringUtil(graph, m, color, 0, n):
            return m, color

    return n, color
```

Generate + Code +

```python
def printColoring(color, chromatic_number):
    print(f"Chromatic number (minimum colors needed): {chromatic_number}")
    print("Vertex coloring:", color)
```

```python
if __name__ == "__main__":
    n1 = 5
    graph1 = [
        [0,1,0,0,1],
        [1,0,1,0,0],
        [0,1,0,1,0],
        [0,0,1,0,1],
        [1,0,0,1,0]
    ]

    print("Graph 1 (C5):")
    chrom_num1, coloring1 = findChromaticNumber(graph1, n1)
    printColoring(coloring1, chrom_num1)
    print()

    n2 = 4
    graph2 = [
        [0,1,1,1],
        [1,0,1,1],
        [1,1,0,1],
        [1,1,1,0]
    ]

    print("Graph 2 (K4):")
    chrom_num2, coloring2 = findChromaticNumber(graph2, n2)
    printColoring(coloring2, chrom_num2)
    print()
```

```
    print("Graph 2 (K4):")
    chrom_num2, coloring2 = findChromaticNumber(graph2, n2)
    printColoring(coloring2, chrom_num2)
    print()

    n3 = 6
    graph3 = [
        [0,1,1,0,0,0],
        [1,0,0,1,0,0],
        [1,0,0,1,0,0],
        [0,1,1,0,1,1],
        [0,0,0,1,0,1],
        [0,0,0,1,1,0]
    ]

    print("Graph 3 (Bipartite):")
    chrom_num3, coloring3 = findChromaticNumber(graph3, n3)
    printColoring(coloring3, chrom_num3)
```

```
Graph 1 (C5):
Chromatic number (minimum colors needed): 3
Vertex coloring: [1, 2, 1, 2, 3]

Graph 2 (K4):
Chromatic number (minimum colors needed): 4
Vertex coloring: [1, 2, 3, 4]

Graph 3 (Bipartite):
Chromatic number (minimum colors needed): 3
Vertex coloring: [1, 2, 2, 1, 2, 3]
```

1. What is the time complexity of the m-coloring backtracking algorithm?
Worst case time complexity is $O(m^n)$, since each vertex may take any of m colors, exploring all combinations

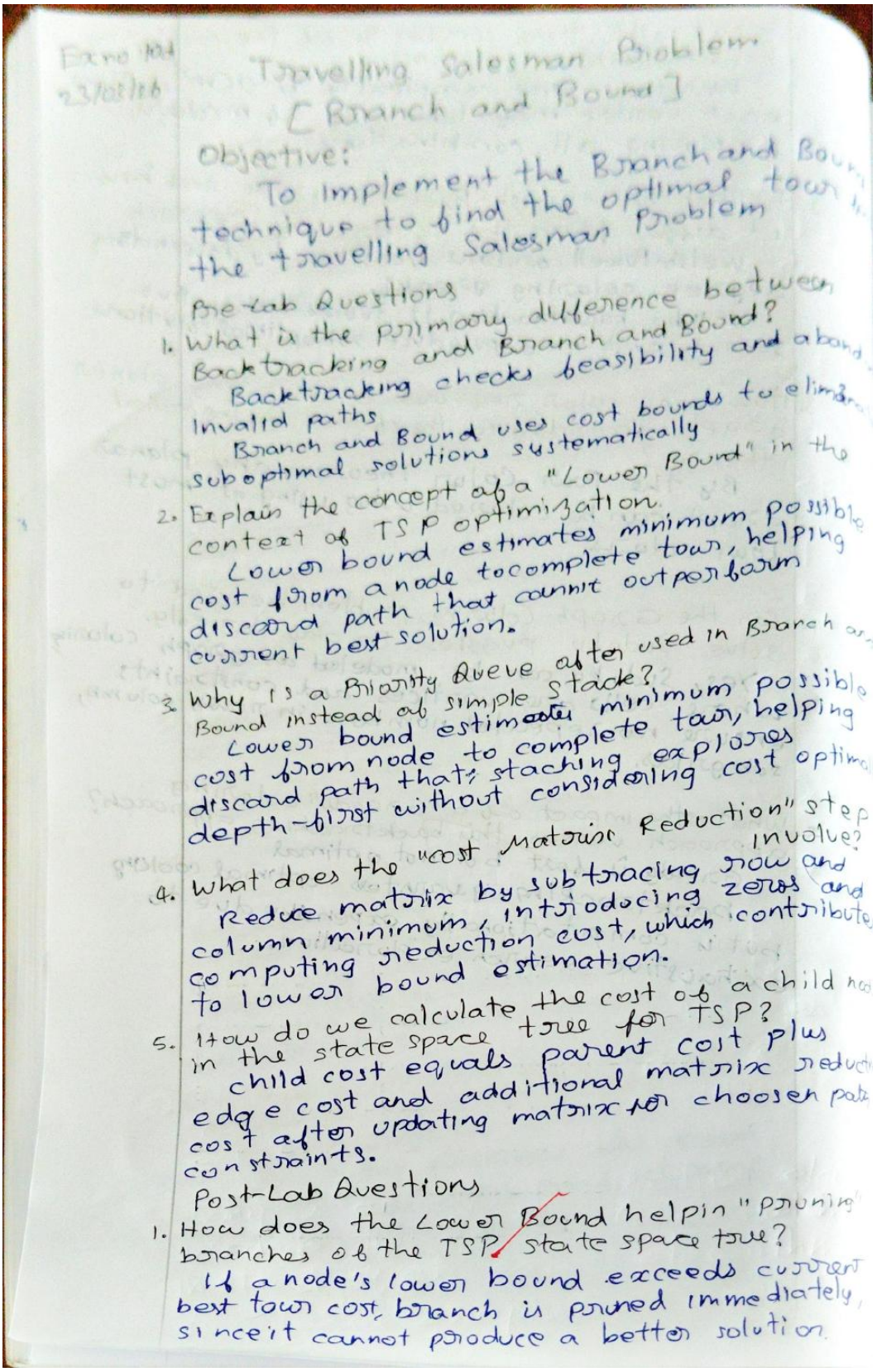
2. Explain the "Welsh-Powell" heuristic and how it differs from the backtracking approach.
Welsh-Powell orders vertices by descending degree coloring greedily
unlike backtracking, it avoids exhaustive search and may produce suboptimal solutions.

3. How many color required to color any planer graph according to the famous mathematical theorm?
By the Four Color Theorem, any planar graph can be colored using using at most four colors.

4. Can the Graph Coloring problem be used to solve Sudoku puzzles? Explain briefly.
Yes, Sudoku can be modeled as graph coloring where cells are vertices and constraints ensure no repeated numbers in rows, column, subgrids.

5. What is the impact of a "Greedy" coloring approach versus this backtracking approach?
Greedy is fast but not optimal
backtracking guarantees optimal coolring but is computationally expensive due to exhaustive search exploration.

EX NO: 10d BRANCH AND BOUND: TRAVELLING SALESMAN PROBLEM



Exno 10d
23/08/25

Travelling Salesman Problem
[Branch and Bound]

Objective:

To implement the Branch and Bound technique to find the optimal tour in the travelling Salesman problem

Pre-Lab Questions

1. What is the primary difference between Backtracking and Branch and Bound?

Backtracking checks feasibility and abandons invalid paths

Branch and Bound uses cost bounds to eliminate suboptimal solutions systematically

2. Explain the concept of a "Lower Bound" in the context of TSP optimization.

Lower bound estimates minimum possible cost from a node to complete tour, helping discard path that cannot outperform current best solution.

3. Why is a Priority Queue often used in Branch and Bound instead of simple stack?

Lower bound estimates minimum possible cost from node to complete tour, helping discard path that; stacking explores depth-first without considering cost optimal

4. What does the "cost matrix Reduction" step involve?

Reduce matrix by subtracing row and column minimum, introducing zeros and computing reduction cost, which contributes to lower bound estimation.

5. How do we calculate the cost of a child node in the state space tree for TSP?

child cost equals parent cost plus edge cost and additional matrix reduction cost after updating matrix for choosen path constraints.

Post-Lab Questions

1. How does the Lower Bound helpin "pruning" branches of the TSP state space tree?

If a node's lower bound exceeds current best tour cost, branch is pruned immediately, since it cannot produce a better solution.

Code:

```
!whoami
!echo 24BAD405 EX10d
```

```
nishanth\nishanth
24BAD405 EX10d
```

```
import heapq

INF = float('inf')

class Node:
    def __init__(self):
        self.matrix = None
        self.cost = 0
        self.vertex = 0
        self.level = 0
        self.path = []

def reduce_matrix(mat):
    n = len(mat)
    reduction = 0
    for i in range(n):
        rowmin = min(mat[i])
        if rowmin != INF and rowmin > 0:
            reduction += rowmin
            for j in range(n):
                if mat[i][j] != INF:
                    mat[i][j] -= rowmin
```

```
    for j in range(n):
        colmin = min(mat[i][j] for i in range(n))
        if colmin != INF and colmin > 0:
            reduction += colmin
            for i in range(n):
                if mat[i][j] != INF:
                    mat[i][j] -= colmin
    return reduction
```

```
def create_child(parent, from_city, to_city):
    n = len(parent.matrix)
    child = Node()
    child.matrix = [row[:] for row in parent.matrix]
    for j in range(n):
        child.matrix[from_city][j] = INF
    for i in range(n):
        child.matrix[i][to_city] = INF
    edge_cost = parent.matrix[from_city][to_city]
    child.cost = parent.cost + edge_cost
    reduction = reduce_matrix(child.matrix)
    child.cost += reduction
    child.vertex = to_city
    child.level = parent.level + 1
    child.path = parent.path[:]
    child.path.append(to_city)
    return child
```

```
def solve_tsp(cost_matrix):
    n = len(cost_matrix)
    root = Node()
    root.matrix = [row[:] for row in cost_matrix]
    root.vertex = 0
    root.level = 0
    root.path = [0]
    root.cost = reduce_matrix(root.matrix)
    pq = []
    counter = 0
    heapq.heappush(pq, (root.cost, counter, root))
    counter += 1
    best_cost = INF
    best_path = []
    while pq:
        cur_cost, _, cur = heapq.heappop(pq)
        if cur.cost ⩾ best_cost:
            continue
        if cur.level == n - 1:
            final_edge = cur.matrix[cur.vertex][0]
            if final_edge == INF:
                continue
            total_cost = cur.cost + final_edge
            if total_cost < best_cost:
                best_cost = total_cost
                best_path = cur.path[:] + [0]
            continue
```

```python
if __name__ == "__main__":
    graph1 = [
        [INF, 10, 15, 20],
        [10, INF, 35, 25],
        [15, 35, INF, 30],
        [20, 25, 30, INF]
    ]
    cost1, path1 = solve_tsp(graph1)
    print("Minimum cost:", int(cost1))
    print("Path:", " → ".join(map(str, path1)))

    graph2 = [
        [INF, 20, 30, 10, 11],
        [15, INF, 16, 4, 2],
        [3, 5, INF, 2, 4],
        [19, 6, 18, INF, 3],
        [16, 4, 7, 16, INF]
    ]
    cost2, path2 = solve_tsp(graph2)
    print("\nMinimum cost:", int(cost2))
    print("Path:", " → ".join(map(str, path2)))
```

```
Minimum cost: 80
Path: 0 → 1 → 3 → 2 → 0

Minimum cost: 28
Path: 0 → 3 → 1 → 4 → 2 → 0
```

Compare the performance of Branch and Bound with the Dynamic Programming approach (3) of TSP.

Branch and Bound prunes search tree heuristically

Dynamic Programming guarantees optimal solution in $O(n^2 2^n)$ time deterministically

3. What is the worst-case space complexity of this algorithm?

worst case space is exponential, $O(n!)$ as priority queue may store large number of partial tours indense search scenarios

4. Under what conditions does the Brach and Bound algorithm perform similarly to Brute Force?

When bounds are weak or similar across node, pruning is ineffective, forcing exploration of nearly all permutation like brute force

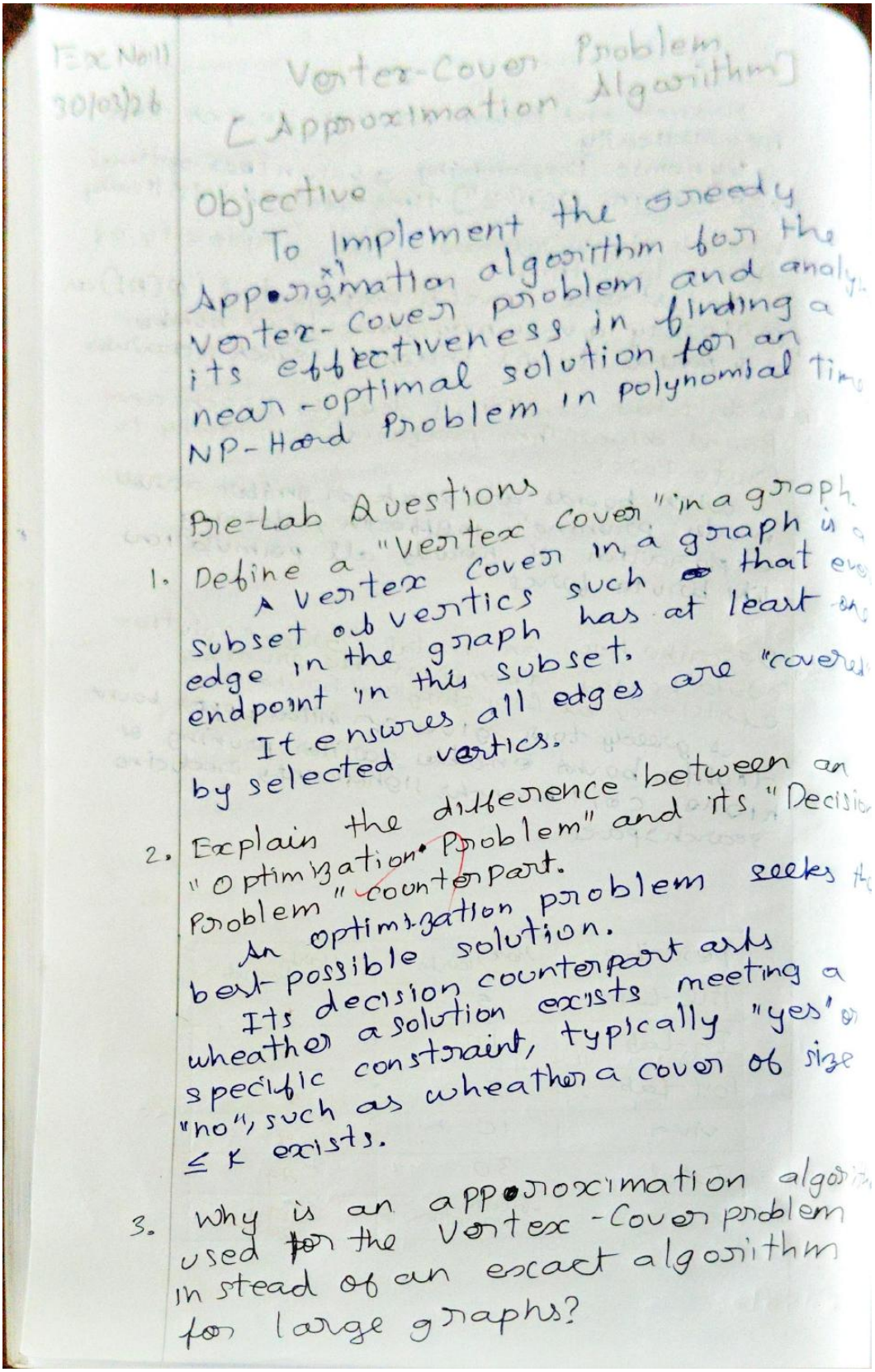
5. Describe how an initial "Greedy" solution could be used to improve the pruning efficiency of Branching and Bound

a greedy tour gives an initial upper bound tighter bound enables earlier pruning of higher cost branches significantly reducing search space

Describe	Max Marks	Mark awarded
Pre-Lab	5	5
In-Lab	10	10
Post-Lab	5	5
viva	10	9
Total	30	29
	faculty signature	24/3/26

EX NO: 11 **30.03.2026**

VERTEX COVER PROBLEM



Ex No:11
30/03/26

Vertex-Cover Problem
[Approximation Algorithm]

Objective

To implement the greedy Approximation algorithm for the Vertex-Cover problem and analyze its effectiveness in finding a near-optimal solution for an NP-Hard Problem in polynomial time.

Pre-Lab Questions

1. Define a "Vertex Cover" in a graph.

A Vertex Cover in a graph is a subset of vertices such that every edge in the graph has at least one endpoint in this subset.

It ensures all edges are "covered" by selected vertics.

2. Explain the difference between an "Optimization Problem" and its "Decision Problem" counterpart.

An optimization problem seeks the best possible solution.

Its decision counterpart asks wheather a solution exists meeting a specific constraint, typically "yes" or "no", such as wheather a cover of size $\leq k$ exists.

3. Why is an approximation algorithm used for the Vertex-Cover problem instead of an exact algorithm for large graphs?

The vertex Cover problem is NP-hard meaning exact solution are computationally expensive for large graph.
Approximation algorithm provide near-optimal solutions efficiently, making them practical for real world large-scale graphs where exact computation is infeasible.

4. What does it mean for an algorithm to be a "2-approximation" algorithm?
A 2-approximation algorithm guarantees the solution size is at most twice the size of the optimal minimum vertex cover for every input graph.

5. What is the relationship between a Vertex Cover and an Independent Set a ~~Gra~~ Graph?
In any graph, the complement of a vertex cover forms an Independent set, and the complement of an Independent set forms a vertex cover.

Code:

```
!whoami
!echo 24BAD405 EX11
```

```
nishanth\nishanth
24BAD405 EX11
```

```
class Graph:
    def __init__(self):
        self.adj = {}
        self.edges = {}
    def add_edge(self, u, v):
        # Add to adjacency list
        if u not in self.adj:
            self.adj[u] = []
        if v not in self.adj:
            self.adj[v] = []

        self.adj[u].append(v)
        self.adj[v].append(u)

        # Store edge as sorted tuple to avoid duplicates
        edge = tuple(sorted((u, v)))
        self.edges[edge] = False  # False = unvisited

    def degree(self):
        return {node: len(neighbors) for node, neighbors in self.adj.items()}
```

```
    def degree(self):
        return {node: len(neighbors) for node, neighbors in self.adj.items()}

    def mark_visited(self, u, v):
        edge = tuple(sorted((u, v)))
        if edge in self.edges:
            self.edges[edge] = True

    def is_visited(self, u, v):
        return self.edges.get(tuple(sorted((u, v))), False)

    def get_unvisited_edges(self):
        return [e for e, visited in self.edges.items() if not visited]
```

```
def vertex_cover_approx(graph):
    visited_edges = graph.edges
    cover = set()

    while True:
        unvisited = graph.get_unvisited_edges()
        if not unvisited:
            break

        # Pick arbitrary edge
        u, v = unvisited[0]
```

Task 3

Star

```
g = Graph()
for i in range(1, 6):
    g.add_edge(0, i)

approx_cover = vertex_cover_approx(g)
print("Approx Cover:", approx_cover)
print("Size:", len(approx_cover))
```

```
Approx Cover: {0, 1}
Size: 2
```

Cycle

Generate

Start Chat to Generate Code (Ct

```
g = Graph()
edges = [(0,1), (1,2), (2,3), (3,0)]
for u, v in edges:
    g.add_edge(u, v)

approx_cover = vertex_cover_approx(g)
print("Approx Cover:", approx_cover)
print("Size:", len(approx_cover))
```

```
    approx_cover = vertex_cover_approx(g)
    print("Approx Cover:", approx_cover)
    print("Size:", len(approx_cover))
```

```
Approx Cover: {0, 1, 2, 3}
Size: 4
```

Complete Kn Graph

```
    g = Graph()
    nodes = [0,1,2,3]

    for i in range(len(nodes)):
        for j in range(i+1, len(nodes)):
            g.add_edge(nodes[i], nodes[j])

    approx_cover = vertex_cover_approx(g)
    print("Approx Cover:", approx_cover)
    print("Size:", len(approx_cover))
```

```
Approx Cover: {0, 1, 2, 3}
Size: 4
```

Post-Lab Question

1. What is the time complexity of your greedy approximation implementation?

The greedy 2-approximation algorithm for vertex cover runs in O(E) time, where E is the number of edges. Each edge is processed once, and incident edges are removed efficiently

2. In Task 3, for the Star Graph, how does the size of the approximated

cover compare to the optimal
In star graph, the optimal cov
has size 1.
The approximation may includ
many leaf nodes as well, giving
cover size close to 2(n-1), whic
is much larger than optimal.

3. Why does the approximation algo
add both endpoints of an edge
instead instead of an edge
just picking the vertex with the
highest degree?
Adding both endpoints ens
that all edge are immediatel
covered and guarnatees the 2-
this guarantee.

4. Is the Vertex-Cover problem
NP-Complete or NP-Hard? Justi
your answer
The decision version of Vertex
Cover is NP-Complete because it i
in NP and can be reduced from othe
NP problems.
The optimization version is
NP-Hard since it seeks the minimum
cover

5. Describe a practical application
of the Vertex-Cover problem.
Vertex Cover is used in placing
minimum number of security came
so every hallway is monitored.
It also applies in netwo
monitoring, where nodes are selecte

to obtain all common marks.

Description	Max Marks	Marks Awarded
Pre-Lab	5	5
In-Lab	10	10
Post-Lab	5	5
Viva	10	8
total	30	28
	faculty signature	